



GIM6010-8 마이크로 모터 사용 설명서

목차

1	모터 사양	5
1.1	도면 및 치수	5
1.2	전기적 특성	6
1.3	기계적 특성	6
2	드라이버 정보	8
2.1	외관 및 3차원 치수	8
2.2	인터페이스 개요	9
2.3	사양	9
2.4	인터페이스 상세 정의	10
2.5	주요 부품 및 사양	14
3	테스트 및 조정 설명	15
3.1	시작 가이드	15
3.2	펌웨어 업데이트 다운로드	39
4	통신 프로토콜 및 예제	41
4.1	CAN SIMPLE 프로토콜	41
4.2	CANOPEN 프로토콜	54
4.3	RS485 프로토콜 (MODBUS RTU)	56
4.4	PWM 입력 제어	81
4.5	펄스/방향 제어	83
4.6	SDK	85
5	자주 묻는 질문 및 오류 코드 (업데이트 예정)	95
5.6	자주 묻는 질문(FAQ)	95
5.7	오류 코드	95

버전 기록

버전 번호	날짜	수정/설명
0.1	2023.12.5	odrivetool을 지원하는 첫 번째 버전
0.2	2023.12.15	명령어 목록 및 분류 추가
0.3	2023.12.19	Python, Arduino, ROS SDK 추가
0.4	2023.12.23	USB 및 CAN 호환성 전환 설명 추가
0.5	2023.12.29	호스트 컴퓨터 디버깅 실전 사례 추가
0.6	2024.1.2	CAN 프로토콜 및 Python 디버깅 실전 사례 추가
0.7	2024.1.8	CAN 인터페이스 120R 매칭 저항 스위치 옵션 추가
0.8	2024.2.15	두 번째 인코더 관련 내용 및 사용자 제로점 설정 추가
0.9	2024.2.23	수정 가능한 파라미터 및 호출 인터페이스 함수를 지원하는 CAN 명령어 추가
0.91	2024.3.12	국민 다운로드 소프트웨어의 드라이버 다운로드 주소 추가
0.92	2024.3.22	CAN 기본 보트레이트에 대한 설명 및 두 번째 인코더의 역할 추가 로터의 초기 위치 범위 설명.
1.0	2024.3.26	모터 온도 보호 기능 설명 추가
1.1	2024.5.20	시운전 및 튜닝 가이드 추가
1.2	2024.5.24	CAN MIT 프로토콜 오류 수정
1.3	2024.7.2	오류 코드 표 추가
1.4	2024.8.18	ID 수정 방법 안내 추가
1.5	2024.8.21	브레이크 저항기 사용 설명 추가
1.6	2024.8.31	다양한 버전의 endpoints.json 파일 다운로드 주소 추가
1.7	2024.9.5	CANOpen 프로토콜 설명 추가
1.8	2024.10.8	예외 유형 정리 및 인코더 예외 코드 목록 추가
1.9	2024.10.15	호스트 컴퓨터용 “모터 엘프”의 사용 설명서 추가
2.0	2024.12.6	Modbus RTU 프로토콜 설명 및 C/C++ SDK 추가
2.1	2025.1.23	Modbus 임의의 엔드포인트 액세스 설명 추가
2.2	2025.5.20	PWM 제어 및 펄스 방향 제어 설명 추가

주의 사항

1. 본 설명서의 작동 매개변수에 따라 사용하십시오. 그렇지 않을 경우 제품에 복구 불가능한 손상을 초래할 수 있습니다!
2. 모터 작동 중에는 드라이버가 손상되지 않도록 전원 과전류 및 과전압 보호 조치를 취하십시오.
3. 사용 전 각 부품의 상태를 확인하십시오. 부품이 누락되거나 손상된 경우, 즉시 기술 지원팀에 연락하십시오.
4. 드라이버에는 역접속 방지 기능이 없으므로, 전원을 연결하기 전에 2.4.1항을 참조하여 전원의 극성이 올바른지 확인하십시오.
5. 드라이버의 노출된 부분은 정전기로 인한 손상을 방지하기 위해 손으로 만지지 마십시오!

법적 고지

본 제품을 사용하기 전에 반드시 본 설명서를 주의 깊게 읽고, 내용에 따라 제품을 조작하십시오. 사용자가 설명서 내용을 위반하여 본 제품을 사용할 경우, 그로 인해 발생하는 어떠한 재산 손실이나 인명 피해 사고에 대해서도 당사는 어떠한 책임도 지지 않습니다. 본 제품은 수많은 부품으로 구성되어 있으므로, 사고 발생을 방지하기 위해 어린이가 본 제품에 접촉하지 않도록 하십시오. 제품의 수명을 연장하기 위해 고온, 고압 환경에서 본 제품을 사용하지 마십시오. 본 설명서는 인쇄 시 각 기능 소개 및 사용 방법을 최대한 포함하도록 노력했습니다. 그러나 제품 기능의 지속적인 개선 및 설계 변경 등으로 인해, 사용자가 구매한 제품과 차이가 있을 수 있습니다.

본 설명서와 실제 제품의 색상, 외관 등이 다소 차이가 있을 수 있으므로, 실제 제품을 기준으로 해 주십시오. 당사는 본 매뉴얼의 인쇄 오류나 부정확한 최신 정보를 수시로 수정 및 변경하거나, 프로그램 및/또는 장비를 개선할 수 있으며, 이 경우 별도의 통지 없이 진행될 수 있습니다. 이러한 변경 사항은 본 매뉴얼의 새 버전에 반영되므로, 기술 지원팀에 문의하여 확인하시기 바랍니다. 모든 그림은 기능 설명을 위한 참고용이며, 실제 제품을 기준으로 하십시오.

애프터서비스 정책

본 제품의 사후 서비스는 《중화인민공화국 소비자권익보호법》 및 《중화인민공화국 제품품질법》에 따라 엄격히 시행되며, 서비스 내용은 다음과 같습니다:

1. 보증 기간 및 내용
 - 1) 온라인 채널을 통해 본 제품을 주문 구매한 고객은 수령일 다음 날부터 7일 이내에 무통증 반품 서비스를 이용할 수 있습니다. 반품 시 고객은 유효한 구매 증빙 자료를 제시하고 세금계산서를 반환해야 합니다. 고객은 반품 상품이 원래의 품질과 기능을 유지하고, 외관이 양호하며, 상품 자체 및 부속품의 상표와 각종 표시가 완전하고 훼손되지 않았음을 보장해야 하며, 사은품이 있는 경우 함께 반환해야 합니다. 상품에 인위적인 손상, 인위적인 분해, 포장 상자 분실, 부속품 누락 등의 상황이 발견될 경우 반품이 불가합니다. 반품 시 발생하는 배송비는 고객이 부담합니다. 고객이 배송비를 미납한 경우, 실제 발생한 금액을 환불 금액에서 공제합니다. 반품 상품을 수령한 날로부터 7일 이내에 사용자에게 지불한 대금을 환불해 드립니다. 환불 방식은 결제 방식과 동일합니다. 구체적인 입금 일자, 은행, 결제 기관 등의 요인에 따라 달라질 수 있습니다.
 - 2) 고객이 상품을 수령한 다음 날부터 7일 이내에 비인위적 손상이나 성능 결함이 발생한 경우, 당사 A/S 센터의 검수 확인을 거쳐 반품 처리를 진행합니다. 반품 시 고객은 유효한 구매 증빙 자료를 제시하고 세금계산서를 반환해야 합니다. 증정품이 있는 경우 함께 반환해야 합니다.
 - 3) 고객 수령일 다음 날부터 7일 후부터 15일 이내에 인위적 손상이 아닌 성능 결함이 발생하여, 당사 A/S 센터의 검사 및 확인을 거친 후, 고객에게 교환 서비스를 제공하며, 상품 전체를 교체해 드립니다. 교환 후, 상품 자체의 3보(수리·교환·환불) 기간은 다시 계산됩니다.

- 4) 고객 수령일 다음 날부터 15일 후부터 365일 이내에, 당사 A/S 센터의 검사 및 확인을 거쳐 제품 자체의 품질 결함으로 판명될 경우, 무상 수리 서비스를 제공합니다. 교체된 결함 제품은 당사의 소유가 됩니다. 결함이 없는 제품은 원상태 그대로 반환됩니다. 본 제품은 각종 엄격한 검사를 거친 후 출고되므로, 제품 자체의 품질 결함이 아닌 경우 당사는 고객의 반품 및 교환 요청을 거부할 권리가 있습니다.

본 매뉴얼의 사후 서비스 정책이 매장 사후 서비스 정책과 일치하지 않을 경우, 매장의 사후 서비스 정책을 따릅니다.

2. 보증 제외 사항

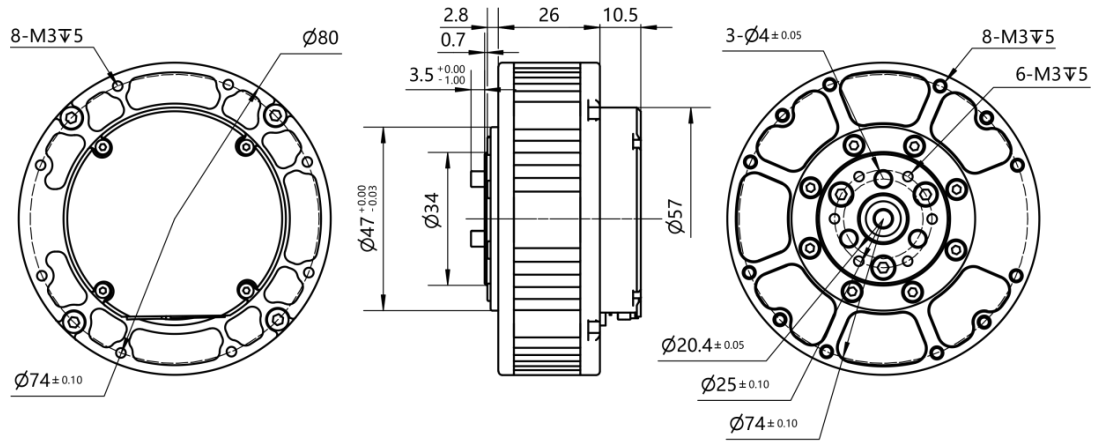
다음의 경우는 보증 대상에서 제외됩니다:

- 1) 보증 조항에서 정한 보증 기간을 초과한 경우.
- 2) 설명서의 요구 사항을 따르지 않고 잘못 사용하여 발생한 제품 손상.
- 3) 부적절한 조작, 수리, 설치, 개조, 테스트 등 부적절한 사용으로 인한 손상 또는 파손.
- 4) 품질 결함이 아닌 일반적인 기계적 마모 및 손상.
- 5) 정상적인 사용 조건이 아닌 상황에서 발생한 손상(낙하, 충돌, 액체 침투, 심한 충격 등을 포함하되 이에 국한되지 않음).
- 6) 천재지변(홍수, 화재, 낙뢰, 지진 등) 또는 불가항력으로 인한 손상.
- 7) 정격 토크를 초과하여 사용함으로써 발생한 손상.
- 8) 당사 정품이 아니거나 합법적인 구매 증빙 자료를 제시할 수 없는 경우.
- 9) 제품의 설계, 기술, 제조, 품질 등의 문제로 인한 고장 또는 손상.
- 10) 본 제품을 무단으로 분해하여 발생한 손상.

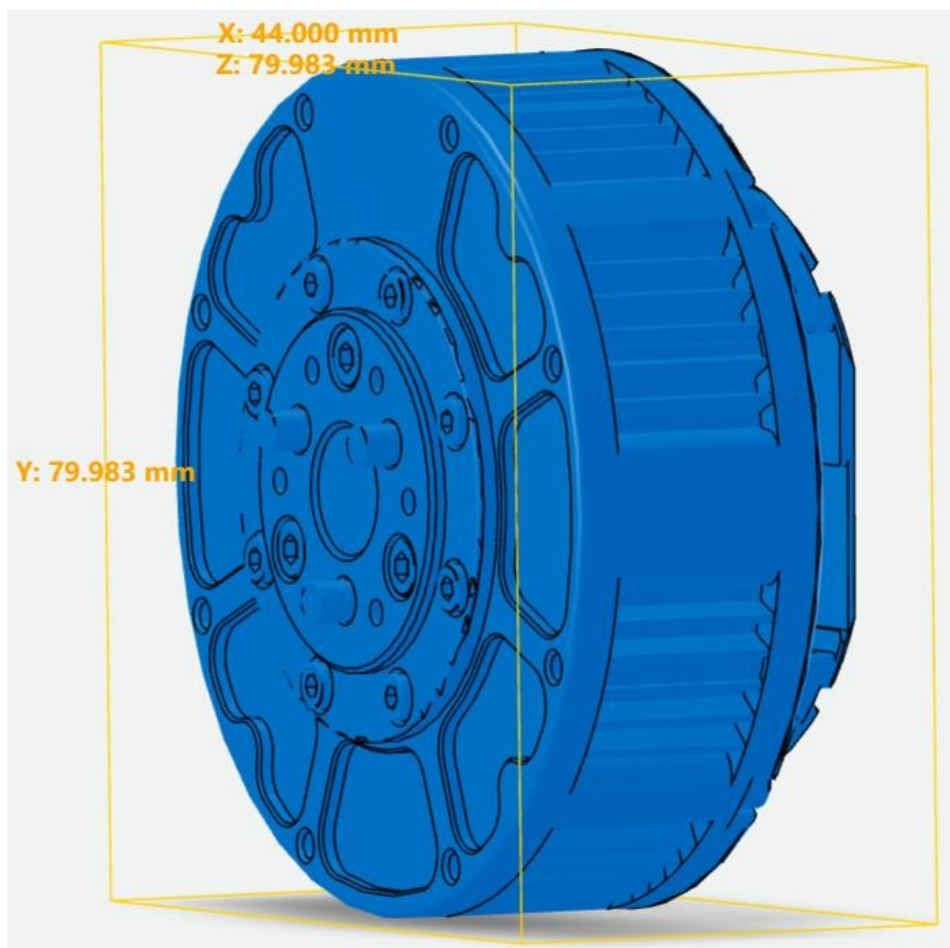
위와 같은 상황이 발생할 경우, 사용자는 비용을 직접 부담해야 합니다.

1 모터 사양

1.1 도면 및 치수



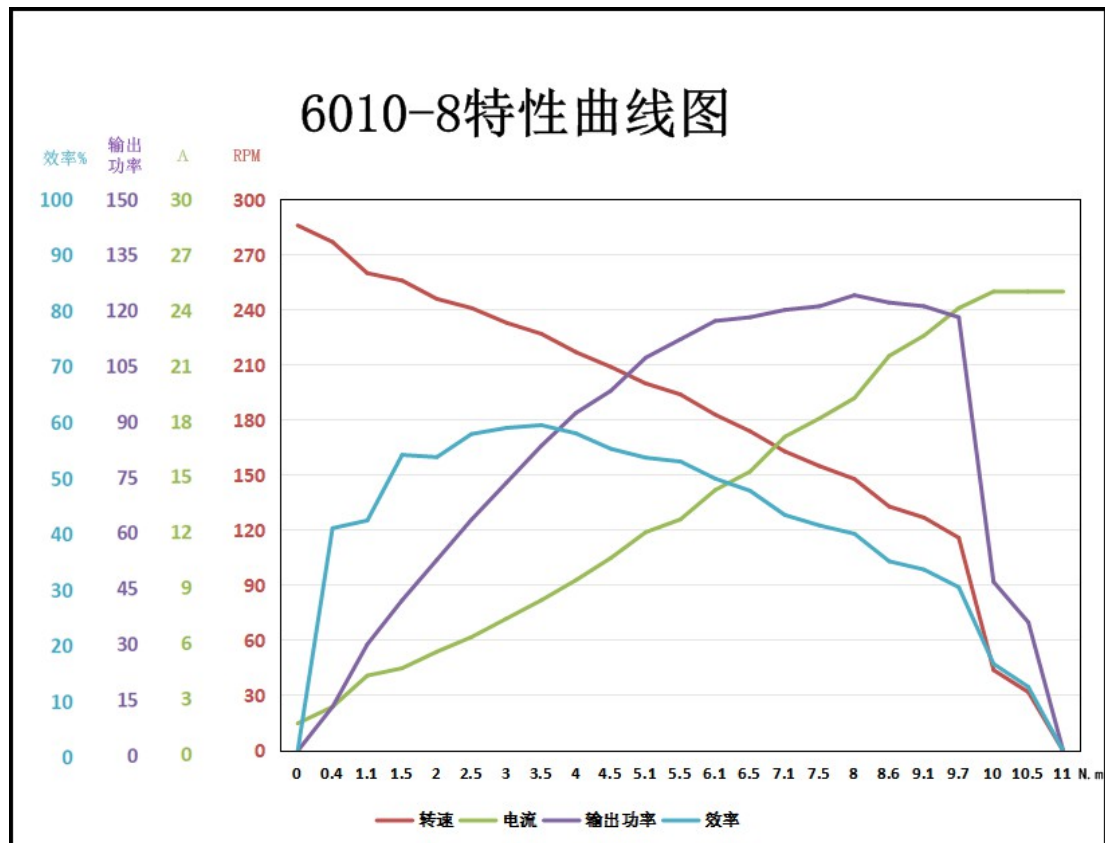
未注公差分段					比例	1 : 1	6010-8安装尺寸图
<=6	>6<=30	>30<=120	>120<=400	>400<=2000	重量kg	无	
± 0.1	± 0.2	± 0.3	± 0.5	± 1.0			



1.2 전기적 특성

정격 회전수	120rpm±10%
최대 회전수	420rpm±10%
정격 토크	5N·m
정지 토크	11N·m
정격 전류	10.5A
정지 전류	25A
무부하 전류	0.4A
절연 저항/고정자 권선	DC 500VAC, 100MΩ
고전압 내성/정자 및 하우징	600 VAC, 1초, 2mA
모터 역전압	0.054~0.057Vrms/rpm
상간 저항	0.48Ω±10%
상간 인덕턴스	368μH±10%
회전수 상수	12.3rpm/V
토크 상수	0.47 N·m/A

특성 곡선은 다음과 같습니다:



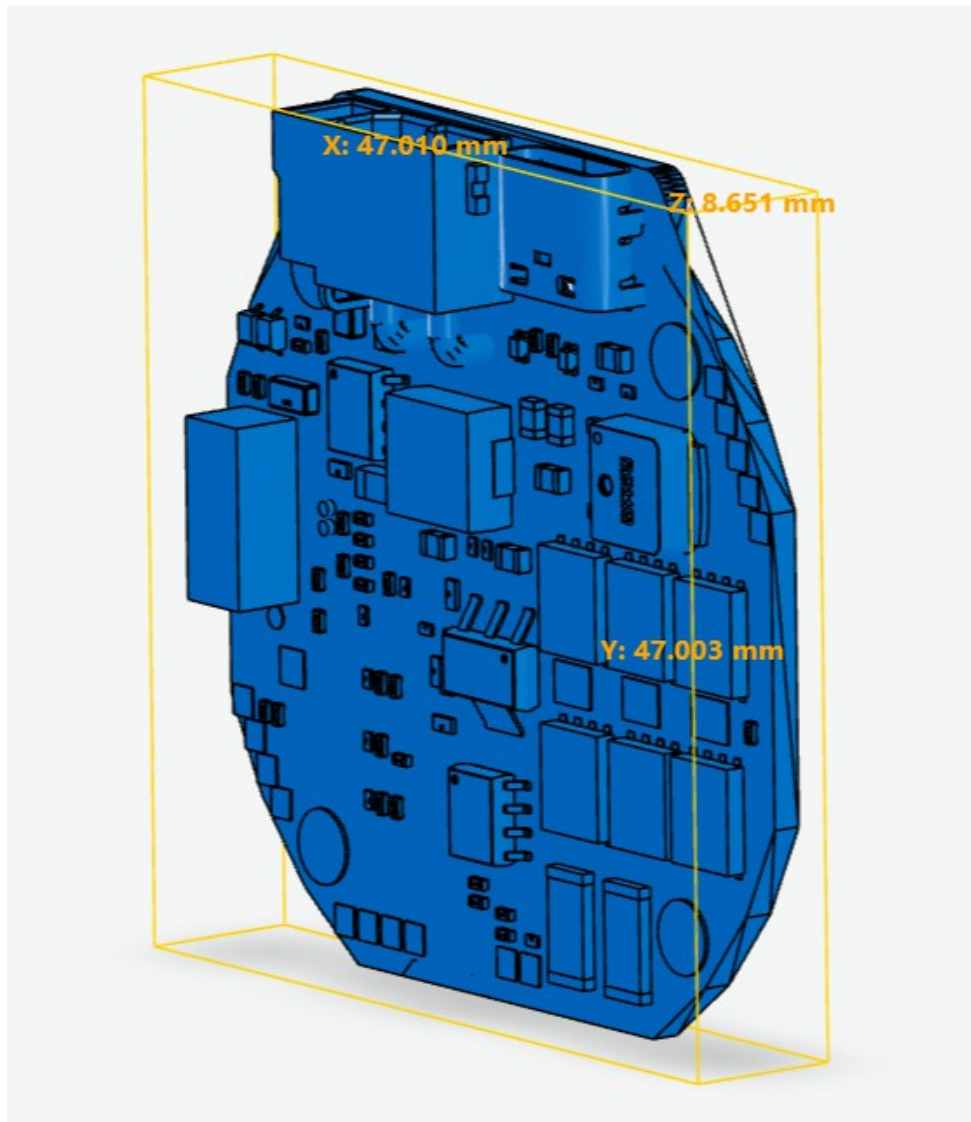
1.3 기계적 특성

무게	388g±3
극 쌍 수	14 쌍

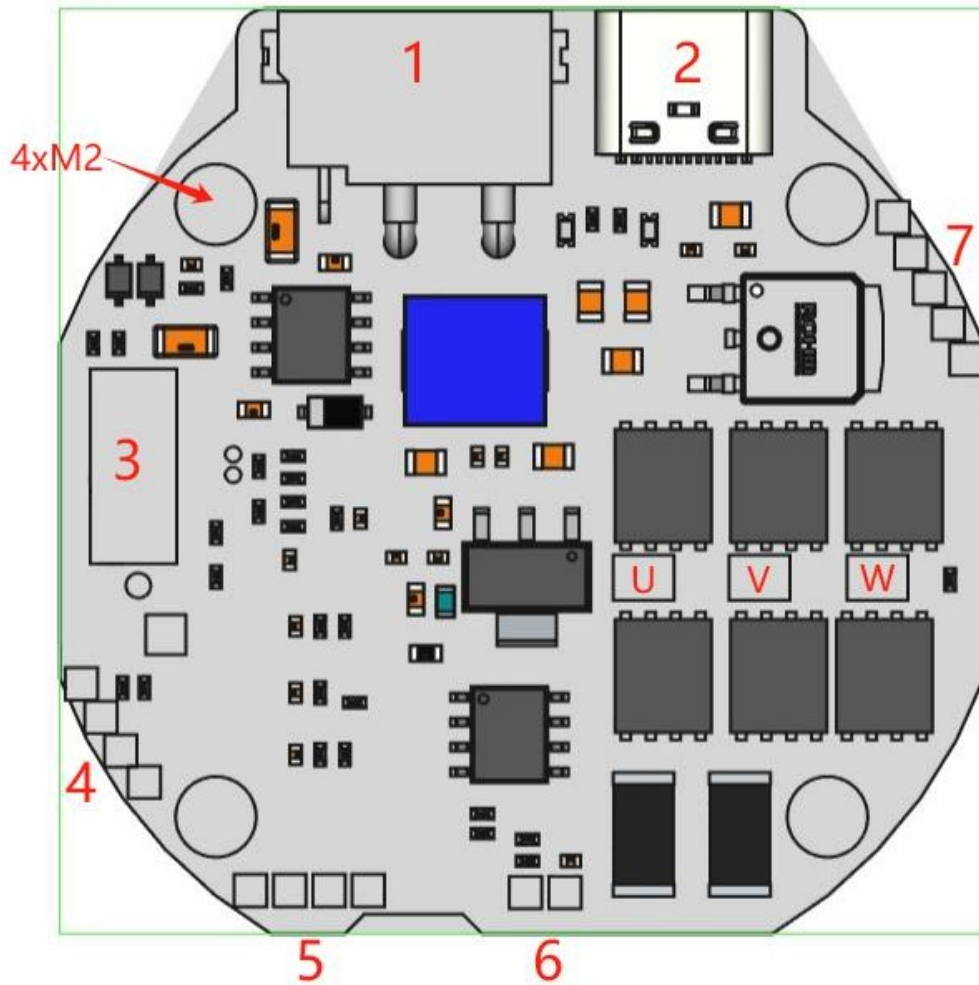
상수	3상
구동 방식	FOC
감속비	8:1

2 드라이버 정보

2.1 외관 및 3D 치수



2.2 인터페이스 개요



인터페이스 번호	정의
1	15~60V 전원 및 CAN 통신 통합 단자
2	Type-C 디버깅 인터페이스 및 호스트 통신 인터페이스
3	인터페이스 확장 슬롯 (RS485, EtherCAT, RC 모델, 펄스 방향, 유압 밸브 제어 등 인터페이스/프로토콜 확장 가능)
4	SWD 디버깅 및 다운로드 인터페이스
5	두 번째 인코더 인터페이스 (I2C 및 UART 지원)
6	모터 온도 인터페이스 (NTC)
7	브레이크/제동 저항 인터페이스, 12V 전원, 최소/최대 리미트 스위치 인터페이스
U/V/W	3상 권선 용접 구멍
4xM2	설치 구멍

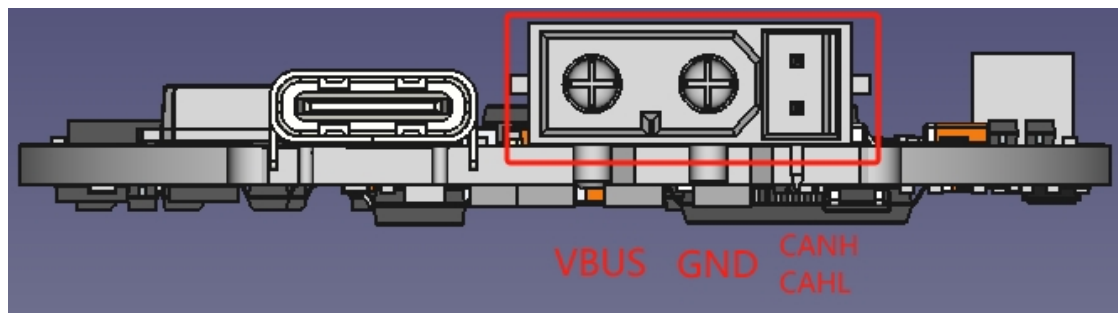
2.3 사양

정격 전압	15~48V DC
-------	-----------

최소/최대 전압	12/72V DC
정격 전류	6A
최대 라인 전류	30A
최대 상 전류	120A
대기 전력 소비	<10mA
CAN 버스 최대 전송 속도	1Mbps
Type-C 전송 속도	10Mbps
인코더 분해능	14비트(1회전 절대값)
작동 환경 온도	-20°C ~ 70°C
모터 온도 경보	90°C(조절 가능)
경보 드라이버 보드 온도	90°C(조절 가능)

2.4 인터페이스 상세 정의

2.4.1 전원 및 CAN 통신/RS485/PWM 단자



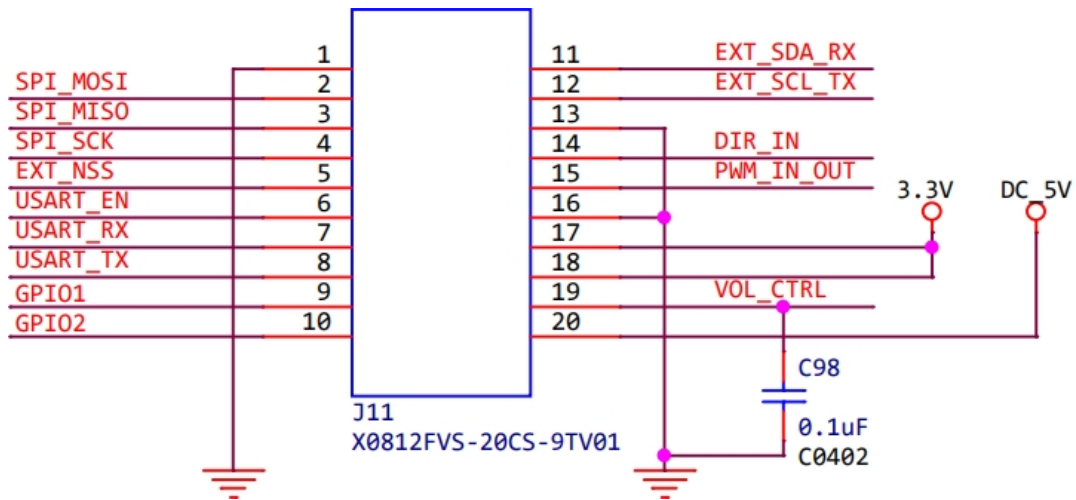
보드 장착 단자 모델 XT30PB(2+2)-M, 케이블 단자 모델 XT30(2+2)-F, 브랜드 AMASS.PWM 모드에서 CANH는 PWM 신호 입력 인터페이스이며, 펄스 모드에서 CANH는 펄스 입력 신호 인터페이스이고, CANL은 펄스 방향 입력 인터페이스입니다. RS485를 지원하는 버전의 경우, CANH/CANL은 동시에 RS485 A/B이기도 하므로 주의하시기 바랍니다.

2.4.2 Type-C 디버깅 인터페이스

Type-C는 표준 데이터 케이블 규격을 채택하여, 일반적인 PC 또는 휴대폰용 Type-C 데이터 케이블과 호환됩니다.

2.4.3 인터페이스 확장 슬롯

이 슬롯은 아래의 설계 방식을 채택하여 풍부한 보드 간 확장 인터페이스를 제공하며, 타사에서 임의의 확장 보드를 개발할 수 있습니다. **하드웨어 버전 3.9 및 이후 버전에서는 이 확장 인터페이스가 더 이상 제공되지 않으므로 주의하시기 바랍니다.**



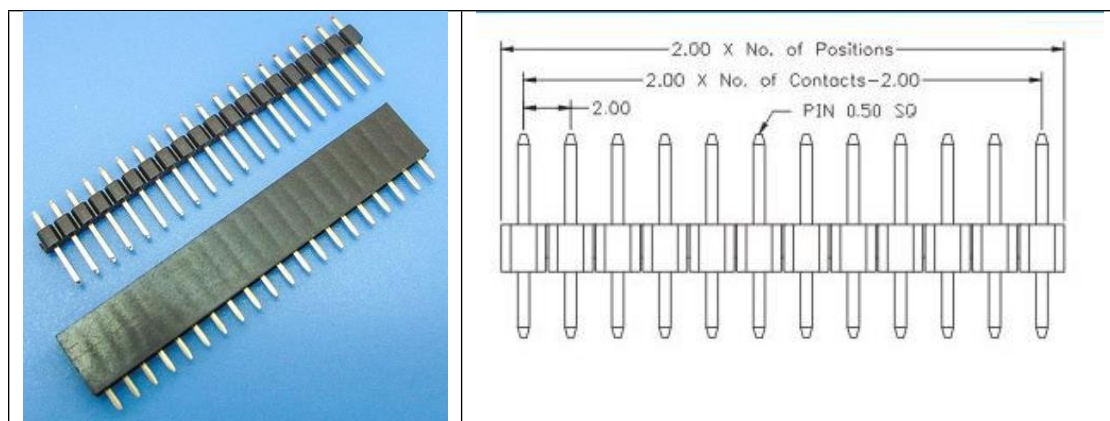
타사 개발자는 SPI, USART, I2C, PWM, ADC, GPIO 등의 방식을 통해 드라이버와 상호 작용하여 다양한 확장 기능을 구현할 수 있습니다.

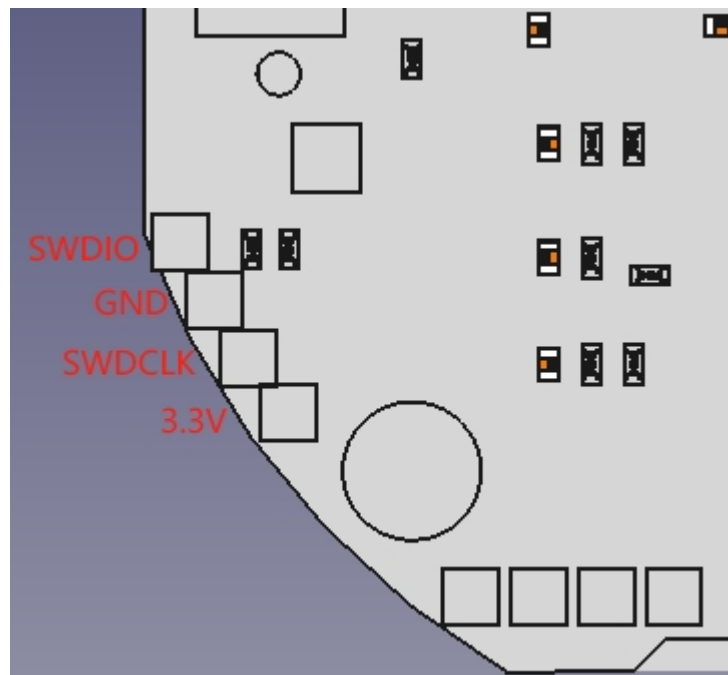
보드 내장 슬롯 모델명은 X0812FVS-20CS-9TV01(암형), 확장 보드 슬롯 모델명은 X0812WVS-20AS-9TV01(수형)이며, 제조사는 싱쿰(Xingkun)입니다.

암 커넥터 X0812FVS-20CS-9TV01	수컷 커넥터 X0812WVS-20AS-9TV01

2.4.4 SWD 디버깅 인터페이스

2mm 간격의 핀 구멍으로, 사용자는 2mm 직입형 단일 행 핀을 납땜할 수 있습니다. 아래 그림 참조:

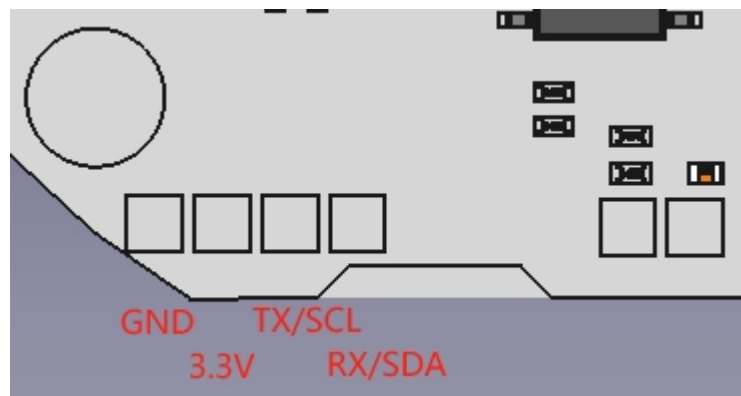




2.4.5 두 번째 인코더 인터페이스

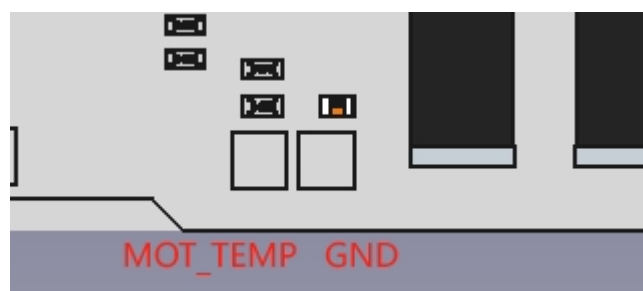
2mm 간격의 핀 구멍으로, 사용자는 2mm 직입형 단일 행 핀을 납땜할 수 있습니다. 2.4.4항을 참조하십시오.

이 인터페이스는 USART(TX/RX) 또는 I2C(SCL/SDA)를 통해 두 번째 인코더와 통신할 수 있습니다.



2.4.6 모터 온도 인터페이스

모터에는 10K NTC 저항이 내장되어 있으며, 두 개의 리드선을 MOT_TEMP 및 GND에 납땜합니다. 배선 순서는 없습니다.

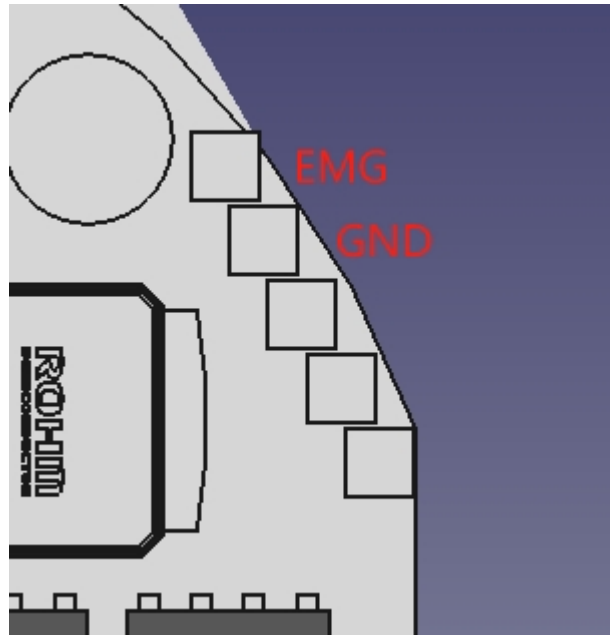


2.4.7 브레이크 저항 인터페이스

그림에 표시된 5핀 인터페이스의 상단 두 개의 납땜 구멍은 홀드 브레이크/브레이크 저항 인터페이스이며, 2mm 간격의 핀 구멍으로, 사용자는 2mm 직입형 단일 행 핀을 납땜할 수 있습니다. 2.4.4항을 참조하십시오.

브레이크 인터페이스로 사용할 경우, 드라이버는 전원이 공급될 때 이 인터페이스에 지속적으로 전류를 공급하여 브레이크를 개방 상태로 유지하고 모터가 정상적으로 작동하도록 합니다. 드라이버의 전원이 차단되면 이 전류가 중단되고 브레이크가 잠기며, 모터는 전원이 차단된 위치에서 고정됩니다.

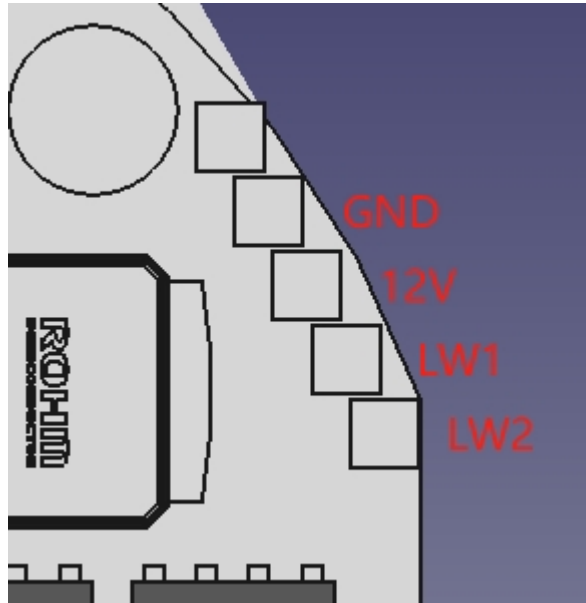
브레이크 저항 인터페이스인 경우, 브레이크 저항(또는 방전 저항)을 외부에 연결할 수 있습니다. 역기전력이 임계 전압보다 높을 때, 이 브레이크 저항을 통해 전류를 방전시켜 비상 제동을 할 수 없거나 역기전력으로 인해 드라이버가 손상되는 것을 방지합니다.



2.4.8 리미트 스위치 인터페이스

드라이버는 두 개의 리미트 스위치 인터페이스를 제공하며, 외부 리미트 스위치에 12V 전원을 공급합니다. 핀 간격은 2mm이며, 사용자는 2mm 직입형 단일 핀 헤더를 납땜할 수 있습니다. 2.4.4항을 참조하십시오.

이 중 LW1은 최소 위치 리미트 스위치이고, LW2는 최대 위치 리미트 스위치입니다. 2선식 스위치 또는 3선식 NPN 스위치를 연결할 수 있습니다.



2.5 주요 부품 및 사양

번호	부품	모델/사양	수량
1	MCU	N32G455	1
2	드라이버 칩	FD6288Q	1
3	자기 인코더 칩	MA732, 14비트 절대값	1
4	MOSFET	JMSH1004NG, 100V/120A	6

3 설정 및 테스트 안내

3.1 시작 가이드

3.1.1 준비 사항

모터를 작동시키려면 다음이 필요합니다:

✓ 전원

제1장의 전원 전압 요구 사항을 참조하십시오. 정전압 전원 공급 장치나 배터리를 사용하는 것을 권장합니다. 사용자들이 자주 궁금해하는 점은 '전원을 어떻게 선택해야 하나요?'입니다. 다음은 참고용으로 제공되는 몇 가지 간단한 제안입니다:

전원 선택 시 고려 사항:

◆ 전류 요구 사항

일반적으로 최소 5A 이상이어야 합니다. 구체적인 수치는 시스템의 전력 요구량과 전압에 따라 달라집니다.

◆ 전압 요구 사항

전압 요구 사항은 두 가지 요인에 따라 결정됩니다: 모터의 Kv와 시스템에 필요한 최대 회전수 RPM_{max}입니다. 필요한 전원 전압의 최대값은 다음 공식을 참고하십시오:

$$V_{max} = \frac{RPM_{max}}{K_v} \times 1.25$$

여기서 1.25는 경험적 계수로, 시스템에 안전한 전압 한계치를 제공합니다.

◆ 전력 요구 사항

전력의 경우, 간단히 말해 최고 회전수에서의 최대 전류값 I_{max}에 따라 결정되며, 다음 공식을 참조할 수 있습니다:

$$P_{max} = I_{max} \times \frac{RPM_{max}}{K_v} \times 1.25$$

✓ 전원+통신 인터페이스 케이블

6010-8 모델은 2+2 전원 통신 소켓을 갖추고 있으므로, 적합한 2+2 케이블을 추천받으려면 고객 지원팀에 문의하십시오.

2.4.1항을 참조하여 반드시

전원 양극과 음극을 반드시 올바르게 연결하십시오. 그렇지 않으면 드라이버가 역극 연결 방지 기능을 갖추고 있지 않아 드라이버가 소손될 위험이 있습니다. 또한, CANH/CANL과 같은 2개의 통신선 정의 순서에 주의하십시오. 잘못 연결하면 통신이 정상적으로 이루어지지 않을 수 있습니다.

경고

- ✧ 반드시 통신 버스를 손으로 만지지 마십시오. 특히 건조한 지역이나 건조한 계절에는 정전기로 인해 드라이버의 인터페이스 칩이 손상될 수 있습니다!
- ✧ 전원이 연결된 상태에서 전원 단자를 뽑거나 꽂지 마십시오!
- ✧ 전원 스위치로 차단기를 사용하지 마십시오. 드라이버의 전원 칩이 파손될 위험이 있습니다!
- ✧ 전원 전압은 72V를 초과하지 마십시오!

✓ Type-C 데이터 케이블

테스트 초기 단계에서는 USB Type-C 데이터 케이블을 사용하여 모터를 테스트할 것을 강력히 권장합니다. 가장 일반적인 휴대폰용 Type-C 데이터 케이블을 사용하면 되며, 충전 전용 Type-C 케이블은 사용하지 마십시오.

Type-C 데이터 케이블은 드라이버에 전원을 공급할 수 없으며, 모터를 구동할 수도 없습니다!

✓ 전원 공급

먼저 전원을 끄고 전원 케이블을 연결한 후 전원을 켜십시오. 절대로 전원이 켜진 상태에서 케이블을 꽂거나 뽑지 마시고, 단극(양극 또는 음극) 케이블을 차단기로 제어하여 전원을 켜거나 끄지 마십시오. 이렇게 하면 과도한 시동 전류로 인해 드라이버가 손상될 수 있습니다.

전원 공급 후에는 언제든지 USB Type-C 데이터 케이블을 꽂거나 뽑을 수 있습니다.

3.1.2 호스트 컴퓨터로 시작



1. 모터 위저드(Motor Wizard) 설치

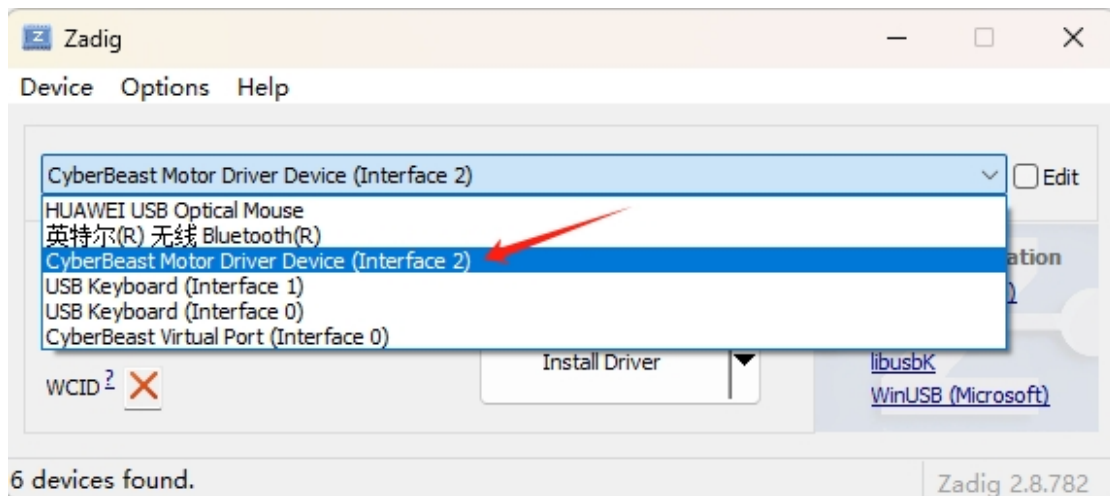
Windows용 설치 파일을 다운로드하십시오

(https://bl.cyberbeast.cn/actuator/steadywin_motorwizard.exe), 설치 단계에 따라 기본 설정으로 설치하시면 됩니다.

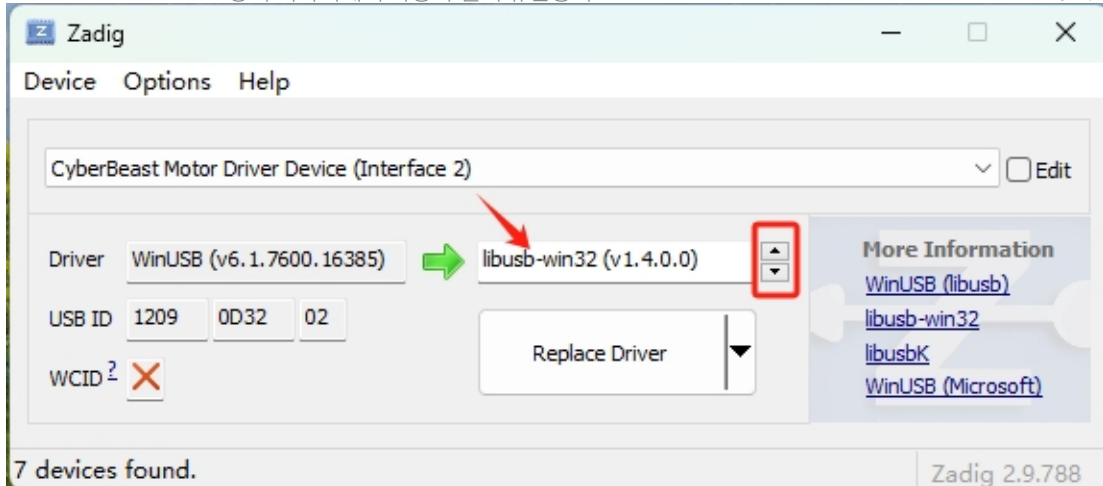
2. USB 드라이버 설치

웹사이트 <https://zadig.akeo.ie>에 접속하여 USB 드라이버 도구 Zadig를 다운로드합니다. Type-C 데이터 케이블로 드라이버를 컴퓨터에 연결하면 드라이버의 전원 표시등이 켜집니다. Zadig를 실행하고 드롭다운 메뉴에서 “CyberBeast Motor Driver Device

Device (Interface 2)를 선택하십시오:



위아래 화살표 버튼을 클릭하여 다른 USB 드라이버를 선택하고, 이 인터페이스에는 “libusb” 드라이버 버전을 선택한 후 “Install Driver” 또는 “Replace Driver”를 클릭하여 이 인터페이스에 드라이버를 설치하십시오:



이 드라이버는 odrivetool(<https://github.com/odriverobotics/odrive.git>)와 호환되므로, odrivetool을 호스트 컴퓨터로 사용하여 테스트 및 튜닝을 진행합니다.

다음 단계에 따라 odrivetool을 설치하십시오:

➤ Windows

1. Python 설치

Python 공식 웹사이트(<https://www.python.org>)에 접속하여 최신 Python 설치 프로그램을 다운로드하고 안내에 따라 설치하십시오. 타사 웹사이트나 Microsoft Store에서 제공하는 Python 버전은 다운로드하지 마십시오.

2. Visual C++ 빌드 도구 설치

Visual C++ 빌드 도구(<https://visualstudio.microsoft.com/visual-cpp-build-tools/>)를 설치하고, 설치 과정에서 아래 그림과 같이 "C++ 데스크톱 개발"을 선택하십시오.



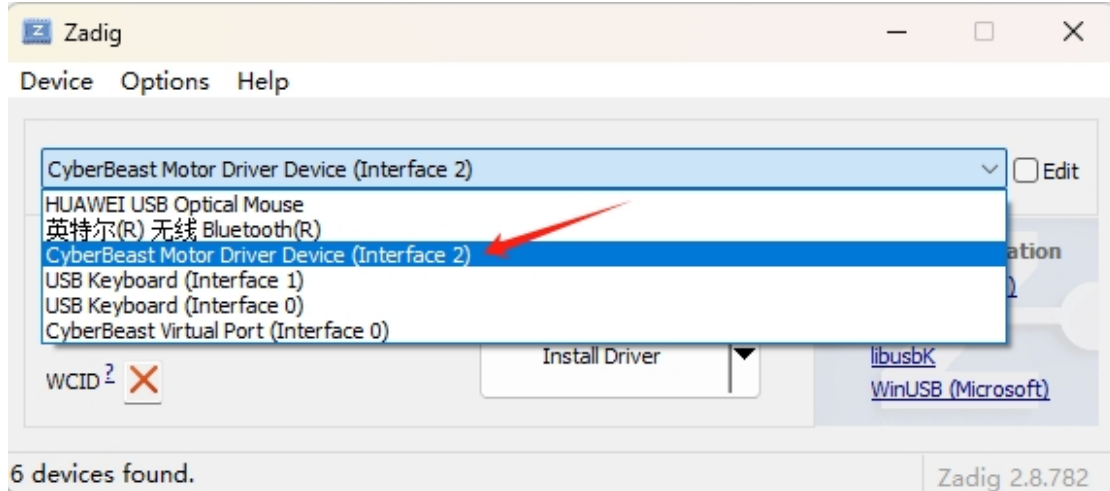
3. odrivetool 설치

관리자 권한으로 Windows PowerShell을 실행한 후, pip install odrive를 입력하고 Enter 키를 눌러 설치합니다. 도중에 오류가 발생하면 다시 시도하십시오. 계속 오류가 발생하면 컴퓨터를 재부팅한 후 다시 시도하십시오.

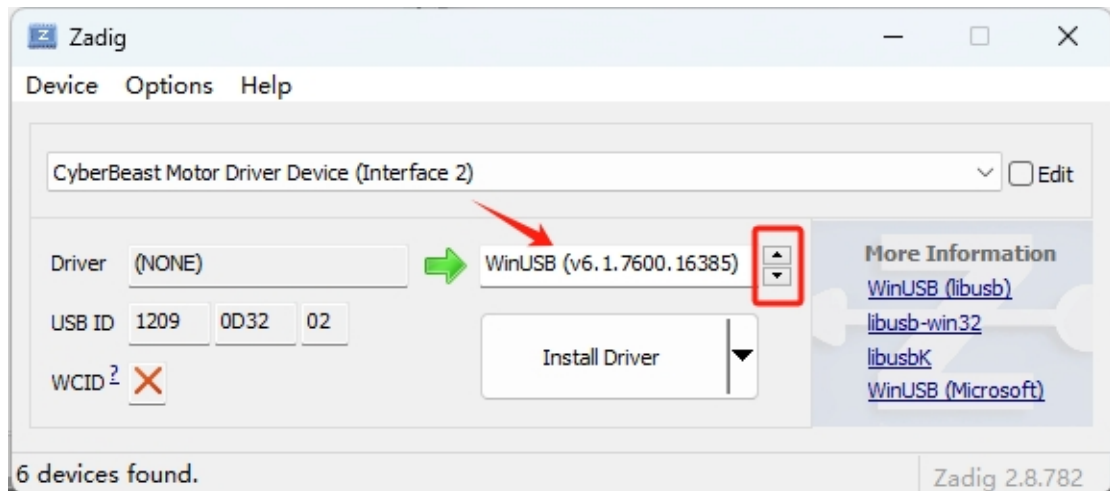
4. USB 드라이버 설치

웹사이트 <https://zadig.akeo.ie> 에 접속하여 USB 드라이버 도구 Zadig를 다운로드합니다. Type-C 데이터 케이블로 드라이브를 컴퓨터에 연결하면 드라이브의 전원 표시등이 켜집니다. Zadig를 실행하고 드롭다운 메뉴에서 “CyberBeast Motor Driver Device (Interface 2)”:

(Interface 2)”:



위아래 화살표 버튼을 클릭하여 다른 USB 드라이버를 선택하고, 이 인터페이스에는 “WinUSB” 드라이버 버전을 선택한 후 “Install Driver”를 클릭하여 이 인터페이스에 드라이버를 설치하십시오:



➤ WSL(Windows Subsystem for Linux)

- 1) python/usb/odrivetool 설치

WSL2를 설치한 경우, WSL2 명령줄로 이동하여 아래 단계에 따라 설치합니다(WSL에 Ubuntu가 설치되어 있다고 가정):

```
sudo apt install python3 python3-pip sudo apt
install libusb-1.0-0
sudo pip install odrivetool numpy matplotlib
```

첫 번째 명령어는 Python을 설치하고, 두 번째 명령어는 USB 드라이버를 설치하며, 세 번째 명령어는 odrivetool 클라이언트를 설치합니다.

2) 드라이버를 WSL에 연결

Type-C 데이터 케이블을 Windows에 연결하면, 기본적으로 Windows는 이 USB 포트의 드라이버를 로드하지만 WSL은 로드하지 않습니다. 이 USB 포트를 WSL에 로드하려면 Microsoft 문서

(<https://learn.microsoft.com/zh-cn/windows/wsl/connect-usb>)의 내용을 참고하여 진행하십시오.

➤ Ubuntu

Ubuntu에서의 설치 과정은 WSL과 매우 유사하므로, 앞 절을 참조하십시오.

3.1.3 모터 매개변수 적절히 설정하기

경고

이 절을 꼼꼼히 읽어보시기 바랍니다. 이는 성공적인 구동의 핵심이며, 모터 소손을 방지하기 위함입니다!

이전 절의 내용에 따라 상위 프로그램(Motor Genie 또는 odrivetool)을 성공적으로 설치한 후, 모터에 전원을 공급하고 USB Type-C 데이터 케이블을 연결하면:



모터 엘프를 실행하면, 연결이 성공한 경우 위 그림 왼쪽과 같이 모터 상태 및 전압이 표시됩니다. 연결에 실패한 경우 정상적인 전압 정보를 표시할 수 없습니다.

“모터 파라미터”를 클릭하면 화면 상단에 드라이버의 하드웨어 버전과 펌웨어 버전이 표시됩니다. 위 그림 오른쪽과 같이 하드웨어 버전은 3.8.3, 펌웨어 버전은 0.5.13입니다.



셸(Windows PowerShell 또는 Linux 터미널)에서 odrivetool을 실행합니다(odrivetool을 입력하고 Enter 키를 누름). 아래 그림은 Windows에서 연결에 성공한 모습입니다(녹색 글꼴로 연결 정보가 표시됨):

```

PS D:\> odrivetool
Website: https://odriverobotics.com/
Docs: https://docs.odriverobotics.com/
Forums: https://discourse.odriverobotics.com/
Discord: https://discord.gg/k3ZZ3mS
Github: https://github.com/odriverobotics/ODrive/

Please connect your ODrive.
You can also type help() or quit().

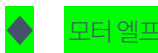
Connected to ODrive v3.6 B53319683238 (firmware v0.5.6) as odrv0
In [1]: odrv0.vbus_voltage
Out[1]: 20.00840187072754
In [2]: _
  
```

명령어 예시: odrv0.axis0.controller.input_vel, odrv0는 현재 연결된 모터를 나타내며, 기본적으로 첫 번째 연결된 모터는 odrv0, 두 번째는 odrv1 등으로 명명됩니다; axis0는 드라이버에 연결된 첫 번째 모터를 나타내며, 현재 버전에서는 하나의 모터 연결만 지원합니다. 이 명령어는 드라이버의 현재 속도 제어 목표값을 조회하는 것입니다.

사용 팁

- ◆ 자주 TAB 키를 사용하면 명령어 자동 완성 기능이 표시됩니다. 이는 Linux의 명령어 자동 완성 기능과 유사하며, 상하좌우 화살표 키를 사용하여 명령을 선택할 수 있습니다;
- ◆ 위/아래 방향 키를 누르면 과거 명령어 기록이 표시됩니다
- ◆ 명령어 입력 시 유사한 과거 명령어가 제안되며, 오른쪽 화살표 키를 사용하면 자동으로 완성됩니다

➤ 한도(limits) 설정



모터 위젯의 “모터 파라미터”를 클릭하면, 아래 그림의 빨간색 테두리 부분과 같이 다음과 같은 주요 한계값 파라미터를 설정할 수 있습니다: 전압 범위, 전류 한계(최대 전류), 속도 한계(최대 속도), 그리고 보정 전류.



전류 한계는 온도 요인의 영향을 받으며, 다음 소절(odrivetool)의 관련 설명을 참조하십시오.

속도 한계는 일반적으로 속도 제어 또는 위치 제어 모드에서의 제한을 의미하지만, 토크 제어 모드에서 속도 제한 스위치를 켤 수도 있습니다. 모터 위젯에서 “구동 파라미터”를 클릭하면 이 스위치를 켤 수 있으며, 아래 그림의 빨간색 테두리 부분과 같습니다:



● 전류 한계

```
odrv0.axis0.motor.config.current_lim = 30
```

위 명령은 전류 임계값을 30A로 설정합니다. 이 전류 임계값은 전원 전류가 아닌 Q축 전류를 의미한다는 점에 유의하십시오. 이 임계값은 출력 토크를 직접 제한합니다. 6010-8 모델의 경우, 이 임계값을 50A 이상으로 설정하지 마십시오!

전류 임계값에 영향을 미치는 기타 요인

◆ 모터 온도

모터 온도 보호 기능을 활성화한 경우(odrv0.axis0.motor.motor_thermistor.config.enabled=1), 모터의 모터의 현재 온도도 Q축 전류에 영향을 미칩니다.

◆ 드라이버 보드 온도

드라이브 보드 온도 보호 기능을 활성화한 경우(odrv0.axis0.motor.fet_thermistor.config.enabled=1), 드라이브 보드의 현재 온도도 Q축 전류에 영향을 미칩니다.

위 두 온도의 영향은 매우 유사하며, 다음 공식으로 표현할 수 있습니다:

$$I_{lim}^{\circ C} = \frac{T - T_l}{T_u - T_l} \times I_{lim}$$

여기서 I_{lim} 는 최종적으로 적용되는 전류 임계값, I_{lim} 는 설정된 전류 임계값, T 는 현재 온도(모터 온도 또는 구동 보드 온도), T_l 는 설정된 하한 온도(motor_thermistor.config.temp_limit_lower 및 fet_thermistor.config.temp_limit_lower), T_u 는 설정된 상한 온도(motor_thermistor.config.temp_limit_upper 및 fet_thermistor.config.temp_limit_upper)입니다.

● 속도 한계

```
odrv0.axis0.controller.config.vel_limit = 30
```

시스템 전체 속도 제한으로, 위 명령은 이를 30turn/s(회전/초)로 제한합니다. 참고로, 기본적으로 순수 토크 모드에서는 이 속도 제한이 작동하지 않지만, 아래 스위치를 켜서 활성화할 수 있습니다:

```
odrv0.axis0.controller.config.enable_torque_mode_vel_limit = 1
```

● 보정 전류

이 전류는 기본적으로 5A이며, 기본적으로 수정할 필요가 없습니다. 그러나 사용자의 전원 전류가 작은 경우 이 값을 줄일 수 있습니다. 그렇지 않으면 보정 시 저전압 경보가 발생할 수 있습니다.

```
odrv0.axis0.motor.config.calibration_current = 2
```

➤ 주요 하드웨어 매개변수 설정



모터 탭

모터 탭에서 "모터 매개변수"를 클릭하면, 아래 그림의 빨간색 테두리 안에 몇 가지 주요 하드웨어 매개변수가 표시됩니다.



이 매개변수들의 의미는 다음 소절(odrivetool)의 설명을 참조하십시오.



● 최대 방전/충전 전류

방전 전류는 전원 공급 장치가 드라이버와 모터로 공급하는 정방향 전류를 의미하며, 충전 전류는 역방향으로 전원 공급 장치로 유입되는 전류를 의미합니다. 이 두 값은 전원 공급 장치와 관련이 있으므로, 전원이 방전할 수 없어 전압이 낮아지거나 역기 전력에 의해 손상되는 것을 방지하기 위해 적절한 값으로 설정하십시오. 단, 이 두 값을 절대값이 작은 값으로 설정할 경우 경보가 발생하기 쉬우므로 주의하십시오.

```
odrv0.config.dc_max_negative_current odrv0.config.dc_max_positive_current
```

● 극 쌍 수

극 쌍 수는 모터 로터의 자극 수를 2로 나눈 값입니다. 사용자는 이 값을 올바르게 설정해야만 보정이 성공하며, 그렇지 않으면 보정 경고가 발생합니다.

```
odrv0.axis0.motor.config.pole_pairs
```

● 토크 상수

토크 상수는 모터가 생성하는 토크를 Q축 전류로 나눈 값이며, 모터의 Kv 값과의 관계는 다음과 같습니다:

$$Torque_Constant = 8.27/K_v.$$

```
odrv0.axis0.motor.config.torque_constant
```

토크 상수가 정확한지 여부는 모터 작동에는 영향을 미치지 않지만, 사용자가 토크 제어를 할 때 입력하는 값의 단위 변환에는 영향을 미칩니다. 사용자가 Nm 대신 A 단위를 사용하여 토크 제어를 하고 싶다면, 이 값을 1로 설정하기만 하면 됩니다.

● 온도 센서

드라이버 보드와 모터 내부에는 NTC 온도 센서가 장착되어 있습니다. 이 기능을 활성화하면 드라이버는 온도에 따라 출력 전류(토크)를 제어하여 드라이버와 모터를 보호합니다.

```
# 모터 온도 보호
odrv0.axis0.motor.motor_thermistor.config.enabled = 1
odrv0.axis0.motor.motor_thermistor.config.temp_limit_lower = 20
odrv0.axis0.motor.motor_thermistor.config.temp_limit_upper = 100 # 드라이버 보드 온도 보호
odrv0.axis0.motor.fet_thermistor.config.enabled = 1
odrv0.axis0.motor.fet_thermistor.config.temp_limit_lower = 20
odrv0.axis0.motor.fet_thermistor.config.temp_limit_upper = 100 # 온도 획득
odrv0.axis0.motor.motor_thermistor.temperature # 모터 온도
odrv0.axis0.motor.fet_thermistor.temperature # 드라이버 보드 온도
```

➤ PID 조정

다음 절차는 사용자가 PID 매개변수를 조정하는 데 참고가 될 수 있습니다:

```
odrv0.axis0.controller.config.pos_gain=20.0
odrv0.axis0.controller.config.vel_gain=0.16
odrv0.axis0.controller.config.vel_integrator_gain=0.32
```

1. PID 초기값 설정
2. vel_integrator_gain을 0으로 설정

```
odrv0.axis0.controller.config.vel_integrator_gain=0
```

3. vel_gain 조정 방법:
 - 1) 속도 제어 모드로 모터를 구동할 때, 회전이 불안정하거나 떨림 또는 진동이 발생하면 vel_gain을 회전 상태가 안정될 때까지 줄입니다
 - 2) 그 다음, vel_gain을 매번 30% 정도씩 늘려가며  뚜렷한 떨림이 나타날 때까지 조정합니다
 - 3) 이때 vel_gain을 약 50% 정도 줄이면 안정화됩니다
4. pos_gain 조정 방법:
 - 1) 위치 제어 모드로 모터를 회전시켜 봅니다. 회전이 불안정하거나 당기는 느낌, 진동이 발생하면 pos_gain을 줄여 회전이 안정될 때까지 조정합니다
 - 2) 이어서 pos_gain을 매번 약 30%씩 증가시켜 위치 제어에서 뚜렷한 오버슈트(즉, 위치 제어 시 모터가 목표 위치를 초과했다가 다시 목표 위치로 진동하며 돌아오는 현상)가 나타날 때까지 조정합니다
 - 3) 그 후, 오버슈트 현상이 사라질 때까지 pos_gain을 서서히 줄입니다
5. 위의 4단계 조정을 마친 후, vel_integrator_gain을 $0.5 * \text{bandwidth} * \text{vel_gain}$ 으로 설정할 수 있습니다. 여기서 bandwidth는 시스템 제어 대역폭입니다. 제어 대역폭이란 무엇인가? 예를 들어, 사용자가 목표 위치를 설정한 시점부터 모터가 실제로 목표 위치에 도달하는 데 걸리는 시간이 10ms라면, 제어 대역폭은 100Hz가 되며, 이 경우 $\text{vel_integrator_gain} = 0.5 * 100 * \text{vel_gain}$ 이 됩니다.

위의 파라미터 조정 과정에서, 육안으로 판단할 때 발생할 수 있는 오차를 피하기 위해 3.1.8절의 그래픽 도구를 사용하여 실시간으로 조정 결과를 확인하는 것을 권장합니다. 모터 엘프(Motor Elf)를 사용할 경우, PID 조정 결과는 "보이는 대로 적용"됩니다. 아래 그림은 PID 파라미터의 예시입니다:



3.1.4 전원 연결 시 보정

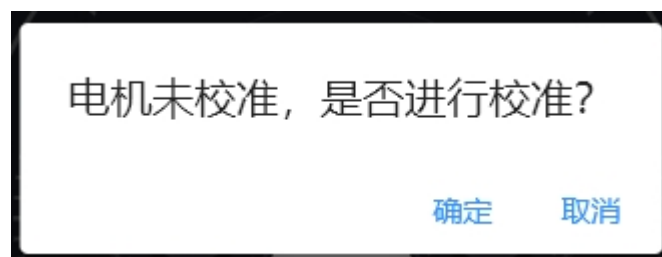
사용자가 마이크로 모터를 처음 사용할 때는 모터와 인코더를 보정해야 합니다. 보정 전에 모터를 단단히 고정하거나 손으로 꼭 잡고, 출력축을 무부하 상태로 유지하십시오. 보정 과정은 다음과 같습니다:

경고

보정 전, 모터를 무부하 상태로 만들고 단단히 고정하십시오!

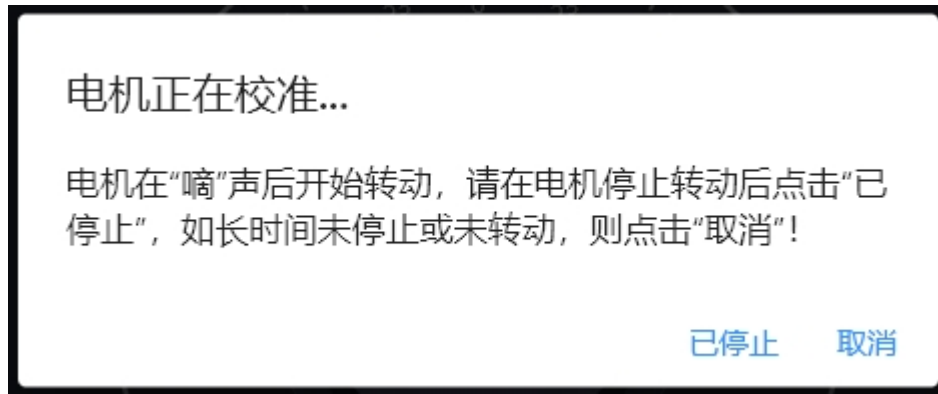


모터 엘프를 실행하고 “모터 시작”을 클릭하면, 모터가 보정되지 않은 경우 다음과 같은 메시지가 표시됩니다:



이때 “확인”을 클릭하면 보정 절차로 들어갑니다. 또는 메인 화면의 “모터 파라미터”를 클릭하고 “기본 파라미터”에서, 모터의 보정 상태가 표시되며, **未校准** 상태라면 상태 오른쪽의 “보정” 버튼을 클릭하면 마찬가지로 보정 절차로 들어갑니다.

보정 절차로 진입하면 화면에 다음과 같은 메시지가 표시됩니다:



이때 모터에서 “삐” 하는 소리가 나며 회전을 시작합니다. 모터의 회전 상태를 관찰해 주십시오. 일정 시간 회전 후 멈추면 “정지됨”을 클릭하여 보정 과정을 종료하십시오. 회전하지 않거나 5분이 지나도 멈추지 않으면 “취소”를 클릭하십시오.

보정 종료 후 드라이버가 재시작되므로 1~3초 정도 기다려 주십시오. 이때 다시 “모터 파라미터”로 들어가 “已校准”가 표시되는지 확인해 주십시오.



```

IPython: C:\Users\yongh
PS C:\Users\yongh> odrivetool
Website: https://odriverobotics.com/
Docs: https://docs.odriverobotics.com/
Forums: https://discourse.odriverobotics.com/
Discord: https://discord.gg/k3ZZ3mS
Github: https://github.com/odriverobotics/ODrive/

Please connect your ODrive.
You can also type help() or quit().

Connected to ODrive v3.6 B948106C3537 (firmware v0.5.6) as odrv0
In [1]: odrv0.axis0.requested_state=AXIS_STATE_MOTOR_CALIBRATION

In [2]: odrv0.axis0.requested_state=AXIS_STATE_ENCODER_OFFSET_CALIBRATION

In [3]: odrv0.axis0.motor.config.pre_calibrated=1

In [4]: odrv0.axis0.encoder.config.pre_calibrated=1

In [5]: odrv0.save_configuration()
    
```

```

odrv0.axis0.requested_state=AXIS_STATE_MOTOR_CALIBRATION dump_errors(odrv0)
odrv0.axis0.requested_state=AXIS_STATE_ENCODER_OFFSET_CALIBRATION dump_errors(odrv0)
odrv0.axis0.motor.config.pre_calibrated = 1
odrv0.axis0.encoder.config.pre_calibrated = 1 odrv0.save_configuration()
    
```

단계별 설명은 다음과 같습니다:

➤ 1단계: 모터 파라미터 자동 인식

모터의 상 저항과 상 인덕턴스를 측정하면 날카로운 "삐" 소리가 들립니다. 상 저항과 상 인덕턴스의 측정 결과는 다음 명령어를 통해 확인할 수 있습니다:

```
odrv0.axis0.motor.config.phase_resistance odrv0.axis0.motor.config.phase_inductance
```

➤ 2단계: 오류 코드 확인

1단계 수행 후 시스템 오류 코드를 확인하십시오. 만약  빨간색 오류 코드가 표시되면 모터를 재시작한 후 다시 시도하거나 고객 지원 센터에 문의하십시오.

➤ 3단계: 인코더 보정

인코더를 보정합니다. 여기에는 인코더의 설치 각도와 모터 기계적 각도의 보정, 그리고 인코더 자체의 보정이 포함됩니다. 이 보정 과정에서 모터는 천천히 한 각도만큼 정회전한 후, 다시 한 각도만큼 역회전합니다. 정회전만 하고 멈추면 오류가 있는 것이므로, 4단계를 통해 오류 코드를 확인하십시오.

➤ 4단계: 오류 코드 확인

3단계 인코더 보정 후 시스템의 오류 코드를 확인하십시오. 일반적으로 발생하는 오류는 ERROR_CPR_POLEPAIRS_MISMATCH이며, 이는 인코더의 CPR 설정 오류 또는 모터의 극쌍 수 설정 오류를 의미합니다. 다음 명령어를 통해 확인/설정하십시오:

```
odrv0.axis0.encoder.config.cpr odrv0.axis0.motor.config.pole_pairs
```

➤ 5단계: 모터 보정 성공 플래그 기록

➤ 6단계: 인코더 보정 성공 플래그 기록

➤ 7단계: 보정 결과 저장 및 재시작

```
odrv0.axis0.encoder.config.pre_calibrated = 1
odrv0.axis0.motor.config.pre_calibrated = 1 odrv0.save_configuration()
```

3.1.5 ID 수정

드라이버 ID는 버스에서 고유하며, 범위는 0~63이고 기본 ID는 0입니다. 사용자가 여러 대의 모터를 버스에 직렬로 연결한 경우, 각 모터에 서로 다른 ID를 설정해야 합니다. 사용자는 USB 또는 버스 방식을 통해 ID를 수정할 수 있습니다:

➤ 모터 엘프에서 ID 수정

모터 엘프를 열고 모터를 연결한 후, "모터 파라미터"를 클릭합니다. "통신 파라미터" 항목에서 ID를 수정하고, "동기화"를 클릭하여 드라이버에 저장합니다.



➤ USB를 통한 ID 수정

사용자가 odrivetool을 통해 드라이버에 연결한 후, 다음 명령어를 사용하여 ID를 수정할 수 있습니다:

```
odrv0.axis0.config.can.node_id = xxx
```

➤ 버스에서 ID 수정

4.1.2절의 Set_Axis_Node_ID 명령 메시지를 참조하십시오.

3.1.6 매개변수 저장 및 백업

파라미터를 수정한 후에는 반드시 저장해야 합니다. 그렇지 않으면 전원이 꺼지거나 재시작된 후 변경 사항이 사라집니다. 파라미터를 저장하면 드라이버가 재시작됩니다.

```
odrv0.save_configuration()
```

매개변수 백업:

```
odrivetool backup-config "d:/test.json"
```

여기서 "d:\test.json"은 사용자가 자유롭게 수정할 수 있는 저장 경로와 파일명입니다.

니다. 매개변수 복원 명령어는 다음과 같습니다:

```
odrivetool restore-config "d:/test.json"
```

3.1.7 네 가지 제어 모드

위의 준비 작업과 매개변수 설정을 마친 후에는 모터를 제어하여 다양한 모드로 회전시켜 볼 수 있습니다. 6010-8은 위치 제어, 속도 제어, 토크 제어 및 모션 제어 모드를 지원합니다.

위치 제어 모드에서는 필터링된 위치 제어(Filtered Position Control), 사다리꼴 곡선 위치 제어(Trajectory Control), 주기 위치 제어(Circular Position Control)를 지원합니다.

속도 제어 모드에서는 직접 속도 제어(Velocity Control), 경사 속도 제어(Ramped Velocity Control)를 지원합니다;

토크 제어 모드에서는 직접 토크 제어(Torque Control)와 램프 토크 제어(Ramped Torque Control)를 지원합니다.

모션 제어 모드는 위치, 속도 및 토크를 종합적으로 제어하는 모드로, 일반적으로 로봇의 무릎 관절과 같이 순간적인 폭발력이 필요한 상황에 사용됩니다. 업계에서는 이를 MIT 제어 모드라고 부르기도 하는데, 이 명칭은 MIT의 오픈소스 로봇 '로봇 개'에서 유래한 것으로, 해당 로봇이 모터를 제어하는 데 이 모션 제어 모드를 채택했기 때문입니다.

후속 각 제어 모드에 대한 상세 설명에서, 본 문서에서는 호스트 컴퓨터와 USB 제어 명령어를 예시로 사용하지만, 통신 프로토콜(예: CAN)을 사용하여 동일한 제어를 수행할 수도 있으며, 논리는 동일합니다.

모터를 어떻게 회전시키나요?

◆ 모터 시작(페루프 제어 상태 진입)

이후의 모든 제어 작업에서 모터를 회전시키려면 모터를 페루프 제어 상태로 전환해야 합니다. 모터 스프라이트에서는 "모터 시작"을 클릭하면 되며, odrivetool에서는 다음과 같은 명령어를 사용합니다:

```
odrv0.axis0.requested_state = 8
```

◆ 모터 정지(유휴 상태 진입)

사용자가 모터의 작동을 중지하거나, 파라미터를 수정 및 저장하려면 먼저 모터를 유휴 상태로 전환해야 합니다.

모터 스프라이트에서는 "모터 정지"를 클릭하면 됩니다. odrivetool에서는 다음과 같은 명령어를 사용합니다:

```
odrv0.axis0.requested_state = 1
```



모터 스프라이트

모터 위젯을 열면 하단에 네 개의 탭이 표시되며, 이는 각각 "속도 제어", "토크 제어", "위치 제어" 및 "MIT 모드"라는 네 가지 제어 모드를 나타냅니다.

➤ 인터페이스 기본 조작

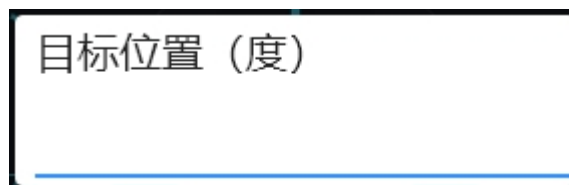
모터를 기동하면 다이얼이 선명하게 빛나며, 거의 모든 조작이 다이얼에서 이루어집니다:

✓ 다이얼 클릭

디스크를 클릭하면 속도/토크/위치를 현재 디스크 값으로 제어합니다. 클릭 한 번이 하나의 제어 명령을 의미합니다. 사용자는 연속해서 여러 번 클릭하여 모터에 대한 제어를 연속적으로 수행할 수 있습니다.

✓ 다이얼에서 오른쪽 클릭

다이얼에서 마우스 오른쪽 버튼을 클릭하면 나타나는 대화 상자에서 사용자는 정확한 목표 속도/토크/위치 값을 입력한 후 Enter 키를 눌러 제어를 실행할 수 있습니다. 대화 상자 밖을 클릭하면 이번 제어를 취소합니다.



✓ 다이얼을 누른 채 드래그

다이얼을 누른 채 드래그하면 연속 제어가 수행되며, 모터에 속도/토크/위치 제어 명령이 지속적으로 전송되고, 목표값은 현재 마우스 커서가 위치한 다이얼 값이 됩니다.

➤ 드라이버 매개변수 수정

“드라이브 파라미터”를 클릭하면 제어 모드, 속도 제한, 3차 루프 이득(대역폭) 등의 드라이브 파라미터를 조정할 수 있습니다. 먼저 모터를 정지시킨 후 파라미터를 수정하고 동기화하는 것이 좋습니다.



➤ 필터링된 위치 제어(Filtered Position Control)

사용자가 직접 위치 곡선을 생성하여 일정한 주파수로 위치 제어 명령을 전송하려는 경우, 필터링된 위치 제어를 사용하는 것이 좋습니다. 이 모드는 명령들을 부드럽게 연결하여 실행하기 때문입니다. 이러한 상황에서 사다리꼴 곡선 위치 제어를 사용하면 모터 회전 시 굽김 현상이나 거친 느낌이 발생할 수 있습니다.

이 모드에서는 전송되는 명령의 주파수에 따라 필터 대역폭을 조정해야 하며, 대역폭을 명령 주파수(단위 Hz)의 절반으로 설정하는 것이 좋은 경험칙입니다. 예를 들어 50Hz 주파수로 명령을 전송하는 경우:

```
odrv0.axis0.controller.config.input_filter_bandwidth = 25
```

필터 위치 제어 활성화:

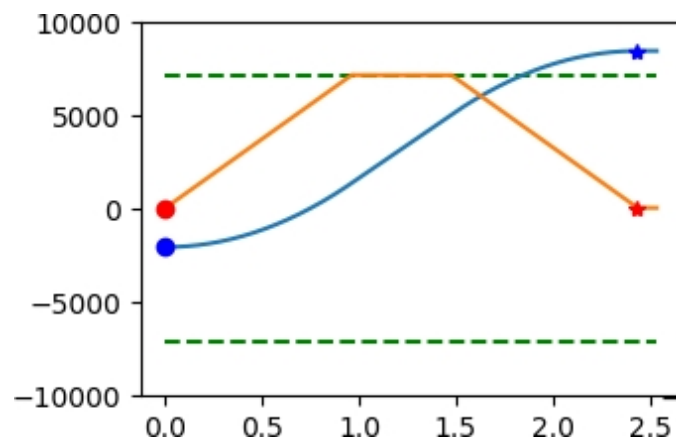
```
odrv0.axis0.controller.config.control_mode = 3
odrv0.axis0.controller.config.input_mode = 3
```

그런 다음 위치 제어 수행:

```
odrv0.axis0.controller.input_pos = 10 #단위: turns
```

➤ 사다리꼴 곡선 위치 제어(Trajectory Control)

이 모드를 사용하면 가속도, 항속 속도 및 감속도를 설정하여 모터가 한 위치에서 다른 위치로 부드럽게 이동하도록 제어할 수 있습니다. 이른바 "사다리꼴"이란 속도 곡선이 사다리꼴 모양을 띠는 것을 의미하며, 아래 그림과 같이 주황색은 속도, 파란색은 위치를 나타냅니다:



조절 가능한 제어 매개변수:

```
odrv0.axis0.traj_config.vel_limit #최대 크롤링 속도, 단위 turn/s
odrv0.axis0.traj_config.accel_limit #가속도 최대값, 단위 turn/s²
odrv0.axis0.traj_config.decel_limit #감속도 최대값, 단위 turn/s²
odrv0.axis0.controller.config.inertia #관성, 단위 Nm/(turn/s²)
```

관성 x 가속도 = 토크임을 유의하십시오. 이 값의 기본값은 0입니다. 이 값은 시스템 응답을 개선할 수 있지만, 모터의 부하와 직접적인 관련이 있습니다. 상기 네 가지 값은 모두 0 이상이어야 합니다. 또한 앞서 언급한 전류 임계값과 속도 임계값은 여전히 전역적으로 적용된다는 점에 유의해야 합니다. 예를 들어, 상기 슬라이딩 속도 최대값이 시스템 수준의 vel_limit보다 높게 설정된 경우, 적용되는 것은 전역 vel_limit입니다.

사다리꼴 곡선 제어 모드를 활성화하려면 먼저:

```
odrv0.axis0.controller.config.control_mode = 3
odrv0.axis0.controller.config.input_mode = 5
```

그런 다음 위치 제어를 수행합니다:

```
odrv0.axis0.controller.input_pos = 10 #단위: turns
```

➤ 순환 위치 제어(Circular Position Control)

이 모드는 연속적인 증분 위치 제어에 적합합니다. 예를 들어 로봇의 허브가 한 방향으로 일정 시간 동안 회전하거나 컨베이어 벨트가 계속 작동하는 경우, 일반적인 위치 제어 모드를 사용하면 목표 위치가 점차 매우 큰 값으로 증가하여 부동 소수점 정밀도 문제로 인해 위치 결정이 부정확해지는 오류가 발생할 수 있습니다.

활성화:

```
odrv0.axis0.controller.config.circular_setpoints = 1
```

이 모드에서는 각 작은 단계가 한 바퀴 범위 내에 있으며, input_pos의 범위는 [0, 1)입니다. input_pos가 이 범위를 초과하여 증가하면 한 바퀴 범위 내의 값으로 변환됩니다. 사용자가 한 단계에서 한 바퀴를 초과하기를 원한다면, 다음 매개변수를 1보다 큰 숫자로 설정할 수 있습니다:

```
odrv0.axis0.controller.config.circular_setpoint_range = <N>
```

➤ 직접 속도 제어(Velocity Control)

이 모드는 가장 간단한 속도 제어 방식이며, 다음과 같이 활성화할 수 있습니다:

```
odrv0.axis0.controller.config.control_mode = 2
odrv0.axis0.controller.config.input_mode = 1
```

그런 다음 목표 속도를 입력하여 제어합니다:

```
odrv0.axis0.controller.input_vel = 10 #단위 turn/s
```

➤ 경사 속도 제어(Ramped Velocity Control)

경사 속도 제어 모드는 일정한 경사도를 따라 속도를 목표값까지 점진적으로 높이는 방식으로, 앞서 언급한 직접 속도 제어보다 더 안전합니다. 활성화 방법은 다음과 같습니다:

```
odrv0.axis0.controller.config.control_mode = 2
odrv0.axis0.controller.config.input_mode = 2
```

가속도를 제어하기 위해 경사도를 조정합니다.

```
odrv0.axis0.controller.config.vel_ramp_rate = 0.5 #경사도 단위는 turn/s^2
```

다: 그런 다음 목표 속도를 입력하여 제어합니다.

니다:

```
odrv0.axis0.controller.input_vel = 10 #단위: turn/s
```

➤ 직접 토크 제어 (Torque Control)

이것은 가장 간단한 토크(전류) 제어 모드이며, 활성화 방법은 다음과 같습니다:

```
odrv0.axis0.controller.config.control_mode = 1
odrv0.axis0.controller.config.input_mode = 1
```

토크 제어의 단위는 Nm이며, 드라이버 펌웨어에서 전류 단위는 A이므로, 드라이버가 Nm을 전류로 변환하여 요구에 따라 모터가 토크를 출력할 수 있도록 토크 상수를 설정해야 합니다.

```
# 토크 상수는 대략 8.23Kv
odrv0.axis0.motor.config.torque_constant = 8.23/12.3
```

그런 다음 목표 속도를 입력하여 제어합니다:

```
odrv0.axis0.controller.input_torque = 1.2 # 단위 Nm
```

또한, 토크 모드에서 최대 속도를 제한하려면 enable_torque_mode_vel_limit을 활성화하고 vel_limit을 설정하면 됩니다. 예를 들어:

```
odrv0.axis0.controller.config.enable_torque_mode_vel_limit = 1
odrv0.axis0.controller.config.vel_limit = 30 #단위 turn/s
```

➤ 램프 토크 제어(Ramped Torque Control)

램프 토크 제어는 램프 속도 제어와 매우 유사하며, 활성화 방법은 다음과 같습니다:

```
odrv0.axis0.controller.config.control_mode = 1
odrv0.axis0.controller.config.input_mode = 6
```

경사도 조정:


```
odrv0.axis0.controller.config.torque_ramp_rate = 0.1 # 경사도 단위는 Nm/s
```

➤ 모션 제어 (MIT Control)

모션 제어 모드는 위치, 속도 및 토크를 종합적으로 제어하여 모터가 목표 위치로 이동하도록 제어하며, 다음 공식으로 표현할 수 있습니다:

$$T_{target} = K_p \times P_{diff} + K_d \times V_{diff} + T_{ff}$$

$$P_{diff} = P_{target} - P_{current}$$

$$V_{diff} = V_{target} - V_{current}$$

여기서 T_{target} 는 목표 토크, P_{diff} 는 위치 오차, V_{diff} 는 속도 오차, K_p 는 위치 제어 이득, K_d 는 속도 제어 이득(또는 감쇠 계수)이며, T_{ff} 는 피드포워드 토크이다.

모션 제어 모드 활성화 방법은 다음과 같습니다:

```
odrv0.axis0.controller.config.control_mode = 3
odrv0.axis0.controller.config.input_mode = 9
```

이득 조정:

```
odrv0.axis0.controller.input_mit_kp = <float> # 위치 이득, 단위 Nm/turn
odrv0.axis0.controller.input_mit_kd = <float> # 감쇠 계수, 단위 Nm/turn/s

odrv0.axis0.controller.input_vel = 30 #단위: turn/s
odrv0.axis0.controller.input_torque = 2 #단위: Nm
```

그런 다음 input_pos, input_vel, input_torque를 입력하여 모션 제어를 수행합니다:

주의: USB 제어 시 입력하는 위치, 속도 및 토크는 모두 로터 측을 가리키는 반면, CAN을 이용한 MIT 제어 시 프로토콜의 위치, 속도 및 토크는 모두 구동축 측을 가리킵니다. 이는 MIT 오픈소스 프로토콜과의 호환성을 유지하기 위함입니다!

3.1.8 고급

1) 자주 사용되는 명령어 목록

연결에 성공한 후, 사용자는 명령어를 통해 모터를 제어하고 모터 작동 매개변수를 확인할 수 있습니다. 다음 표는 자주 사용되는 명령어와 테스트 절차 및 설명입니다:

유형	명령	설명
기본 명령어	dump_errors(odrv0)	모든 오류 메시지를 출력합니다
	odrv0.clear_errors()	모든 오류 정보를 지웁니다
	odrv0.save_configuration()	매개변수를 수정했거나, 모터 자동 인식 매개변수 설정 또는 보정 후, 반드시 이 명령을 실행하여 변경 사항을 저장하십시오. 그렇지 않으면 전원이 꺼지면 모든 수정 내용이 손실됩니다.
	odrv0.reboot()	드라이버 재시작
	odrv0.vbus_voltage	전원 전압(V) 가져오기
	odrv0.ibus	전원 전류(A) 가져오기
	odrv0.hw_version_major	하드웨어 메이저 버전 번호, 6010-8 현재 메이저 버전 번호는 3

	odrv0.hw_version_minor	하드웨어 부 버전 번호, 6010-8 현재 부 버전 번호는 8
	odrv0.hw_version_variant	동일한 하드웨어 구성 내의 서로 다른 모델 번호, 6010-8에 해당 모델 번호는 1
	odrv0.can.config.r120_gpio_num	CAN 인터페이스의 120R 매칭 저항 스위치를 제어하는 GPIO 번호
	odrv0.can.config.enable_r120	CAN 인터페이스의 120R 매칭 저항 스위치를 제어합니다
	odrv0.can.config.baud_rate	CAN의 보드 속도 설정
매개 변수 설정 명령어	odrv0.config.dc_bus_undervoltage_trip_level	저전압 경고 임계값 (V)
	odrv0.config.dc_bus_overvoltage_trip_level	과전압 경고 임계값 (V)
	odrv0.config.dc_max_positive_current	선 전류 최대값(양극)(A)
	odrv0.config.dc_max_negative_current	선 전류 역방향 충전 최대값(음수)(A)
	odrv0.axis0.motor.config.resistance_calib_max_voltage	모터 파라미터 식별 시 최대 전압값. 일반적으로 이 값은 전원 전압의 절반보다 약간 작으며, 예를 들어 24V 전원을 사용할 경우 10
	odrv0.axis0.motor.config.calibration_current	모터 파라미터 인식 시 최대 전류 값으로, 이 값은 일반적으로 2~5A로 설정할 수 있으며, 너무 크거나 작아서는 안 됩니다.
	odrv0.axis0.motor.config.torque_constant	모터의 토크 상수(Nm/A)
	odrv0.axis0.min_endstop.config odrv0.axis0.max_endstop.config	최소(LW1)/최대(LW2) 리미트 스위치 설정: enabled: 활성화 여부 gpio_num: 해당 IO 번호. 최소 리미트 스위치의 IO 번호를 1로, 최대 리미트 스위치의 IO 번호를 2로 설정하십시오
	odrv0.axis0.encoder.config.index_offset	사용자가 설정한 제로 오프셋으로, 이 값은 사용자 제로 점을 인코더 제로점에 대한 오프셋 값입니다. 이 오프셋 값을 설정하고 저장한 후, 사용자가 입력하는 모든 비트 위치 제어 목표값은 모두 이 사용자 제로점을 기준으로 합니다.
	odrv0.axis0.motor.motor_thermistor.config	모터 온도 센서 구성: enabled: 활성화 여부 temp_limit_lower: 온도 하한 temp_limit_upper: 온도 상한
	odrv0.axis0.motor.fet_thermistor.config	
	odrv0.axis0.motor.motor_thermistor.temperature	모터 온도
	odrv0.axis0.motor.fet_thermistor.temperature	드라이버 온도
보정 명령	odrv0.axis0.requested_state=4	모터 파라미터 식별 수행, 여기에는 상 저항, 상 인덕턴스 식별 및 3상 전류 균형에 대한  보정

		합니다. 이 과정은 3~6초 정도 소요되며, 모터에서  날카로운 소리를 냅니다. 소리가 멈춘 후, 또는 6초가 지나도 소리가 나지 않을 경우, dump_errors(odrv0)를 실행하여 오류를 확인하고, 오류가 없음을 확인한 후 다른 작업을 합니다.
	odrv0.axis0.requested_state=7	인코더를 보정합니다. 이 작업을 수행하기 전에 모터 출력축에 부하가 전혀 없는지 확인하고, 손이나 다른 장치로 모터를 고정하십시오. 이 작업이 수행되면 모터가 정방향 및 역방향으로 일정 시간 회전하며 인코더를 인식하고 보정합니다. 모터가 정지한 후, dump_errors(odrv0)를 실행하여 오류를 확인하고, 오류가 없음을 없음을 확인한 후 다른 후속 단계를 진행하십시오.
	odrv0.axis0.encoder.config.pre_calibrate d=1	사전 보정 쓰기가 성공하면, 매번 전원을 켤 때마다 보정할 필요가 없음을 의미합니다. 이 매개변수는 위에서 언급한 보정이 성공한 후에만 작성할 수 있으며, 그렇지 않으면 작성에 실패합니다.
	odrv0.axis0.controller.config.load_encoder_axis=0	현재 작동 중인 모터가 0번 모터인지 확인합니다. 이 작업은 BETA 버전에서만 필요하며, 양산 버전에서는 무효입니다.
제어 명령	odrv0.axis0.requested_state=1	모터를 정지시키고 유휴 상태로 전환
	odrv0.axis0.requested_state=8	모터 시작, 페루프 상태로 진입
	odrv0.axis0.motor.config.current_limit	모터 구동 시 최대 라인 전류(A). 이 값을 초과하면 과전류 경보가 발생합니다. 이 값은 100을 초과해서는 안 됩니다.
	odrv0.axis0.controller.config.vel_limit	모터 작동 최대 속도(turn/s), 모터 로터의 이 값을 초과하면 과속 경보가 발생합니다.
	odrv0.axis0.controller.config.enable_vel_limit	속도 제한 스위치, True일 때 상기 vel_limit이, False일 때는 무효입니다.
	odrv0.axis0.controller.config.control_mode	제어 모드. 0: 전압 제어 1: 토크 제어 2: 속도 제어 3: 위치 제어
	odrv0.axis0.controller.config.input_mode	입력 모드. 사용자가 입력한 제어값이 어떤 방식으로 모터 작동을 제어하는지를 나타냅니다: 0: 유휴 1: 직접 제어 2: 속도 경사 3: 위치 필터링 5: 사다리꼴 곡선 6: 토크 램프 9: 모션 제어(MIT)
	odrv0.axis0.controller.config.vel_gain	속도 루프 PID 제어의 P 값
	odrv0.axis0.controller.config.vel_integrator_gain	속도 루프 PID 제어의 I 값
	odrv0.axis0.controller.input_mit_kp	모션 제어(MIT) 위치 이득

odrv0.axis0.controller.input_mit_kd	모션 제어(MIT) 속도 이득(댐핑 계수)
odrv0.axis0.controller.config.pos_gain	위치 루프 PID 제어의 P 값
odrv0.axis0.controller.input_torque	토크 제어의 목표, 또는 속도 제어/위치 제어의 토크 피드포워드(Nm)
odrv0.axis0.controller.input_vel	속도 제어의 목표값, 또는 위치 제어의 속도 피드포워드 (turn/s)
odrv0.axis0.controller.input_pos	위치 제어 목표 (턴)
odrv0.axis0.encoder.set_linear_count()	인코더의 절대 위치를 설정합니다. 괄호 안에 32 비트 정수, 이 정수의 절대값은 odrv0.axis0.encoder.config.cpr
odrv0.axis0.trap_traj.config	세 가지 매개변수를 포함합니다: ➤ accel_limit: 최대 가속도 (rev/s²) ➤ decel_limit: 최대 감속도 (rev/s²) ➤ vel_limit: 최대 속도 (rev/s) 이 세 가지 매개변수는 odrv0.axis0.controller.config.input_mode 가 사다리꼴 곡선일 때 적용되며, 위치 제어의 가속 및 감속 효과를 조정합니다.
odrv0.axis0.controller.config.input_filter_bandwidth	위치 필터 대역폭. 이 매개변수는 odrv0.axis0.controller.config.input_mode 가 위치 필터일 때 작동하며, 위치 제어의 가속 및 감속 효과를 조절합니다.

2) 그래픽 기반 튜닝

모터를 튜닝할 때 특정 작동 매개변수를 실시간으로 모니터링해야 하는 경우, Python의 강력한 계산 라이브러리와 그래픽 라이브러리, 그리고 Type-C 인터페이스의 고속 전송 능력을 활용하여 모터 매개변수를 실시간으로 출력할 수 있습니다.

1. 환경 준비

계산 라이브러리와 그래픽 라이브러리 설치:

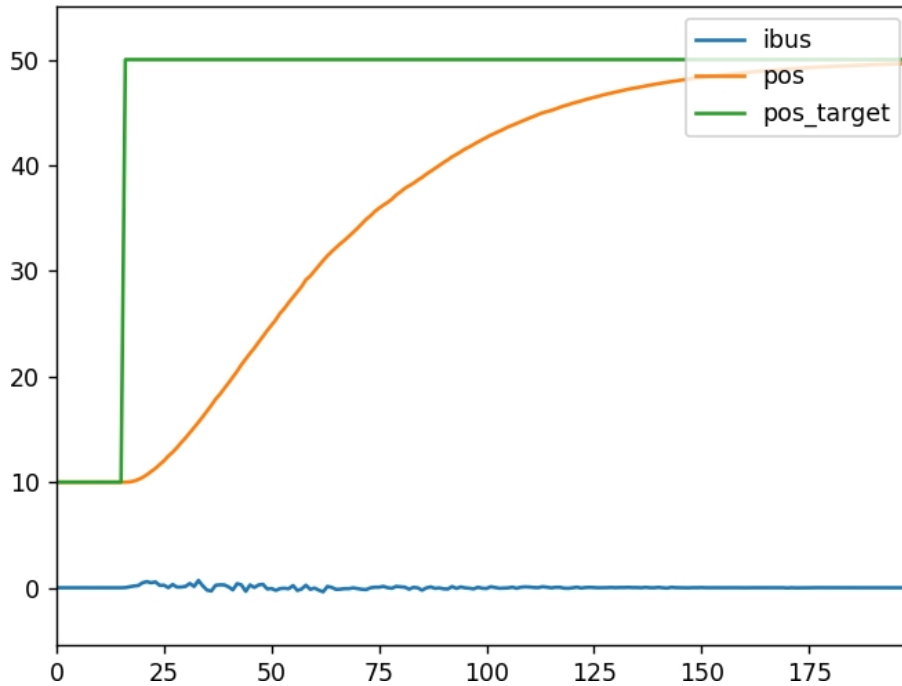
```
pip install numpy matplotlib
```

2. 그래픽 기반 파라미터 출력

odrivetool 명령줄 인터페이스에서 그래픽 라이브러리를 호출하고, 모터 작동 지표(예:

```
start_liveplotter(lambda:[odrv0.ibus,odrv0.axis0.encoder.pos_estimate,
odrv0.axis0.controller.input_pos],["ibus","pos","pos_target"])
```

이 명령은 그래픽 인터페이스를 실행하여 다음 세 가지 지표(선 전류, 위치, 목표 위치)를 실시간으로 출력합니다. 이어서 모터의 위치 제어를 수행하면 모터의 실시간 위치 제어 곡선을 확인할 수 있습니다:



3) USB 및 CAN 호환성

본 제품의 초기 버전(하드웨어 버전 3.7 이하, 다음 소절 3.1.8의 명령어 odrv0.hw_version_major 및 odrv0.hw_version_minor를 통해 확인 가능)에서는 USB와 CAN이 호환되지 않으며, 다음 방법을 통해 두 통신 방식 간에 전환할 수 있습니다(하드웨어 버전 3.7 이상인 경우 이 절의 내용은 무시해도 됩니다):

➤ USB 통신 시 CAN으로 전환

CAN이 비활성화된 상태에서 사용자는 Type-C 인터페이스를 통해 통신할 수 있으며, 이때 다음 명령어를 사용하여 CAN으로 전환할 수 있습니다:

```
odrv0.config.enable_can_a = True
odrv0.axis0.requested_state = AXIS_STATE_IDLE
odrv0.save_configuration()
```

➤ CAN 통신 시 USB로 전환

CAN을 활성화할 때, 사용자는 먼저 Set_Axis_State 메시지(매개변수 1, 유휴(IDLE) 상태 표시)를 전송하여 모터를 유휴 상태로 전환한 다음, Disable_Can 메시지를 전송하여 USB로 전환합니다(4.1.2 참조).

USB에서 CAN으로 전환하든, CAN에서 USB로 전환하든, 반드시 먼저 모터를 유휴 (IDLE) 상태로 만들어야 하며, 그렇지 않으면 전환에 실패합니다.

4) CAN 매칭 저항 스위치

드라이버에는 120Ω 임피던스 매칭 저항이 이미 내장되어 있으며, 사용자는 필요에 따라 이를 켜거나 끌 수 있습니다. 명령어 예시는 다음과 같습니다:

```
odrv0.can.config.r120_gpio_num = 5
odrv0.can.config.enable_r120 = True
```

5) 사용자 제로점 설정

기본적으로 사용자가 모터에서 읽어오는 위치 및 위치 제어 시의 입력 값은 모두 드라이버에 장착된 절대값 인코더의 제로 점을 기준으로 합니다. 그러나 실제 사용 환경에서는 인코더의 제로점이 대부분의 경우 사용자 제로점과 일치하지 않으므로, 사용자는 이 제로점 오프셋을 수동으로 설정해야 합니다.

일반적으로 사용자는 두 가지 방법으로 이 제로점을 설정할 수 있습니다. 하나는 리미트 스위치를 사용하는 것이고, 다른 하나는 제로점 오프셋, 즉 인코더 제로점에 대한 사용자 제로점의 오프셋 값을 수동으로 설정하는 것입니다:

```
# 사용자가 먼저 수동으로 또는 위치 제어를 통해 원하는 사용자 제로점 위치로 이동한 후:
odrv0.axis0.encoder.config.index_offset = odrv0.axis0.encoder.pos_estimate
```

6) 리미트 스위치

드라이버는 두 개의 리미트 스위치(LW1 및 LW2)를 지원하며, LW1은 최소 위치이자 제로 위치이고, LW2는 최대 위치입니다. 두 개의 리미트 스위치를 사용하려면 다음 구성을 사용하십시오:

```
odrv0.axis0.min_endstop.config.enabled = True odrv0.axis0.min_endstop.config.gpio_num = 1
odrv0.axis0.max_endstop.config.enabled = True odrv0.axis0.max_endstop.config.gpio_num = 2
```

리미트 스위치가 작동되면 시스템은 MIN_ENDSTOP_PRESSED 또는 MAX_ENDSTOP_PRESSED 오류를 보고하며, 호스트 컴퓨터는 이때 관련 작업을 수행할 수 있습니다.

하드웨어 버전 3.7은 리미트 스위치 기능을 지원하지 않으므로 주의하십시오.

7) 브레이크 저항

사용자는 2.4.7에 설명된 제동 저항 인터페이스(EMG 및 GND) 사이에 제동 저항(방전 저항이라고도 함)을 연결하여, 버스 전류가 전원 공급 장치로 역류할 때 분로 전류를 소비할 수 있습니다.

사용자는 다음 구성을 통해 이 기능을 활성화할 수 있습니다.

```
odrv0.config.enable_brake_resistor = True odrv0.config.brake_resistance =
xxx # 외부 브레이크 저항 값 설정 odrv0.save_configuration()
```

드라이버가 자동으로 방전되도록 하려면 어떻게 해야 할까요? 다음 두 가지 방법을 조합하여 컨트롤러가 자동으로 방전 전류를 조절하도록 할 수 있습니다:

➤ 최대 역전류를 조정하여

```
odrv0.config.max_regen_current = xxx # 역전류 최대값, 양수
```

역전류가 위 설정값보다 클 경우, 드라이버는 초과 전류를 방전 저항으로 분로시킵니다.

➤ 역기전력 전압 범위 조정

```
odrv0.config.enable_dc_bus_overvoltage_ramp = True
odrv0.config.dc_bus_overvoltage_ramp_start = xxx # 분로 전압 하한
odrv0.config.dc_bus_overvoltage_ramp_end = xxx # 분로 전압 상한
```

역기전력에 의해 발생하는 전압이 상기 하한보다 높을 때, 드라이버는 방전 저항으로 전류를 분산시키기 시작합니다. 전압이 상기 상한 이상일 때, 드라이버는 모든 전압을 방전 저항에 가하여 분산을 시도합니다. 전압이 상기 하한과 상한 사이일 때, 드라이버는 비례적으로 전원 전압(역기전력 전압)을 방전 저항에 가합니다.

참고로, 상기 두 모드는 동시에 활성화될 수 있으며, 드라이버는 두 방식을 종합하여 방전을 시도합니다.

3.2 펌웨어 업데이트 다운로드

SWD 인터페이스(2.4.4) 또는 Type-C 인터페이스(2.4.2)를 통해 펌웨어를 플래시할 수 있으며, 다음 세 가지 방법이 제공됩니다:

3.2.1 국민 다운로드 소프트웨어

1. USB(DFU) 플래싱

참고: 국민 다운로드 소프트웨어는 Type-C 인터페이스를 통해 펌웨어를 플래시할 수 있으며, SWD 인터페이스(JLink 및 DAP만 지원)를 통해서도 플래시할 수 있습니다. 본 절에서는 주로 Type-C 인터페이스를 통한 플래시를 예시로 다룹니다.

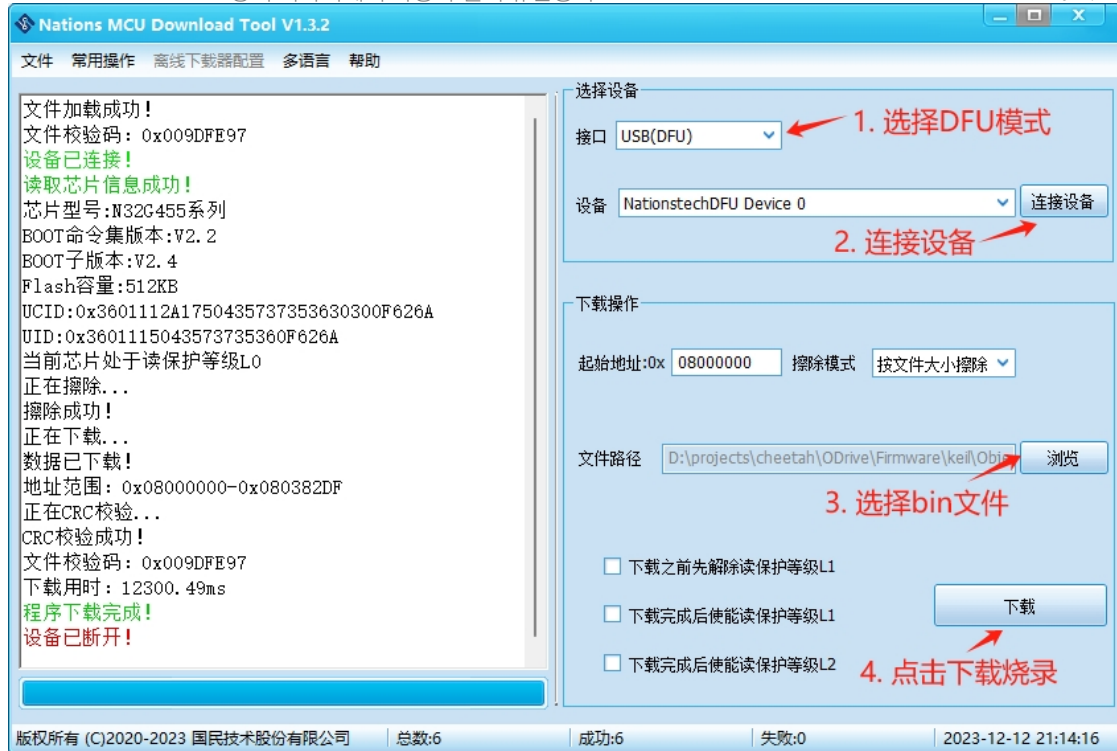
먼저, 국민 펌웨어 갱신 소프트웨어의 USB 드라이버(<https://www.cyberbeast.cn/filedownload/789489>)를 다운로드하여 해당 시스템의 드라이버를 설치하십시오. 그런 다음, 국민 펌웨어 갱신 소프트웨어

(<https://cyberbeast.cn/filedownload/766844>)을 다운로드하여 원하는 디렉터리에 압축을 풀고 실행하십시오.

그 다음, Type-C 인터페이스를 연결하고 odrivetool로 이동한 후, 다음 명령어를 실행하여 드라이버를 DFU 모드로 설정합니다:

```
odrv0.enter_dfu_mode()
```

마지막으로, 국민 플래싱 소프트웨어를 사용하여 플래싱을 수행합니다(아래 그림 참조). 참고: 플래싱이 완료된 후, "일반 작업"을 클릭한 다음 "리셋"을 클릭하면 드라이버를 재시작할 수 있으며, odrivetool을 통해 정상적으로 연결 및 테스트할 수 있습니다.



2. SWD(JLink 또는 DAP) 펌웨어 쓰기

SWD 방식을 이용한 다운로드 방법은 DFU 모드와 유사하지만, SWD 디버깅 인터페이스(2.4.4)를 통해 연결해야 하며, 위 그림에서 해당 디버깅 도구(JLink 또는 DAP)를 선택해야 합니다.

3.2.2 pyocd

pyocd는 openOCD의 파이썬 버전으로, STLink, JLink, DAP 등 범용 디버깅 도구를 통해 지우기, 펌웨어 쓰기, 리셋 등의 작업을 수행할 수 있습니다. 반드시 SWD 인터페이스를 통해 드라이버를 연결해야 합니다. SWD의 배선 순서에 대해서는 다음을 참조하십시오.

2.4.4항을 참조하십시오. **SWD 인터페이스에는 3.3V 전원이 공급되므로, 드라이버가 손상되지 않도록 배선 순서를 잘못 연결하지 마십시오!**

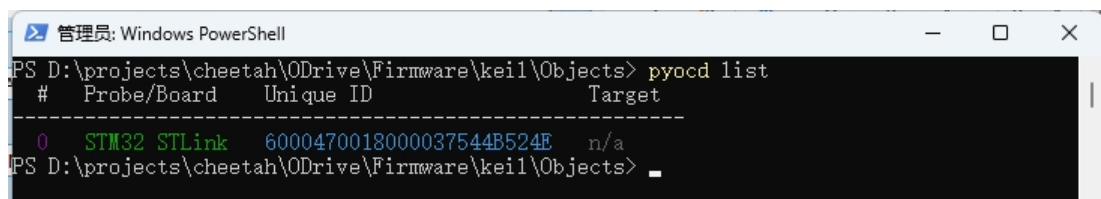
1. 설치

```
pip install pyocd
```

2. 프로그래밍

먼저, 연결된 디버깅 도구를 나열합니다:

```
pyocd list
```



그런 다음, 다음 명령어를 실행하여 bin 파일을 업로드합니다:

```
pyocd load .\ODrive_N32G455.bin -a 0x8000000
```

```
PS D:\projects\cheetah\ODrive\Firmware\keil\Objects> pyocd load .\ODrive_N32G455.bin -a 0x8000000
0000477 W Generic 'cortex_m' target type is selected by default; is this intentional? You will be able
to debug most devices, but not program flash. To set the target type use the '--target' argument or
'target_override' option. Use 'pyocd list --targets' to see available targets types. [board]
0000529 I Loading D:\projects\cheetah\ODrive\Firmware\keil\Objects\ODrive_N32G455.bin at 0x08000000 [1
load_cmd]
[=====] 100%
0004543 I Erased 0 bytes (0 sectors), programmed 230112 bytes (0 pages), skipped 0 bytes (0 pages) at
56.02 kB/s [loader]
PS D:\projects\cheetah\ODrive\Firmware\keil\Objects> _
```

4 통신 프로토콜 및 예시

4.1 CAN Simple 프로토콜

기본 통신 인터페이스는 CAN이며, 최대 통신 속도는 1Mbps(odrv0.can.config.baud_rate를 통해 읽기 및 설정 가능)이고, 기본 속도는 500Kbps입니다. **주의: 초기 하드웨어 버전(3.7 이하)에서는 USB와 CAN이 호환되지 않으므로, 3.1.8절의 3)항을 참조하여 USB에서 CAN으로 전환하는 방법을 확인하십시오.**

드라이버의 기본 프로토콜은 CAN Simple입니다. 사용자가 CANOpen 프로토콜로 전환한 경우, 다음 명령을 통해 CAN Simple로 다시 전환해야 합니다:

```
odrv0.can.config.protocol = 1
```

버전이 3.10 이상인 하드웨어 버전에서는 통신 인터페이스가 RS485, 펄스 방향 및 PWM을 동시에 지원합니다. 다음 명령어를 통해 통신 인터페이스가 CAN인지 확인하십시오:

```
odrv0.config.comm_intf_mux = 0
```

Motor Genie에서는 통신 매개변수에서 물리적 인터페이스와 CAN 프로토콜 버전을 전환할 수도 있습니다.

4.1.1 프로토콜 프레임 형식

CAN 통신은 표준 프레임 형식을 채택하며, 데이터 프레임은 11비트 ID와 8바이트 데이터로 구성됩니다. 아래 표와 같습니다(왼쪽이 MSB, 오른쪽이 LSB):

데이터 영역	CAN ID(11비트)		데이터(8바이트)
세그먼트	Bit10 ~ Bit5	4비트 ~ 0비트	바이트 0 ~ 바이트 7
설명	node_id	cmd_id	통신 데이터

- node_id: 이 모터가 버스에서 가지는 고유 ID를 나타내며, odrivetool에서 odriv0.axis0.config.can.node_id를 사용하여 읽거나 설정할 수 있습니다.
- cmd_id: 명령어 코드이며, 프로토콜의 메시지 유형을 나타냅니다. 이 섹션의 나머지 내용을 참조하십시오.

- 통신 데이터: 8바이트, 각 메시지에 포함된 매개변수는 정수 또는 부동 소수점으로 인코딩되며, 바이트 순서는 리틀 엔디안(little endian)입니다. 부동 소수점은 IEEE 754 표준에 따라 인코딩됩니다(웹사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 인코딩을 테스트할 수 있습니다).

4.1.2절에서 설명한 Set_Input_Pos 메시지를 예로 들면, 세 개의 매개변수가 각각 Input_Pos=3.14, Vel_FF=1000(1rev/s를 나타냄), Torque_FF=5000(5Nm를 나타냄)이며, Set_Input_Pos 메시지의 CMD ID=0x00C이고, 드라이버의 노드 ID(node_id)가 0x05로 설정되었다고 가정하면:

- 11비트 CAN ID=(0x05<<5)+0x0C=0xAC
- 4.1.2절의 Set_Input_Pos에 대한 설명에 따르면, Input_Pos는 0번째 바이트부터 시작하는 4바이트이며, C3 F5 48 40(IEEE 754 표준에 따라 32비트 수로 인코딩된 부동소수점 수 3.14 0x4048f5c3), Vel_FF는 4번째 바이트부터 시작하는 2바이트로, E8 03 (1000=0x03E8)로 인코딩되며, Torque_FF는 6번째 바이트부터 시작하는 2바이트로 88 13 (5000=0x1388)로 인코딩되며, 8바이트의 통신 데이터는 다음과 같습니다:

Byte0	Byte1	바이트2	바이트3	바이트 4	바이트 5	바이트 6	바이트 7
C3	F5	48	40	E8	03	88	13

4.1.2 프레임 메시지

다음 표에는 사용 가능한 모든 메시지가 나열되어 있습니다:

CMD ID	이름	방향	매개변수
0x001	하트비트	모터→호스트	Axis_Error Axis_State Motor_Flag Encoder_Flag Controller_Flag Traj_Done Life
0x002	Estop	호스트→모터	
0x003	Get_Error	모터→호스트	Error_Type
0x004	RxSdo	모터→호스트	
0x005	TxSdo모터 xml-ph-0000@deepi.internal 호스트	모터→호스트	
0x006	Set_Axis_Node_ID	호스트→모터	Axis_Node_ID
0x007	Set_Axis_State	호스트→모터	Axis_Requested_State
0x008	Mit_Control	호스트→모터	
0x009	Get_Encoder_Estimates	모터→호스트	Pos_Estimate Vel_Estimate
0x00A	Get_Encoder_Count	모터→호스트	Shadow_Count Count_In_Cpr
0x00B	Set_Controller_Mode	호스트→모터	Control_Mode Input_Mode
0x00C	Set_Input_Pos	호스트→모터	Input_Pos
			Vel_FF Torque_FF

0x00D	Set_Input_Vel	호스트→모터	Input_Vel Torque_FF
0x00E	Set_Input_Torque	호스트→모터	Input_Torque
0x00F	Set_Limits	호스트→모터	Velocity_Limit 전류 제한
0x010	Start_Anticogging	호스트→모터	
0x011	Set_Traj_Vel_Limit	호스트→모터	Traj_Vel_Limit
0x012	Set_Traj_Accel_Limits	호스트→모터	Traj_Accel_Limit Traj_Decel_Limit
0x013	Set_Traj_Inertia	호스트→모터	Traj_Inertia
0x014	Get_Iq	모터→호스트	Iq_Setpoint Iq_Measured
0x016	재부팅	호스트→모터	
0x017	Get_Bus_Voltage_Current	모터→호스트	Bus_Voltage Bus_Current
0x018	Clear_Errors	호스트→모터	
0x019	Set_Linear_Count	호스트→모터	Linear_Count
0x01A	Set_Pos_Gain	호스트→모터	Pos_Gain
0x01B	Set_Vel_Gains	호스트→모터	Vel_Gain Vel_Integrator_Gain
0x01C	Get_Torques	모터→호스트	Torque_Setpoint 토크
0x01D	Get_Powers	모터→호스트	Electrical_Power Mechanical_Power
0x01E	Disable_Can	호스트→모터	
0x01F	Save_Configuration	호스트→모터	

모든 메시지에 대한 자세한 설명은 다음과 같습니다:

➤ 하트비트

CMD ID: 0x001 (모터→호스트)

펌웨어 버전이 0.5.11 미만(포함)인 하트비트 형식은 다음과 같습니다:

시작 바이트	이름	유형	설명
0	Axis_Error	uint32	드라이버 오류 코드 (odrv0.axis0.error)
4	Axis_State	uint8	드라이버 상태 (odrv0.axis0.current_state)
5	Motor_Flag	uint8	1: 모터 이상 (odrv0.axis0.motor.error가 0이 아님) 0: 모터 정상 (odrv0.axis0.motor.error가 0임)
6	Encoder_Flag	uint8	1: 인코더 오류 (odrv0.axis0.encoder.error가 0이 아님)
			0: 인코더 정상 (odrv0.axis0.encoder.error가 0)

7	Controller_Flag	uint8	bit7: odrv0.axis0.controller.trajectory_done, 즉 위치 곡선 실행 완료 여부 bit0: 1: 제어 이상 (odrv0.axis0.controller.error가 0이 아님) 0: 제어 정상 (odrv0.axis0.controller.error가 0임)
---	-----------------	-------	--

펌웨어 버전이 0.5.12 이상인 경우의 심박수 형식은 다음과 같습니다:

시작 바이트	이름	유형	설명
0	Axis_Error	uint32	드라이버 오류 코드 (odrv0.axis0.error)
4	Axis_State	uint8	드라이버 상태 (odrv0.axis0.current_state)
5	플래그	uint8	bit0: 모터 오류 비트 (odrv0.axis0.motor.error가 0인지 여부) bit1: 인코더 오류 비트 (odrv0.axis0.encoder.error가 0인지 여부) bit2: 제어 이상 비트 (odrv0.axis0.controller.error가 0인지 여부) bit7: odrv0.axis0.controller.trajectory_done, 즉 위치 곡선 실행 완료 여부
6	예약	uint8	예약
7	Life	uint8	심박수 메시지의 수명 값으로, 각 심박수 메시지마다 1이 증가하며, 범위는 0~255입니다. 이 수명 값이 연속적이지 않다면 심박수 메시지가 누락되었음을 , 즉 통신이 불안정함을 의미합니다.

펌웨어 버전이 0.5.13 이상(포함)인 경우의 하트비트 형식은 다음과 같습니다:

시작 바이트	이름	유형	설명
0	Axis_Error	uint32	드라이버 오류 코드 (odrv0.axis0.error)
4	Axis_State	uint8	드라이버 상태 (odrv0.axis0.current_state)
5	플래그	uint8	bit0: 모터 오류 비트 (odrv0.axis0.motor.error가 0인지 여부) bit1: 인코더 오류 비트 (odrv0.axis0.encoder.error가 0인지 여부) bit2: 제어 이상 비트 (odrv0.axis0.controller.error가 0인지 여부) bit3: 시스템 오류 비트 (odrv0.error가 0인지 여부) bit7: odrv0.axis0.controller.trajectory_done, 즉 위치 곡선 실행 완료 여부
6	예약	uint8	예약
7	Life	uint8	주기 메시지의 수명 값으로, 각 하트비트 메시지마다 1이 증가하며, 범위는 0~255입니다. 이 수명 값이 연속적이지 않다면 하트비트 메시지가 누락되었음을 , 즉 통신이 불안정함을 의미합니다.

➤ Estop

CMD ID: 0x002(호스트→ 모터) 매개변수 없음, 데이터 없음.

이 명령은 모터를 긴급 정지시키고 ESTOP_REQUESTED 예외를 보고합니다.

➤ Get_Error

CMD ID: 0x003(모터→ 호스트)

입력(호스트→ 모터):

시작 바이트	이름	유형	설명
0	Error_Type	uint8	0: 모터 읽기 오류 1: 인코더 오류 발생 3: 제어 오류 발생 4: 시스템 오류 확인

➤ 출력 (모터→ 호스트):

시작 바이트	이름	유형	설명
0	오류	uint32/uint64	입력된 Error_Type에 따라 반환되는 데이터 및 길이: 0: 모터 오류 (odrv0.axis0.motor.error), uint64, 8바이트 1: 인코더 오류 (odrv0.axis0.encoder.error), uint32, 4바이트 3: 제어 오류 (odrv0.axis0.controller.error), uint32, 4바이트 4: 시스템 오류 (odrv0.error), uint32, 4바이트

➤ RxSdo

CMD ID: 0x004 (호스트→ 모터)

입력:

시작 바이트	이름	유형	설명
0	opcode	uint8	0: 읽기 1: 쓰기
1	Endpoint_ID	uint16	모든 매개변수 및 인터페이스 함수에 해당하는 ID가 포함된 JSON 파일을 다운로드하십시오: ➤ 버전 0.5.13
			https://bl.cyberbeast.cn/actuator/endpoints_0.5.13.js ➤ 버전 0.5.14 https://bl.cyberbeast.cn/actuator/endpoints_0.5.14.js on
3	예비	uint8	

4	값	uint8[4]	Endpoint_ID에 따라 다르며, 상기 JSON의 설명을 참조하십시오. 예를 들어 Endpoint_ID가 읽기/쓰기 가능한 float 값에 대응하는 경우, 이곳의 4바이트는 IEEE 인코딩된 float 값이며, opcode=1일 때 이 값을 해당 float 값에 기록합니다.
---	---	----------	---

출력 (상기 opcode=0일 때):

시작 바이트	이름	유형	설명
0	opcode	uint8	0으로 고정
1	Endpoint_ID	uint16	모든 매개변수 및 인터페이스 함수에 해당하는 ID가 포함된 JSON 파일을 다운로드하십시오: ➤ 버전 0.5.13 https://bl.cyberbeast.cn/actuator/endpoints_0.5.13.js ➤ 버전 0.5.14 https://bl.cyberbeast.cn/actuator/endpoints_0.5.14.js on
3	예비	uint8	
4	값	uint8[4]	Endpoint_ID에 따라 다르며, 상기 JSON의 설명을 참조하십시오. 예를 들어 Endpoint_ID가 읽을 수 있는 uint32 값에 해당한다면, 여기서의 4바이트는 리틀 엔디안 형식의 uint32입니다.

➤ TxSdo

CMD ID: 0x005(모터→ 호스트)

사용법은 opcode=1일 때의 RxSdo와 동일합니다.

➤ Set_Axis_Node_ID

CMD ID: 0x006 (호스트→ 모터)

시작 바이트	이름	유형	설명
0	Axis_Node_ID	uint32	odrv0.axis0.config.can.node_id CAN 버스에서 모터의 고유 ID

➤ Set_Axis_State

CMD ID: 0x007 (호스트→ 모터)

시작 바이트	이름	유형	설명
0	Axis_Requested_State	uint32	odrv0.axis0.requested_state ➤ 0: 정의되지 않음 ➤ 1: 유힬 ➤ 3: 보정(모터 보정 + 인코더 보정)) ➤ 4: 모터 보정 ➤ 7: 인코더 보정 ➤ 8: 페루프

➤ Mit_Control

CMD ID: 0x008

이는 MIT 오픈소스 모션 제어 프로토콜(<https://github.com/mit-biomimetics/Cheetah-Software>)를 구현한 것입니다.

USB 제어 시 입력되는 위치, 속도 및 토크는 모두 로터 측을 의미하는 반면, CAN을 사용하여 MIT 제어를 수행할 때 프로토콜의 위치, 속도 및 토크는 모두 출력측 측을 의미합니다. 이는 MIT 오픈 소스 프로토콜과의 일관성을 유지하기 위함입니다!

✓ 호스트→모터

CAN 번호 데이터 프레임 임 위치	의미	설명
BYTE0	위치: 총 16비트, BYTE0은 상위 8비트, BYTE1은 하위 8비트	실제 위치는 double 형이며, 16비트 int 형으로 변환해야 합니다. 변환 과정은 다음과 같습니다: $\text{pos_int} = (\text{pos_double} + 12.5) * 65535 / 25$
BYTE1	출력측의 다중 회전 위치, 단위는 라디안 (RAD)	
BYTE2	속도: 총 12비트, BYTE2는 상위 8비트, BYTE3[7-4](상위 4비트)는 하위 4비트입니다. 출력 측의 각속도를 나타내며, 단위는 RAD/s입니다.	실제 속도 는 double 형이며, 12비트 int 형으로 변환해야 합니다. 변환 과정은 다음과 같습니다: $\text{vel_int} = (\text{vel_double} + 65) * 4095 / 130$ KP 값은 실제로 double 형이므로 12비트 int 형으로 변환해야 하며, 변환 과정은 다음과 같습니다: $\text{kp_int} = \text{kp_double} * 4095 / 500$
BYTE3		
BYTE4	KP 값: 총 12비트, BYTE3[3-0](하위 4비트)가 상위 4비트이고, BYTE4가 하위 8비트입니다.	
BYTE5	KD 값: 총 12비트이며, BYTE5는 상위 8비트, BYTE6[7-4](하위 4비트)는 하위 4비트입니다.	KD 값은 실제로 double 형이므로 12비트 int 형으로 변환해야 하며, 변환 과정은 다음과 같습니다: $\text{kd_int} = \text{kd_double} * 4095 / 5$ 실제 토크는 double 형이며, 12비트 int 형으로 변환해야 합니다. 변환 과정은 다음과 같습니다: $\text{t_int} = (\text{t_double} + 50) * 4095 / 100$ 토크 상수의 단위는 N·m/A입니다
BYTE6		
BYTE7	토크: 총 12비트, BYTE6[3-0](하위 4비트)가 상위 4비트, BYTE7이 하위 8비트입니다. 단위는 N.m입니다. 비트가 상위 4비트이고, BYTE7이 하위 8비트입니다. 단위는 N.m입니다.	

✓ 모터→호스트

CAN 번호 데이터 프레임 임 위치	의미	설명
---------------------------	----	----

BYTE0	노드 ID	드라이버 노드 ID
BYTE1	위치: 총 16비트, BYTE1은 상위 8비트, BYTE2는 하위 8비트 출력축의 다중 회전 위치, 단위는 라디안	실제 위치는 double 형이며, 16비트 int 형에서 변환해야 합니다. 변환 과정은 다음과 같습니다: $\text{pos_double} = \text{pos_int} * 25 / 65535 - 12.5$
BYTE2	(RAD)	
BYTE3	속도: 총 12비트, BYTE3은 상위 8비트, BYTE4[7-4](상위 4비트)는 하위 4비트입니다. 출력 축의 각속도를 나타내며, 단위는 RAD/s	실제 속도는 double 형이며, 12비트 int 형에서 변환해야 합니다. 변환 과정은 다음과 같습니다: $\text{vel_double} = \text{vel_int} * 130 / 4095 - 65$ 실제 토크는 double 형이며, 12비트 int 형에서 변환해야 합니다. 변환 과정은 다음과 같습니다: $\text{t_double} = \text{t_int} * 100 / 4095 - 50$ 토크 상수의 단위는 N·m/A입니다
BYTE4	토크: 총 12비트, BYTE4[3-0](하위 4비트)가 상위 4비트, BYTE5가 하위 8비트입니다.	
BYTE5	단위는 N·m입니다.	

➤ Get_Encoder_Estimates CMD
ID: 0x009 (모터→ 호스트)

시작 바이트	이름	유형	단위	설명
0	Pos_Estimate	float32	rev	odrv0.axis0.encoder.pos_estimate 현재 모터 로터의 위치
4	Vel_Estimate	float32	rev/s	odrv0.axis0.encoder.vel_estimate 현재 모터 로터의 회전 속도

➤ Get_Encoder_Count
CMD ID: 0x00A (모터→ 호스트)

시작 바이트	이름	유형	설명
0	Shadow_Count	int32	odrv0.axis0.encoder.shadow_count 인코더 다중 회전 카운트
4	Count_In_Cpr	int32	odrv0.axis0.encoder.count_in_cpr 인코더 단일 회전 카운트

➤ Set_Controller_Mode CMD ID:
0x00B (호스트→ 모터)

시작 바이트	이름	유형	설명
0	Control_Mode	uint32	odrv0.axis0.controller.config.control_mode 제어 모드: 0: 전압 제어 1: 토크 제어 2: 속도 제어 3: 위치 제어
4	Input_Mode	uint32	odrv0.axis0.controller.config.input_mode

			입력 모드. 사용자 입력 제어값이 모터 작동을 제어하는 방식을 나타냅니다: 0: 유힬 1: 직접 제어 2: 속도 경사 3: 위치 필터링 5: 사다리꼴 곡선 6: 토크 램프 9: 모션 제어 (MIT)
--	--	--	--

➤ Set_Input_Pos

CMD ID: 0x00C (호스트→모터)

시작 바이트	이름	유형	단위	설명
0	Input_Pos	float32	rev	odrv0.axis0.controller.input_pos 위치 제어의 목표값.
4	Vel_FF	int16	0.001rev/s	odrv0.axis0.controller.input_vel 위치 제어의 속도 피드포워드.
6	Torque_FF	int16	0.001Nm	odrv0.axis0.controller.input_torque 위치 제어의 토크 피드포워드.

➤ Set_Input_Vel

CMD ID: 0x00D (호스트→모터)

시작 바이트	이름	유형	단위	설명
0	Input_Vel	float32	rev/s	odrv0.axis0.controller.input_vel 속도 제어의 목표값.
4	Torque_FF	float32	Nm	odrv0.axis0.controller.input_torque 속도 제어의 토크 피드포워드.

➤ Set_Input_Torque

CMD ID: 0x00E (호스트→모터)

시작 바이트	이름	유형	단위	설명
0	Input_Torque	float32	Nm	odrv0.axis0.controller.input_torque 토크 제어의 목표값.

➤ Set_Limits

CMD ID: 0x00F (호스트→모터)

시작 바이트	이름	유형	단위	설명
0	Velocity_Limit	float32	rev/s	odrv0.axis0.controller.config.vel_limit 전역 속도 제한.

4	Current_Limit	float32	A	odrv0.axis0.motor.config.current_lim 모터 전류 제한.
---	---------------	---------	---	---

➤ Start_Anticogging

CMD ID: 0x010 (호스트→모터)

토크 리플 보정을 수행합니다.

➤ Set_Traj_Vel_Limit

CMD ID: 0x011 (호스트→모터)

시작 바이트	이름	유형	단위	설명
0	Traj_Vel_Limit	float32	rev/s	odrv0.axis0.trap_traj.config.vel_limit 사다리꼴 위치 제어의 속도 제한.

➤ Set_Traj_Accel_Limits CMD ID:

0x012 (호스트→모터)

시작 바이트	이름	유형	단위	설명
0	Traj_Accel_Limit	float32	rev/s ²	odrv0.axis0.trap_traj.config.accel_limit 사다리꼴 위치 제어의 가속도 제한.
4	Traj_Decel_Limit	float32	rev/s ²	odrv0.axis0.trap_traj.config.decel_limit 사다리꼴 위치 제어의 감속도 제한.

➤ Set_Traj_Inertia

CMD ID: 0x013 (호스트→모터)

시작 바이트	이름	유형	단위	설명
0	Traj_Inertia	float32	Nm/(rev/s ²)	odrv0.axis0.controller.config.inertia 관성.

➤ Get_Iq

CMD ID: 0x014 (모터→호스트)

시작 바이트	이름	유형	단위	설명
0	Iq_Setpoint	float32	A	odrv0.axis0.motor.current_control.Idq_setpoint Q축 전류 설정값.
4	Iq_Measured	float32	A	odrv0.axis0.motor.current_control.Iq_measured Q축 전류 실제 측정값.

➤ 재부팅

CMD ID: 0x016 (호스트→모터)

➤ Get_Bus_Voltage_Current CMD
ID: 0x017(모터→호스트)

시작 바이트	이름	유형	단위	설명
0	Bus_Voltage	float32	V	odrv0.vbus_voltage 버스 전압.
4	Bus_Current	float32	A	odrv0.ibus 버스 전류.

➤ Clear_Errors

CMD ID: 0x018 (호스트→모터)

모든 오류 및 예외를 지웁니다.

➤ Set_Linear_Count

CMD ID: 0x019 (호스트→모터)

인코더의 절대 위치를 설정합니다.

시작 바이트	이름	유형	설명
0	Linear_Count	int32	odrv0.axis0.encoder.set_linear_count() 인코더의 절대 위치(카운트 값)를 설정합니다.

➤ Set_Pos_Gain

CMD ID: 0x01A (호스트→모터)

시작 바이트	이름	유형	단위	설명
0	Pos_Gain	float32	(rev/s)/rev	odrv0.axis0.controller.config.pos_gain 위치 루프 이득 Kp.

➤ Set_Vel_Gains

CMD ID: 0x01B (호스트→모터)

시작 바이트	이름	유형	단위	설명
0	Vel_Gain	float32	Nm/(rev/s)	odrv0.axis0.controller.config.vel_gain 속도 루프 이득 Kp.
4	Vel_Integrator_Gain	float32	Nm/rev	odrv0.axis0.controller.config.vel_integrator_gain 속도 루프 이득 Ki.

➤ Get_Torques

CMD ID: 0x01C (모터→ 호스트)

시작 바이트	이름	유형	단위	설명
0	Torque_Setpoint	float32	Nm	odrv0.axis0.controller.torque_setpoint 전류 루프 토크 설정값.
4	토크	float32	Nm	현재 토크 값을 나타냅니다.

➤ Get_Powers

CMD ID: 0x01D (모터→ 호스트)

시작 바이트	이름	유형	설명
0	Electrical_Power	float32	odrv0.axis0.controller.electrical_power 전력
4	Mechanical_Power	float32	odrv0.axis0.controller.mechanical_power 기계적 출력

➤ Disable_Can

CMD ID: 0x01E (호스트→ 모터)

CAN을 비활성화하고 드라이버를 재시작합니다.

➤ Save_Configuration

CMD ID: 0x01F (호스트→ 모터)

현재 구성을 저장하고 적용한 후 재시작합니다.

4.1.3 CAN 프로토콜 실전

4.1.3.1 실전: 전원 공급 시 보정

CAN 메시지 전송 순서는 다음과 같습니다:

CAN ID	프레임 유형	프레임 데이터	설명
0x007	데이터 프레임	04 00 00 00 00 00 00 00	메시지: Set_Axis_State 매 개변수: 4 모터 보정
0x007	데이터 프레임	07 00 00 00 00 00 00 00	메시지: Set_Axis_State 매 개변수: 7 인코더 보정

4.1.3.2 실전: 속도 제어

CAN 메시지 전송 순서는 다음과 같습니다:

CAN ID	프레임 유형	프레임 데이터	설명
0x00B	데이터 프레임	02 00 00 00 02 00 00 00	메시지: Set_Controller_Mode

			매개변수: 2/2 제어 모드를 속도 제어, 입력 모드 모드는 속도 경사로 설정
0x007	데이터 프레임	08 00 00 00 00 00 00 00	메시지: Set_Axis_State 매 개변수: 8 페루프 제어 상태로 진입
0x0D	데이터 프레임	00 00 20 41 00 00 00 00	메시지: Set_Input_Vel 매 개변수: 10/0 목표 속도와 토크 피드포워드를 설정합니다. 여 기서 목표 속도는 10(부동 소수점: 0x41200000), 토크 피드포워드는 0(부동 수: 0x00000000)

4.1.3.3 실전: 위치 제어

CAN 메시지 전송 시퀀스는 다음과 같습니다:

CAN ID	프레임 유형	프레임 데이터	설명
0x00B	데이터 프레임	03 00 00 00 03 00 00 00	메시지: Set_Controller_Mode 매개 변수: 3/3 제어 모드를 위치 제어, 입력 모드 는 위치 필터링
0x007	데이터 프레임	08 00 00 00 00 00 00 00	메시지: Set_Axis_State 매 개변수: 8 페루프 제어 상태로 진입
0x0C	데이터 프레임	CD CC 0C 40 00 00 00 00	메시지: Set_Input_Pos 매 개변수: 2.2/0/0 목표 위치, 속도 피드포워드 및 토크 피드포워드 설정. 여기서 목표 위치는 2.2(부동 소수점: 0x400CCCCD)이며, 토크 피드포워드와 속도 피드백은 0

4.1.4 CANOpen 호환성

노드 ID가 적절하게 할당된 경우, CANOpen과 상호 운용이 가능합니다. 다음 표에는 CANOpen과 본 프로토콜의 유효한 노드 ID 조합이 나열되어 있습니다:

CANOpen 노드 ID	본 프로토콜의 노드 ID
32 ... 127	0x10, 0x18, 0x20, 0x28
64 ... 127	0x10, 0x11, 0x18, 0x19, 0x20, 0x21, 0x28, 0x29
96 ... 127	0x10, 0x11, 0x12, 0x18, 0x19, 0x1A, 0x20, 0x21, 0x22, 0x28, 0x29, 0x2A

4.1.5 주기적 메시지

사용자는 상위 장치에서 모터로 요청 메시지를 전송할 필요 없이, 모터가 상위 장치로 주기적으로 메시지를 전송하도록 구성할 수 있습니다. odrv0.axis0.config.can 아래의 일련의 설정을 통해 주기 메시지를 켜거나 끌 수 있습니다(값이 0이면 꺼짐, 다른 값은 주기 시간을 나타내며 단위는 ms입니다). 다음 표와 같습니다:

메시지	odrivetool 설정	기본값
하트비트	odrv0.axis0.config.can.heartbeat_rate_ms	100
Get_Encoder_Estimates	odrv0.axis0.config.can.encoder_rate_ms	10
Get_Motor_Error	odrv0.axis0.config.can.motor_error_rate_ms	0
Get_Encoder_Error	odrv0.axis0.config.can.encoder_error_rate_ms	0
Get_Controller_Error	odrv0.axis0.config.can.controller_error_rate_ms	0
센서리스 오류율 가져오기	odrv0.axis0.config.can.sensorless_error_rate_ms	0
Get_Encoder_Count	odrv0.axis0.config.can.encoder_count_rate_ms	0
Get_Iq	odrv0.axis0.config.can.iq_rate_ms	0
Get_Sensorless_Estimates	odrv0.axis0.config.can.sensorless_rate_ms	0
Get_Bus_Voltage_Current	odrv0.axis0.config.can.bus_vi_rate_ms	0

기본적으로 처음 두 가지 주기 메시지는  시 활성화되므로, 사용자가 CAN 버스를 모니터링할 때 두 가지 메시지가 설정된 주기로 브로드캐스팅되는 것을 확인할 수 있습니다. 사용자는 다음 명령을 통해 이를 비활성화할 수 있습니다:

```
odrv0.axis0.config.can.heartbeat_rate_ms = 0
odrv0.axis0.config.can.encoder_rate_ms = 0
```

각 메시지의 자세한 내용은 4.1.2항을 참조하십시오.

4.2 CANOpen 프로토콜

CANOpen은 자동화 분야용 통신 프로토콜 사양으로, 본 제품은 CiA301, CiA302, CiA402 사양을 준수하여 구현되었습니다. 본 제품의 CANOpen 프로토콜을 사용하기 전에, 다음 odrivetool 명령을 통해 CAN 프로토콜을 CANOpen으로 전환하십시오:

```
odrv0.can.config.protocol = 2
```

4.2.1 장치 설명서(EDS)

EDS 파일은 장치의 기능과 통신 매개변수를 기술합니다. CANOpen을 통해 본 제품과 통신하려면 본 제품의 EDS 파일을 불러와야 합니다. 다음 링크에서 다운로드하십시오: https://bl.cyberbeast.cn/actuator/cb_0.1.eds

4.2.2 프로토콜 프레임 형식

CANOpen은 표준 CAN을 기반으로 하며, 11비트 ID는 4비트 기능 코드와 7비트 노드 ID(node_id)로 구성됩니다. 이는 버스에서 최대 127개의 노드를 지원함을 의미합니다(0은 브로드캐스트에 사용됨). 다음 표는 프레임 형식을 보여줍니다:

데이터 영역	COB-ID / CAN ID(11비트)		데이터(8바이트)
세그먼트	Bit10 ~ Bit7	Bit6 ~ Bit0	바이트 0 ~ 바이트 7
설명	기능 코드	node_id	통신 데이터

기능 코드 표는 다음과 같습니다:

통신 기능	COB-ID	설명
NMT	0x000	네트워크 관리
SYNC	0x080	동기화
EMCY	0x080+node_id	비상 관리

TPDO1~TPDO4	0x180/0x280/0x380/0x480+node_id	실시간 프로세스 데이터 전송
RPDO1~RPDO4	0x200/0x300/0x400/0x500+node_id	실시간 프로세스 데이터 수신
TSDO	0x600+node_id	서비스 데이터 전송
RSDO	0x580+node_id	서비스 데이터 수신
NMT(심장 박동)	0x700+node_id	심장 박동

4.2.3 프레임 메시지

CAN ID(COB-ID)에 따라 프레임 메시지의 형식도 다릅니다.

4.2.3.1 NMT(네트워크 관리)

NMT는 버스의 모든 노드 또는 특정 노드에 상태 변경 명령을 전송하는 데 사용되며, COB-ID는 항상 0이고 데이터는 2바이트로 구성됩니다. 자세한 내용은 다음 표를 참조하십시오:

COB-ID	데이터 바이트 0	데이터 바이트 1
0x000	상태 명령어	node_id

여기서 상태 명령어에는 다음이 포함됩니다:

상태 명령어	설명
0x01	상태가 "operational"로 변경됨
0x02	상태가 "stopped"로 변경됨
0x80	상태가 "pre-operational"로 변경됨
0x81	노드 재설정 "reset node"
0x82	통신 재설정 "reset communication"

예를 들어, 사용자가 드라이브를 연결한 후, 먼저 다음 NMT 명령을 전송하여 버스상의 모든 노드를 작동시킵니다:

COB-ID	Data
0x000	01 00

4.2.3.2 SYNC

동기화 기능은 버스의 모든 노드를 동기화하기 위해 사용되며, 주로 노드 간 PDO 메시지 동기화에 사용됩니다. COB-ID는 0x80으로 고정되어 있으며, 데이터 필드는 비어 있습니다.

예를 들어, 사용자가 NMT 명령을 사용하여 버스 노드를 구동시킨 후에는 일반적으로 SYNC 메시지를 전송하여 노드가 PDO 데이터 전송을 시작하도록 합니다.

4.2.3.3 PDO

PDO는 실시간 데이터 전송을 위해 마련된 통신 방식이며, 사용자가 주기적으로 노드 데이터를 획득하거나 이벤트 기반 방식으로 데이터를 보고하고자 할 때 모두 PDO를 사용하여 전송합니다. PDO는 노드 상태가 "operational"일 때만 사용되므로, 사용자는 먼저 NMT 메시지를 사용하여 노드 상태를 변경해야 합니다.

PDO 메시지의 데이터 영역 형식은 매핑 구조에 따라 결정되며, 이 매핑 구조는 표준이 아닌 EDS 파일에 의해 규정됩니다.

4.2.3.4 SDO

SDO 통신은 본 제품의 객체 사전 데이터(객체 사전은 EDS에 의해 규정됨)에 대한 액세스를 허용합니다. 소위 객체 사전 데이터는 제품의 파라미터 정보로 이해할 수 있습니다. 일반적으로 SDO는 구성 과정에만 사용되며, 실행 과정 중의 데이터 교환에는 사용되지 않습니다. 실시간 실행 데이터 교환에는 PDO를 사용하는 것이 좋습니다.

SDO는 읽기 및 쓰기 두 가지 다른 프로세스로 나뉘며, 아래 표는 쓰기 프로세스(업로드라고도 함)입니다:

	COB-ID	Data Byte0	Data 바이트 1	데이터 바이트2	데이터 바이트3	데이터 바이트4	데이터 바이트 5	데이터 바이트 6	데이터 바이트 7
쓰기	0x600+ node_id	0x23	index		subindex	data			
		0x27	인덱스		하위 인덱스	데이터			/
		0x2B	인덱스		하위 인덱스	data		/	/
		0x2F	index		하위 인덱스	데이터	/	/	/
응답	0x580+ node_id	0x60	index		하위 인덱스	/	/	/	/
		0x80	Index		하위 인덱스	중단 코드			

읽기 프로세스(다운로드라고도 함):

	COB-ID	데이터 바이트 0	데이터 바이트 1	데이터 바이트2	데이터 바이트 3	데이터 바이트4	데이터 바이트 5	데이터 바이트 6	데이터 바이트 7
읽기	0x600+ node_id	0x40	index		하위 인덱스	/	/	/	/
응답	0x580+ node_id	0x43	index		subindex	data			
		0x47	인덱스		하위 인덱스	데이터			/
		0x4B	인덱스		하위 인덱스	data		/	/
		0x4F	index		하위 인덱스	데이터	/	/	/
		0x80	index		하위 인덱스	중단 코드			

4.2.3.5 NMT(하트비트)

하트비트 메시지는 노드의 상태를 버스에 알리기 위한 용도로만 사용되며, 노드가 정상적으로 작동하고 있음을 나타냅니다. 프

레이밍 형식은 다음과 같습니다:

COB-ID	데이터 바이트 0
0x700+node_id	상태

4.3 RS485 프로토콜 (Modbus RTU)

버전 3.10 이상의 하드웨어 버전에서는 통신 인터페이스가 RS485, 펄스 방향 및 PWM을 동시에 지원합니다. 다음 명령을 사용하여 통신 인터페이스가 RS485인지 확인하십시오:

```
odrv0.config.comm_intf_mux = 1
```


모터 엘프(Motor Elf)에서는 통신 매개변수에서 물리적 인터페이스를 전환할 수도 있습니다.

RS485 기반 Modbus RTU의 최대 통신 속도는 115200bps이며, 485 기반 속도는 115200bps입니다.

4.3.1 Modbus RTU

산업 표준인 Modbus RTU를 기반으로 하며, 현재 지원하는 기능 코드는 다음과 같습니다:

기능 코드	의미
0x04	하나 또는 여러 개의 입력 레지스터 읽기
0x03	여러 개의 홀드 레지스터 읽기
0x06	단일 유지 레지스터 쓰기
0x10	여러 홀드 레지스터 쓰기

4.3.2 입력 레지스터

입력 레지스터는 읽기 전용이며 쓰기가 불가능하며, 오류 메시지, 전압/전류 등과 같은 시스템의 상태나 동적 지표를 나타냅니다. 현재 지원되는 입력 레지스터는 다음과 같습니다:

➤ 0x01: 모터 상태

호스트 요청:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	04	00 01	00 01	XX XX

모터 반응:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	04	02	State0 State1	XX XX

이 중 State0은 항상 0이며, State1은 모터 상태 코드입니다:

모터 상태 코드	설명
1	유휴
3	완전 보정
4	모터 보정
7	인코더 보정
8	페루프 작동

11	제로 복귀
----	-------

➤ **0x02: 이상 플래그**

호스트 요청:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	04	00 02	00 01	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	04	02	Flag0 Flag1	XX XX

이 중 Flag0은 항상 0이며, Flag1은 예외 플래그입니다. 각 비트가 1이면 해당 예외가 발생했음을 나타내며, 예외의 의미는 다음과 같습니다:

bit7~bit5	bit4	bit3	bit2	bit1	bit0
의미 없음	드라이버 오류	시스템 오류	제어 오류	인코더 오류	모터 오류

➤ **0x03: 드라이버 오류 코드 1**

➤ **0x04: 드라이버 오류 코드 2**

드라이버 오류 코드는 32비트 부호 없는 정수이며, 드라이버 오류 코드 1은 상위 16비트, 드라이버 오류 코드 2는 하위 16비트를 차지합니다. 사용자는 레지스터 주소 0x03에 액세스하여 두 레지스터의 데이터를 동시에 가져와야만 전체 드라이버 오류 코드를 확인할 수 있습니다. 어느 한 레지스터만 단독으로 액세스할 수는 없습니다.

호스트 요청:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	04	00 03	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	04	04	E0 E1 E2 E3	XX XX

E0 E1 E2 E3는 구동 오류 코드를 구성하며, 5.2절의 오류 코드 표를 참조하십시오.

➤ **0x05: 모터 오류 코드 1**

➤ **0x06: 모터 오류 코드 2**

➤ **0x07: 모터 오류 코드 3**

➤ **0x08: 모터 오류 코드 4**

모터 오류 코드는 64비트 부호 없는 정수이며, 이 중 모터 오류 코드 1은 63~48비트, 모터 오류 코드 2는 47~32비트, 모터 오류 코드 3은 31~16비트, 모터 오류 코드 4는 15~0비트를 차지합니다. 사용자는 레지스터 주소 0x05에 액세스하여 네 개의 레지스터 데이터를 동시에 가져와야만 전체 모터 오류 코드를 확인할 수 있습니다. 개별 레지스터에 단독으로 액세스할 수는 없습니다.

호스트 요청:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	04	00 05	00 04	XX XX

모터 반응:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	8바이트	2바이트
XX	04	08	E0 E1 E2 E3 E4 E5 E6 E7	XX XX

E0 E1 E2 E3 E4 E5 E6 E7은 모터 오류 코드를 구성하며, 5.2절의 오류 코드 표를 참조하십시오.

➤ **0x09: 인코더 오류 코드 1**

➤ **0x0A: 인코더 오류 코드 2**

인코더 오류 코드는 32비트 부호 없는 정수이며, 인코더 오류 코드 1은 상위 16비트, 인코더 오류 코드 2는 하위 16비트를 차지합니다. 사용자는 레지스터 주소 0x09에 액세스하여 두 레지스터의 데이터를 동시에 가져와야만 전체 인코더 오류 코드를 얻을 수 있습니다. 어느 한 레지스터만 별도로 액세스할 수는 없습니다.

호스트 요청:

주소	기능 코드	레지스터 주소	레지스터 수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	04	00 09	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	04	04	E0 E1 E2 E3	XX XX

E0 E1 E2 E3는 인코더 오류 코드를 구성합니다. 5.2절의 오류 코드 표를 참조하십시오.

➤ **0x0B: 제어 오류 코드 1**

➤ **0x0C: 제어 예외 코드 2**

제어 예외 코드는 32비트 부호 없는 정수이며, 제어 예외 코드 1은 상위 16비트, 제어 예외 코드 2는 하위 16비트를 차지합니다. 사용자는 레지스터 주소 0x0B에 액세스하여 두 레지스터의 데이터를 동시에 가져와야만 전체 제어 예외 코드를 얻을 수 있습니다. 어느 한 레지스터만 단독으로 액세스할 수는 없습니다.

호스트 요청:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	04	00 0B	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	04	04	E0 E1 E2 E3	XX XX

E0 E1 E2 E3는 제어 오류 코드를 구성하며, 5.2절의 오류 코드 표를 참조하십시오.

➤ **0x0D: 시스템 오류 코드 1**

➤ **0x0E: 시스템 오류 코드 2**

시스템 오류 코드는 32비트 부호 없는 정수이며, 시스템 오류 코드 1은 상위 16비트, 시스템 오류 코드 2는 하위 16비트를 차지합니다. 사용자는 레지스터 주소 0x0D에 액세스하여 두 레지스터의 데이터를 동시에 가져와야만 전체 시스템 오류 코드를 얻을 수 있습니다. 어느 한 레지스터만 따로 액세스할 수는 없습니다.

호스트 요청:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	04	00 0D	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	04	04	E0 E1 E2 E3	XX XX

E0 E1 E2 E3는 시스템 오류 코드를 구성하며, 5.2절의 오류 코드 표를 참조하십시오.

➤ **0x0F: 위치 피드백 1**

➤ **0x10: 위치 피드백 2**

위치 피드백은 32비트 부동 소수점 숫자이며, 위치 피드백 1은 상위 16비트, 위치 피드백 2는 하위 16비트입니다. 사용자는 레지스터 주소 0x0F에 액세스하여 두 레지스터 데이터를 동시에 가져와야만 전체 위치 피드백을 얻을 수 있습니다. 어느 한 레지스터만 개별적으로 액세스할 수는 없습니다.

호스트 요청:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	04	00 0F	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	04	04	P0 P1 P2 P3	XX XX

P0 P1 P2 P3는 위치 피드백을 구성하며, 32비트 부동소수점은 IEEE 754 표준에 따라 인코딩됩니다(인코딩 테스트는 웹 사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 확인할 수 있습니다).

➤ **0x11: 속도 피드백 1**

➤ **0x12: 속도 피드백 2**

속도 피드백은 32비트 부동 소수점 수이며, 여기서 속도 피드백 1은 상위 16비트, 속도 피드백 2는 하위 16비트를 차지합니다. 사용자는 레지스터 주소 0x11에 액세스하여 두 레지스터의 데이터를 동시에 가져와야만 완전한 속도 피드백을 얻을 수 있습니다. 어느 한 레지스터만 단독으로 액세스할 수는 없습니다.

호스트 요청:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	04	00 11	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	04	04	V0 V1 V2 V3	XX XX

V0 V1 V2 V3는 속도 피드백을 구성하며, 32비트 부동소수점은 IEEE 754 표준에 따라 인코딩됩니다(인코딩 테스트는 웹 사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 확인할 수 있습니다).

➤ **0x13: 버스 전압 피드백 1**

➤ **0x14: 버스 전압 피드백 2**

버스 전압은 32비트 부동 소수점 수이며, 여기서 버스 전압 1은 상위 16비트, 버스 전압 2는 하위 16비트를 차지합니다. 사용자는 레지스터 주소 0x13에 액세스하여 두 레지스터의 데이터를 동시에 가져와야만 전체 버스 전압을 얻을 수 있습니다. 어느 한 레지스터만 단독으로 액세스할 수는 없습니다.

호스트 요청:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	04	00 13	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	04	04	V0 V1 V2 V3	XX XX

V0 V1 V2 V3는 버스 전압을 구성하며, 32비트 부동 소수점은 IEEE 754 표준에 따라 인코딩됩니다(웹사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 인코딩을 테스트할 수 있음).

- **0x15: 버스 전류 피드백 1**
- **0x16: 버스 전류 피드백 2**

버스 전류 피드백은 32비트 부동소수점 수이며, 이 중 버스 전류 피드백 1은 상위 16비트, 버스 전류 피드백 2는 하위 16비트를 차지합니다. 사용자는 레지스터 주소 0x15에 액세스하여 두 레지스터의 데이터를 동시에 가져와야만 완전한 버스 전류 피드백을 얻을 수 있습니다. 어느 한 레지스터만 단독으로 액세스할 수는 없습니다.

호스트 요청:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	04	00 15	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	04	04	I0 I1 I2 I3	XX XX

I0 I1 I2 I3는 버스 전류 피드백을 구성하며, 32비트 부동 소수점은 IEEE 754 표준에 따라 인코딩됩니다(웹사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 인코딩을 테스트할 수 있음).

➤ **0x17: Q축 전류 목표값 1**

➤ **0x18: Q축 전류 목표값 2**

Q축 전류 목표값은 32비트 부동 소수점 수이며, Q축 전류 목표값 1은 상위 16비트, Q축 전류 목표값 2는 하위 16비트입니다. 사용자는 레지스터 주소 0x17에 액세스하여 두 레지스터 데이터를 동시에 가져와야만 전체 Q축 전류 목표값을 얻을 수 있습니다. 어느 한 레지스터만 개별적으로 액세스할 수는 없습니다.

호스트 요청:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	04	00 17	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1 바이트	1 바이트	1 바이트	2바이트	2 바이트
XX	04	04	I0 I1 I2 I3	XX XX

I0 I1 I2 I3는 Q축 전류 목표값을 구성하며, 32비트 부동소수점은 IEEE 754 표준에 따라 인코딩됩니다(웹사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 인코딩을 테스트할 수 있음).

➤ **0x19: Q축 전류 피드백 1**

➤ **0x1A: Q축 전류 피드백 2**

Q축 전류 피드백은 32비트 부동 소수점 수이며, Q축 전류 피드백 1은 상위 16비트, Q축 전류 피드백 2는 하위 16비트를 나타냅니다. 사용자는 레지스터 주소 0x19에 액세스하여 두 레지스터 데이터를 동시에 가져와야만 완전한 Q축 전류 피드백을 얻을 수 있습니다. 어느 한 레지스터만 개별적으로 액세스할 수는 없습니다.

호스트 요청:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	04	00 19	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	04	04	I0 I1 I2 I3	XX XX

I0 I1 I2 I3는 Q축 전류 피드백을 구성하며, 32비트 부동소수점은 IEEE 754 표준에 따라 인코딩됩니다(인코딩은 웹사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 테스트할 수 있습니다).

- **0x31: 임의의 엔드포인트 액세스 반환값 1**
- **0x32: 임의 엔드포인트 액세스 반환값 2**

임의 엔드포인트 액세스는 사용자가 지원되는 모든 매개변수를 읽거나 쓸 수 있음을 의미하며, 여기서 "엔드포인트"는 매개변수를 가리키며 각 매개변수에는 고유한 엔드포인트 번호가 있습니다. 사용자가 특정 매개변수를 읽으려면 엔드포인트 설명 파일을 통해 해당 매개변수의 엔드포인트 번호를 확인한 후, 이를 유지 레지스터 0xA1에 기록하고, 입력 레지스터 0x31을 읽어 해당 매개변수의 값을 얻을 수 있습니다. 엔드포인트 설명 파일을 다운로드할 주소를 확인하려면 유지 레지스터 0xA1의 설명을 참조하십시오.

임의 엔드포인트 액세스 반환 값은 32비트이며, 16비트 정수, 32비트 정수 또는 32비트 부동 소수점 수일 수 있습니다. 이 중

임의

임의 엔드포인트 액세스 반환값 1은 상위 16비트, 임의 엔드포인트 액세스 반환값 2는 하위 16비트를 나타냅니다. 사용자는 레지스터 주소 0x31을 액세스하여 두 레지스터 데이터를 동시에 읽어야만 전체 반환값(매개변수 값)을 얻을 수 있습니다. 어느 한 레지스터만 단독으로 액세스할 수는 없습니다.

호스트 요청:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	04	00 31	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	04	04	V0 V1 V2 V3	XX XX

여기서 V0, V1, V2, V3는 임의의 엔드포인트 액세스 반환값의 높은 바이트부터 낮은 바이트 순서로 배열된 4바이트이며, 32비트 부동소수점은 IEEE 754 표준에 따라 인코딩됩니다(인코딩 테스트는 웹사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 확인할 수 있습니다).

4.3.3 레지스터 유지

레지스터는 읽기 및 쓰기가 가능하며, 현재 지원되는 레지스터와 그 기능은 다음과 같습니다:

- **0x61: 비상 정지**

이 레지스터에 임의의 값을 쓰면 모터가 비상 정지합니다.

호스트 요청:

주소	기능 코드	레지스터 주소	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	06	00 61	XX XX	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 값	검증 코드
1바이트	1바이트	2바이트	2바이트	2바이트
XX	06	00 61	XX XX	XX XX

➤ **0x62: 노드 ID**

06 기능 코드를 사용하여 이 노드에 노드 ID를 쓰거나, 03 기능 코드를 사용하여 현재 노드 ID를 읽습니다. 노드 ID 쓰기 요청:

주소	기능 코드	레지스터 주소	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	06	00 62	ID0 ID1	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	06	00 62	ID0 ID1	XX XX

여기서 ID0와 ID1은 새 노드 ID의 상위 8비트와 하위 8비트입니다.

현재 노드 ID 읽기 요청:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	03	00 62	00 01	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	03	02	ID0 ID1	XX XX

➤ **0x63: 상태 제어**

06 기능 코드를 사용하여 이 노드에 모터 상태를 기록(모터 상태 변경). 호스트

요청:

주소	기능 코드	레지스터 주소	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	06	00 63	S0 S1	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	06	00 63	S0 S1	XX XX

여기서 S0와 S1은 모터 상태의 상위 8비트와 하위 8비트입니다. 상태 정의는 다음과 같습니다:

모터 상태 코드	설명
1	유헴
3	완전 보정
4	모터 보정
7	인코더 보정
8	페루프 작동
11	제로 복귀

예를 들어, 모터 상태를 8(페루프 운전)으로 설정하면 통상적인 의미에서 "모터 켜기"가 되며, 모터 상태를 1(대기)으로 설정하면 통상적인 의미에서 "모터 끄기"가 됩니다.

➤ **0x64: 제어 모드**

사용자는 06 기능 코드를 사용하여 이 레지스터의 값을 수정하여 토크 제어, 속도 제어 또는 위치 제어 모드로 진입하거나, 03 기능 코드를 사용하여 현재 제어 모드를 읽을 수 있습니다.

제어 모드 변경 요청:

주소	기능 코드	레지스터 주소	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	06	00 64	M0 M1	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	06	00 64	M0 M1	XX XX

읽기 제어 모드 요청:

주소	기능 코드	레지스터 주소	레지스터 수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	03	00 64	00 01	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	03	02	M0 M1	XX XX

여기서 M0와 M1은 제어 모드의 상위 8비트와 하위 8비트입니다. 제어 모드는 다음과 같이 정의됩니다:

제어 모드	설명
1	토크 제어
2	속도 제어
3	위치 제어

➤ 0x65: 입력 모드

사용자는 06 기능 코드를 사용하여 이 레지스터의 값을 수정함으로써 직접 입력, 필터

파형 입력 등, 또는 03 기능 코드를 사용하여 현재 입력 모드를 읽을 수 있습니다.

입력 모드 변경 요청:

주소	기능 코드	레지스터 주소	레지스터 값	검증 코드
1바이트	1바이트	2바이트	2바이트	2바이트
XX	06	00 65	M0 M1	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	06	00 65	M0 M1	XX XX

입력 모드 요청 읽기:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	03	00 65	00 01	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	03	02	M0 M1	XX XX

여기서 M0와 M1은 입력 모드의 상위 8비트와 하위 8비트입니다. 입력 모드는 다음과 같이 정의됩니다:

제어 모드	설명	상세 설명
1	직접 입력	사용자가 입력한 목표값을 드라이버의 목표값으로 직접 사용
2	속도 경사	드라이버의 속도 목표값은 일정 시간 동안 일정한 경사도를 따라 도달합니다 사용자가 입력한 속도 목표값에 도달하며, 속도 제어에만 사용됩니다
3	위치 필터링	드라이버의 위치 목표값은 사용자가 입력한 위치 목표값을 필터링 처리된 값이며, 위치 제어에만 사용됩니다
5	사다리꼴 곡선	드라이버는 모터가 사다리꼴 속도 곡선을 따라 사용자가 입력한 목표값에 도달하도록 구동하며, 이 때 사다리꼴 곡선의 상승 기울기(가속도), 하강 기울기 (감속도), 감속 속도는 모두 설정할 수 있으며, 후속 레지스터 0x70~0x75
6	토크 슬로프	드라이버의 토크 목표값은 일정 시간 동안 일정한 경사도를 따라 사용자가 입력한 목표 토크 값에 도달하며, 토크 제어에만 사용됩니다

➤ **0x66: 위치 제어 목표 1**

➤ **0x67: 위치 제어 대상 2**

위치 제어 대상은 32비트 부동 소수점 수이며, 이 중 위치 제어 대상 1은 상위 16비트, 위치 제어 대상 2는 하위 16비트를 차지합니다. 사용자는 레지스터 주소 0x66에 액세스하여 두 레지스터의 데이터를 동시에 읽거나 쓰면서 전체 위치 제어 목표를 획득하거나 수정할 수 있습니다. 어느 한 레지스터만 단독으로 액세스할 수는 없습니다.

위치 제어 대상 수정/업데이트 요청:

주소	기능 코드	레지스터 주소	레지스터 수	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	1바이트	4바이트	2바이트
XX	10	00 66	00 02	04	P0 P1 P2 P3	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	10	00 66	00 02	XX XX

위치 제어 대상 요청 읽기:

주소	기능 코드	레지스터 주소	레지스터 수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	03	00 66	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	03	04	P0 P1 P2 P3	XX XX

여기서 P0, P1, P2, P3은 위치 제어 목표의 높은 값부터 낮은 값까지의 4바이트이며, 32비트 부동소수점은 IEEE 754 표준에 따라 인코딩됩니다(웹사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 인코딩을 테스트할 수 있습니다

).

➤ **0x68: 속도 제어 목표 1**

➤ **0x69: 속도 제어 목표 2**

속도 제어 목표는 32비트 부동 소수점 수이며, 이 중 속도 제어 목표 1은 상위 16비트, 속도 제어 목표 2는 하위 16비트를 차지합니다. 사용자는 레지스터 주소 0x68에 액세스하여 두 레지스터 데이터를 동시에 읽거나 쓰면서 전체 속도 제어 목표를 가져오거나 수정할 수 있습니다. 어느 한 레지스터만 개별적으로 액세스할 수는 없습니다.

속도 제어 목표 수정/업데이트 요청:

주소	기능 코드	레지스터 주소	레지스터 수	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	1바이트	4바이트	2바이트
XX	10	00 68	00 02	04	V0 V1 V2 V3	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	10	00 68	00 02	XX XX

읽기 속도 제어 대상 요청:

주소	기능 코드	레지스터 주소	레지스터 수	체크섬
----	-------	---------	--------	-----

1바이트	1바이트	2바이트	2바이트	2바이트
XX	03	00 68	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	03	04	V0 V1 V2 V3	XX XX

여기서 V0, V1, V2, V3는 속도 제어 목표값을 나타내는 높은 값부터 낮은 값 순서의 4바이트이며, 32비트 부동소수점은 IEEE 754 표준에 따라 인코딩됩니다(웹사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 인코딩을 테스트 할 수 있습니다

).

➤ **0x6A: 토크 제어 목표 1**

➤ **0x6B: 토크 제어 목표 2**

토크 제어 목표는 32비트 부동 소수점 수이며, 토크 제어 목표 1은 상위 16비트, 토크 제어 목표 2는 하위 16비트를 차지합니다. 사용자는 레지스터 주소 0x6A에 액세스하여 두 레지스터 데이터를 동시에 읽거나 쓰면서 전체 토크 제어 목표를 가져오거나 수정할 수 있습니다. 어느 한 레지스터만 개별적으로 액세스할 수는 없습니다.

토크 제어 목표 수정/업데이트 요청:

주소	기능 코드	레지스터 주소	레지스터 수	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	1바이트	4바이트	2바이트
XX	10	00 6A	00 02	04	T0 T1 T2 T3	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	10	00 6A	00 02	XX XX

토크 제어 목표 요청 읽기:

주소	기능 코드	레지스터 주소	레지스터 수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	03	00 6A	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	03	04	T0 T1 T2 T3	XX XX

여기서 T0, T1, T2, T3는 토크 제어 목표값의 높은 값부터 낮은 값까지의 4바이트이며, 32비트 부동소수점은 IEEE 754 표준에 따라 인코딩됩니다(웹사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 인코딩을 테스트할 수 있습니다).

➤ **0x6C: 속도 제한 1**

➤ **0x6D: 속도 제한 2**

속도 제한은 32비트 부동 소수점 수이며, 속도 제한 1은 상위 16비트, 속도 제한 2는 하위 16비트입니다. 사용자는 레지스터 주소 0x6C에 액세스하여 두 레지스터 데이터를 동시에 읽거나 쓰면서 전체 속도 제한을 가져오거나 수정할 수 있습니다. 어느 한 레지스터만 개별적으로 액세스할 수는 없습니다.

속도 제한 수정/업데이트 요청:

주소	기능 코드	레지스터 주소	레지스터 수	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	1바이트	4바이트	2바이트
XX	10	00 6C	00 02	04	V0 V1 V2 V3	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	10	00 6C	00 02	XX XX

읽기 속도 제한 요청:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	03	00 6C	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	03	04	V0 V1 V2 V3	XX XX

여기서 V0, V1, V2, V3는 속도 제한을 나타내는 높은 값부터 낮은 값 순서의 4바이트이며, 32비트 부동소수점은 IEEE 754 표준에 따라 인코딩됩니다(웹사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 인코딩을 테스트할 수 있습니다).

➤ **0x6E: 전류 제한 1**

➤ **0x6F: 전류 제한 2**

전류 제한은 32비트 부동 소수점 숫자이며, 전류 제한 1은 상위 16비트, 전류 제한 2는 하위 16비트입니다. 사용자는 레지스터 주소 0x6E에 액세스하여 두 레지스터 데이터를 동시에 읽거나 쓰면서 전체 전류 제한을 가져오거나 수정할 수 있습니다. 어느 한 레지스터만 개별적으로 액세스할 수는 없습니다.

전류 제한 수정/업데이트 요청:

주소	기능 코드	레지스터 주소	레지스터 수	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	1바이트	4바이트	2바이트
XX	10	00 6E	00 02	04	I0 I1 I2 I3	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	10	00 6E	00 02	XX XX

전류 제한 요청 읽기:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	03	00 6E	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	03	04	I0 I1 I2 I3	XX XX

여기서 I0, I1, I2, I3은 전류 제한 값을 높은 순서부터 낮은 순서로 나타내는 4바이트이며, 32비트 부동소수점은 IEEE 754 표준에 따라 인코딩됩니다(인코딩은 웹사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 테스트할 수 있습니다).

- **0x70: 사다리꼴 위치 제어 속도 제한 1**
- **0x71: 사다리꼴 위치 제어 속도 제한 2**

사다리꼴 위치 제어 속도 제한은 32비트 부동 소수점 수이며, 이 중 사다리꼴 위치 제어 속도 제한 1은 상위 16비트, 사다리꼴 위치 제어 속도 제한 2는 하위 16비트를 구성합니다. 사용자는 레지스터 주소 0x70에 액세스하여 두 레지스터 데이터를 동시에 읽거나 쓰면서 전체 사다리꼴 위치 제어 속도 제한을 획득하거나 수정할 수 있습니다. 개별 레지스터에 단독으로 액세스할 수는 없습니다.

사다리꼴 위치 제어 속도 제한 수정/업데이트 요청:

주소	기능 코드	레지스터 주소	레지스터 수	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	1바이트	4바이트	2바이트
XX	10	00 70	00 02	04	V0 V1 V2 V3	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	10	00 70	00 02	XX XX

사다리꼴 위치 제어 속도 제한 요청 읽기:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	03	00 70	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1 바이트	2바이트	2바이트
XX	03	04	V0 V1 V2 V3	XX XX

여기서 V0, V1, V2, V3는 사다리꼴 위치 제어 속도 제한의 높은 값부터 낮은 값까지의 4바이트이며, 32비트 부동소수점은 IEEE 754 표준에 따라 인코딩됩니다(웹사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 인코딩을 테스트할 수 있습니다).

- **0x72: 사다리꼴 위치 제어 가속도 제한 1**
- **0x73: 사다리꼴 위치 제어 가속도 제한 2**

사다리꼴 위치 제어 가속도 제한은 32비트 부동 소수점 수이며, 사다리꼴 위치 제어 가속도 제한 1은 상위 16비트, 사다리꼴 위치 제어 가속도 제한 2는 하위 16비트를 구성합니다. 사용자는 레지스터 주소 0x72에 액세스하여 두 레지스터 데이터를 동시에 읽거나 쓰면서 전체 사다리꼴 위치 제어 가속도 제한을 획득하거나 수정할 수 있습니다. 개별 레지스터에 단독으로 액세스할 수는 없습니다.

사다리꼴 위치 제어 가속도 제한 수정/업데이트 요청:

주소	기능 코드	레지스터 주소	레지스터 수	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	1바이트	4바이트	2바이트
XX	10	00 72	00 02	04	A0 A1 A2 A3	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	10	00 72	00 02	XX XX

사다리꼴 위치 제어 가속도 제한 요청 읽기:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	03	00 72	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	03	04	A0 A1 A2 A3	XX XX

여기서 A0, A1, A2, A3은 사다리꼴 위치 제어 가속도 제한의 높은 값부터 낮은 값까지의 4바이트이며, 32비트 부동소수 점은 IEEE 754 표준에 따라 인코딩됩니다(웹사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 인코딩을 테스트할 수 있습니다).

- **0x74: 사다리꼴 위치 제어 감속도 제한 1**
- **0x75: 사다리꼴 위치 제어 감속도 제한 2**

트래퍼즈 위치 제어 감속도 제한은 32비트 부동 소수점 수이며, 트래퍼즈 위치 제어 감속도 제한 1은 그 상위 16비트이고, 사다리꼴 위치 제어 감속도 제한 2는 하위 16비트를 구성합니다. 사용자는 레지스터 주소 0x74에 액세스하여 두 레지스터 데이터를 동시에 읽거나 쓰면서 전체 사다리꼴 위치 제어 감속도 제한을 획득하거나 수정할 수 있습니다. 개별 레지스터에 단독으로 액세스할 수는 없습니다.

사다리꼴 위치 제어 감속도 제한 수정/업데이트 요청:

주소	기능 코드	레지스터 주소	레지스터 수	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	1바이트	4바이트	2바이트
XX	10	00 74	00 02	04	D0 D1 D2 D3	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	10	00 74	00 02	XX XX

사다리꼴 위치 제어 감속도 제한 요청 읽기:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
----	-------	---------	---------	-----

1바이트	1바이트	2바이트	2바이트	2바이트
XX	03	00 74	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	03	04	D0 D1 D2 D3	XX XX

여기서 D0, D1, D2, D3는 사다리꼴 위치 제어 감속도 제한을 나타내는 높은 값에서 낮은 값 순으로 배열된 4바이트이며, 32비트 부동소수점은 IEEE 754 표준에 따라 인코딩됩니다(웹사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 인코딩을 테스트할 수 있습니다).

➤ **0x76: 관성 1**

➤ **0x77: 관성 2**

관성은 32비트 부동 소수점 수이며, 관성 1은 상위 16비트, 관성 2는 하위 16비트입니다. 사용자는 레지스터 주소 0x76에 액세스하여 두 레지스터 데이터를 동시에 읽거나 쓰면서 전체 관성을 가져오거나 수정할 수 있습니다. 어느 한 레지스터만 개별적으로 액세스할 수는 없습니다.

관성 수정/업데이트 요청:

주소	기능 코드	레지스터 주소	레지스터 수	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	1바이트	4바이트	2바이트
XX	10	00 76	00 02	04	I0 I1 I2 I3	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1 바이트	2바이트	2바이트	2바이트
XX	10	00 76	00 02	XX XX

관성 읽기 요청:

주소	기능 코드	레지스터 주소	레지스터 수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	03	00 76	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	03	04	I0 I1 I2 I3	XX XX

여기서 I0, I1, I2, I3는 관성 값을 높은 순서부터 낮은 순서로 나타내는 4바이트이며, 32비트 부동소수점은 IEEE 754 표준에

따라 인코딩됩니다(인코딩은 웹사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 테스트할 수 있습니다).

➤ **0x78: 위치 이득 1**

➤ **0x79: 위치 이득 2**

위치 이득은 32비트 부동 소수점 숫자이며, 위치 이득 1은 상위 16비트, 위치 이득 2는 하위 16비트입니다. 사용자는 레지스터 주소 0x78에 액세스하여 두 레지스터 데이터를 동시에 읽거나 쓰면서 전체 위치 이득을 가져오거나 수정할 수 있습니다. 어느 한 레지스터만 개별적으로 액세스할 수는 없습니다.

위치 이득 수정/업데이트 요청:

주소	기능 코드	레지스터 주소	레지스터 수	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	1바이트	4바이트	2바이트
XX	10	00 78	00 02	04	G0 G1 G2 G3	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	10	00 78	00 02	XX XX

위치 이득 요청 읽기:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	03	00 78	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	03	04	G0 G1 G2 G3	XX XX

여기서 G0, G1, G2, G3는 위치 이득을 나타내는 높은 값에서 낮은 값 순으로 배열된 4바이트이며, 32비트 부동소수점은 IEEE 754 표준에 따라 인코딩됩니다(웹사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 인코딩을 테스트할 수 있습니다).

).

➤ **0x7A: 속도 이득 1**

➤ **0x7B: 속도 이득 2**

속도 이득은 32비트 부동 소수점 숫자이며, 속도 이득 1은 상위 16비트, 속도 이득 2는 하위 16비트입니다. 사용자는 레지스터 주소 0x7A에 액세스하여 두 레지스터 데이터를 동시에 읽거나 쓰면서 전체 속도 이득을 가져오거나 수정할 수 있습니다. 어느 한 레지스터만 개별적으로 액세스할 수는 없습니다.

속도 이득 수정/업데이트 요청:

주소	기능 코드	레지스터 주소	레지스터 수	바이트 길이	레지스터 값	체크섬
----	-------	---------	--------	--------	--------	-----

1바이트	1바이트	2바이트	2바이트	1바이트	4바이트	2바이트
XX	10	00 7A	00 02	04	G0 G1 G2 G3	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	10	00 7A	00 02	XX XX

읽기 속도 향상 요청:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	03	00 7A	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	03	04	G0 G1 G2 G3	XX XX

여기서 G0, G1, G2, G3는 속도 이득을 나타내는 높은 값에서 낮은 값 순으로 배열된 4바이트이며, 32비트 부동소수점은 IEEE 754 표준에 따라 인코딩됩니다(웹사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 인코딩을 테스트할 수 있습니다

).

- **0x7C: 속도 적분 이득 1**
- **0x7D: 속도 적분 이득 2**

속도 적분 이득은 32비트 부동 소수점 수이며, 이 중 속도 적분 이득 1은 상위 16비트, 속도 적분 이득 2는 하위 16비트를 차지합니다. 사용자는 레지스터 주소 0x7C에 액세스하여 두 레지스터 데이터를 동시에 읽거나 쓰면서 전체 속도 적분 이득을 가져오거나 수정할 수 있습니다. 어느 한 레지스터에도 개별적으로 액세스할 수 없습니다.

속도 적분 이득 수정/업데이트 요청:

주소	기능 코드	레지스터 주소	레지스터 수	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	1바이트	4바이트	2바이트
XX	10	00 7C	00 02	04	G0 G1 G2 G3	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	10	00 7C	00 02	XX XX

읽기 속도 적분 이득 요청:

주소	기능 코드	레지스터 주소	레지스터 수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	03	00 7C	00 02	XX XX

모터 응답:

주소	기능 코드	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	1바이트	2바이트	2바이트
XX	03	04	G0 G1 G2 G3	XX XX

여기서 G0, G1, G2, G3는 속도 적분 이득을 나타내는 높은 값에서 낮은 값 순으로 배열된 4바이트이며, 32비트 부동소수점은 IEEE 754 표준에 따라 인코딩됩니다(인코딩은 웹사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 테스트할 수 있습니다).

➤ 0x7E: 재시작

이 레지스터에 임의의 값을 쓰면 모터 드라이버가 재시작됩니다. 호스트

요청:

주소	기능 코드	레지스터 주소	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	06	00 7E	XX XX	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	06	00 7E	XX XX	XX XX

➤ **0x7F: 오류 초기화**

이 레지스터에 임의의 값을 쓰면 모든 오류와 예외가 초기화됩니다. 호스트

요청:

주소	기능 코드	레지스터 주소	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	06	00 7F	XX XX	XX XX

모터 반응:

주소	기능 코드	레지스터 주소	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	06	00 7F	XX XX	XX XX

➤ **0x80: 구성 저장**

이 레지스터에 임의의 값을 쓰면, 현재 모든 매개변수가 드라이브의 플래시 메모리에 영구적으로 저장됩니다.

호스트 요청:

주소	기능 코드	레지스터 주소	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	06	00 80	XX XX	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	06	00 80	XX XX	XX XX

➤ **0xA1: 임의 엔드포인트 액세스 엔드포인트 번호**

임의 엔드포인트 액세스는 사용자가 지원되는 모든 매개변수를 읽거나 쓸 수 있음을 의미하며, 이른바 “엔드포인트”는 매개변수를 지칭하며, 각 매개변수에는 고유한 엔드포인트 번호가 있습니다. 사용자가 특정 매개변수를 읽거나 쓰려고 할 때, 엔드포인트 설명 파일을 통해 해당 매개변수의 엔드포인트 번호를 확인한 후, 이를 홀드 레지스터 0xA1에 기록합니다. 그런 다음 입력 레지스터 0xA1을 읽어 해당 매개변수 값을 얻거나, 홀드 레지스터 0xA2에 기록하여 해당 매개변수 값을 업데이트할 수 있습니다.

엔드포인트 번호는 16비트 부호 없는 정수입니다. 모든 매개변수 및 인터페이스 함수에 해당하는 엔드포인트 번호가 포함된

JSON 파일을 다운로드하십시오:

- ✓ 버전 0.5.13: https://bl.cyberbeast.cn/actuator/endpoints_0.5.13.json
- ✓ 버전 0.5.14: https://bl.cyberbeast.cn/actuator/endpoints_0.5.14.json

임의의 엔드포인트 번호로 요청 전송:

주소	기능 코드	레지스터 주소	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	06	00 A1	E0 E1	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	06	00 A1	E0 E1	XX XX

여기서 E0 E1은 16비트 엔드포인트 번호를 높은 비트부터 낮은 비트 순서로 나타낸 2바이트입니다.

- **0xA2: 임의의 엔드포인트 액세스, 값 1 쓰기**
- **0xA3: 임의 엔드포인트 액세스 쓰기 값 2**

임의 엔드포인트 액세스 쓰기 값은 32비트 수이며, 16비트 정수, 32비트 정수 또는 32비트 부동 소수점 수일 수 있습니다.

임의 엔드포인트 액세스 쓰기 값 1은 상위 16비트, 임의 엔드포인트 액세스 쓰기 값 2는 하위 16비트입니다. 사용자는 레지스터 주소 0xA2에 액세스하여 두 레지스터 데이터를 동시에 쓰기만 할 수 있으며, 이를 통해 엔드포인트 값(매개변수 값)을 업데이트할 수 있습니다. 어느 한 레지스터에도 개별적으로 액세스할 수 없습니다.

쓰기 요청:

주소	기능 코드	레지스터 주소	레지스터 수	바이트 길이	레지스터 값	체크섬
1바이트	1바이트	2바이트	2바이트	1바이트	4바이트	2바이트
XX	10	00 A2	00 02	04	V0 V1 V2 V3	XX XX

모터 응답:

주소	기능 코드	레지스터 주소	레지스터 개수	체크섬
1바이트	1바이트	2바이트	2바이트	2바이트
XX	10	00 A2	00 02	XX XX

여기서 V0, V1, V2, V3는 쓰기 값의 상위에서 하위로 정렬된 4바이트이며, 해당 엔드포인트가 32비트 부동소수점 수인 경우 IEEE 754 표준에 따라 인코딩됩니다(인코딩은 웹사이트 <https://www.h-schmidt.net/FloatConverter/IEEE754.html>에서 테스트할 수 있습니다).

4.3.4 Modbus 프로토콜 실전

모터의 주소가 1이라고 가정합니다.

1) 실전: 속도 제어

프레임 전송 순서는 다음과 같습니다:

프레임 데이터	레지스터	설명
01 06 00 64 00 02 XX XX	레지스터 0x64 유지	제어 모드를 2(속도 제어)로 설정
01 06 00 65 00 02 XX XX	레지스터 0x65 유지	입력 모드를 2(속도 경사)로 설정
01 06 00 63 00 08 XX XX	레지스터 0x63 유지	모터 상태를 8(페루프 작동)으로 변경).
01 10 00 68 00 02 04 40 13 33 33 XX XX	레지스터 0x68 유지	목표 속도를 2.3r/s로 설정하고, 32 비트 부동 소수점 수 40 13 33 33으로 변환.

2) 실전: 위치 제어

프레임 전송 시퀀스는 다음과 같습니다:

프레임 데이터	레지스터	설명
01 06 00 64 00 03 XX XX	레지스터 0x64 유지	제어 모드를 3(위치 제어)로 설정
01 06 00 65 00 03 XX XX	레지스터 0x65 유지	입력 모드를 3(위치 필터링)으로 설정
01 06 00 63 00 08 XX XX	레지스터 0x63 유지	모터 상태를 8(페루프 작동)으로 변경).
01 10 00 68 00 02 04 40 49 0F F9 XX XX	레지스터 0x68 유지	목표 속도를 3.1416r로 설정하고, 이를 32비트 부동 소수점 수 40 49 0F F9로 변환.

4.4 PWM 입력 제어

이 기능을 사용하기 전에 하드웨어 버전이 3.10 이상인지 확인하십시오.

이 기능은 표준 RC PWM 신호(1ms-2ms 펄스 폭, 50Hz)를 통해 위치, 속도 또는 토크와 같은 다양한 쓰기 가능한 매개 변수를 제어할 수 있습니다. GPIO 핀을 PWM 모드로 구성하고 매핑 관계를 설정하면 RC 수신기 또는 기타 PWM 컨트롤러를 통해 모터를 직접 제어할 수 있습니다.

4.4.1 배선 설명

PWM 신호선은 2.4.1절에 설명된 CANH 위치에 연결합니다. 이 인터페이스는 5V 이하의 PWM 신호를 지원하며, PWM 신호와 모터 드라이버의 접지를 공통으로 연결해야 합니다.

4.4.2 소프트웨어 설정

- 통신 인터페이스를 PWM으로 전환:

```
odrv0.config.comm_intf_mux = 3
```

모터 엘프(Motor Elf)에서 통신 매개변수 설정 시 물리적 인터페이스를 전환할 수도 있습니다. 모터 매개변수에서 해당 항목을 찾아 설정하십시오.



- GPIO를 PWM 모드로 설정

```
odrv0.config.gpio1_mode = GPIO_MODE_PWM
```

- PWM 매핑 제어 모드 설정

속도/위치/토크 모드로 매핑할 수 있으며, 필요에 따라 다음 세 가지 모드 중 하나를 구성하면 됩니다. 속도 모드

모터 제어 모드를 속도 슬로프 모드로 설정하십시오(control_mode = 2, input_mode = 2)

```
odrv0.config.gpio1_pwm_mapping.endpoint = odrv0.axis0.controller._input_vel_property
```

위치 모드

모터 제어 모드를 사다리꼴 곡선 모드로 설정하십시오(control_mode = 3, input_mode = 5).

```
odrv0.config.gpio1_pwm_mapping.endpoint = odrv0.axis0.controller._input_pos_property
```

토크 모드

모터 제어 모드를 속도 경사 모드로 설정하십시오(control_mode = 1, input_mode = 6).

```
odrv0.config.gpio1_pwm_mapping.endpoint = odrv0.axis0.controller._input_torque_property
```

➤ PWM 매핑 매개변수 설정

```
odrv0.config.gpio1_pwm_mapping.min = -8.0
odrv0.config.gpio1_pwm_mapping.max = 8.0
```

min과 max는 PWM 펄스 폭(1ms~2ms)이 매핑되는 제어량 범위를 정의합니다. 드라이버 내부에서는 펄스 폭 값을 [min, max] 범위 내의 실수값으로 선형 매핑한 후, 사용자가 설정한 엔드포인트 제어 목표값으로 기록합니다. min과 max는 필요에 따라 임의의 값으로 설정할 수 있습니다.

➤ 구성 저장(선택 사항)

```
odrv0.save_configuration()
```

위의 설정을 완료한 후, 모터를 펄스로 전환하면 PWM 제어를 수행할 수 있습니다.

4.5 펄스/방향 제어

이 기능을 사용하기 전에 하드웨어 버전이 3.10 이상인지 확인하십시오. 이 기능은 리미트 스위치 기능과 충돌합니다. 이 기능은 드라이버를 제어하는 가장 간단한 방법이지만, 동시에 가장 원시적이고 취약한 방법이기도 합니다. 따라서 특별한 경우가 아닌 한 이 제어 방식을 사용하지 마십시오.

이 기능은 기존의 Step/Dir(펄스/방향) 인터페이스를 통해 브러시리스 모터를 제어하여 일반 스테핑 모터 드라이버와 호환되는 제어 방식을 구현합니다. 이 모드에서는:

Step(펄스) 신호: 각 상승 에지마다 모터가 미리 설정된 스텝 거리(steps_per_circular_range에 의해 결정됨)만큼 이동합니다. 권장 최대 Step 주파수: 50kHz

Dir(방향) 신호: 레벨에 따라 이동 방향이 결정됩니다(고전압: 시계 방향 / 저전압: 반시계 방향)

4.5.1 배선 설명

Step(펄스 신호) 신호선은 2.4.1절에 설명된 CANH 위치에 연결하고, Dir(방향 신호) 신호선은 2.4.1절에 설명된 CANL 위치에 연결하며, 본 인터페이스는 5V 이하의 펄스 신호를 지원합니다. Step 및 Dir 신호가 모터 드라이버와 공통 접지를 이루도록 하십시오. 그렇지 않으면 신호 인식에 실패할 수 있습니다.

4.5.2 소프트웨어 설정

➤ 통신 인터페이스를 Step/Dir로 전환:

```
odrv0.config.comm_intf_mux = 2
```

모터 엘프(Motor Elf)에서도 통신 매개변수에서 물리적 인터페이스를 전환할 수 있습니다. 모터 매개변수를 열어 이 항목을 찾아주세요.



- GPIO를 Step/Dir 모드로 설정

```
odrv0.axis0.config.step_gpio_pin = 6
odrv0.axis0.config.dir_gpio_pin = 7
```

- Step/Dir 핀 구성

```
odrv0.config.gpio6_mode = GPIO_MODE_DIGITAL_PULL_UP
odrv0.config.gpio7_mode = GPIO_MODE_DIGITAL_PULL_UP
```

- Step/Dir 활성화

```
odrv0.axis0.config.enable_step_dir = True
```

펄스/방향 제어에서 벗어나려면 enable_step_dir을 False로 설정한 후, 유휴 상태로 전환한 다음 다시 페루프 제어로 복귀해야 합니다.

- Circular Setpoint 활성화 (선택 사항)

광범위한 목표 위치에서 발생하는 정밀도 드리프트 문제를 해결하기 위해 ODrive는 서클러 세트포인트(Circular Setpoint) 기능을 지원합니다. 서클러 세트포인트 모드를 사용하면 부동 소수점 오차를 제어 가능한 범위 내로 유지할 수 있습니다. 모터가 지정된 서클러 범위를 벗어나도록 명령을 받으면, 위치 세트포인트가 자동으로 회전하여 범위 내에 유지됩니다.

컨트롤러의 서클러 세트포인트 모드 활성화

```
odrv0.axis0.controller.config.circular_setpoints = True
```

원형 위치 범위 설정(단위: turns). 설정값은 0~이 범위 내에서 제한되며, 이 범위를 초과하면

회전

```
# 128회전을 한 바퀴로 간주
odrv0.axis0.controller.config.circular_setpoint_range = 128.0
```

원형 위치 범위 내에서 필요한 STEP 펄스 수에 따라 1 스텝당 변위량이 결정됩니다

```
# 1회전당 1024개의 스텝 odrv0.axis0.controller.config.steps_per_circular_range = 128 * 1024
```

➤ 구성 저장 (선택 사항)

```
odrv0.save_configuration()
```

위의 설정을 완료한 후, 모터를 **위치** 모드에서 페루프로 전환하면 PWM 제어를 수행할 수 있습니다.

4.6 SDK

4.6.1 C/C++ SDK

CAN Simple 프로토콜을 기반으로 하며, 다음 C/C++ SDK를 다운로드하여 통합할 수 있습니다:

<https://gitee.com/cyberbeast/mwmotorsdk>. 상기 오픈소스 웹사이트의 상세 테스트 예제를 참조하여 통합 테스트를 진행하십시오.

4.6.2 Python SDK

먼저 3.1절의 단계를 참조하여 odrivetool을 설치하십시오(pip install --upgrade odrive). 3.1.8절을 참조하여, 해당 절에 설명된 모든 명령어를 활용하여 Python 개발을 진행할 수 있습니다.

다음은 세 가지 예시입니다:

1) 실전: 전원 공급 시 보정

```
import odrive import
time

odrv0 = odrive.find_any()
odrive.utils.dump_errors(odrv0) odrv0.clear_errors()
odrv0.axis0.requested_state = odrive.utils.AxisState.MOTOR_CALIBRATION time.sleep(5)
while (odrv0.axis0.current_state != 1):
    time.sleep(0.5) odrive.utils.dump_errors(odrv0)
odrv0.axis0.requested_state = odrive.utils.AxisState.ENCODER_OFFSET_CALIBRATION
time.sleep(6)
while (odrv0.axis0.current_state!=1): time.sleep(0.5)
odrive.utils.dump_errors(odrv0)
odrv0.axis0.motor.config.pre_calibrated=1
odrv0.axis0.encoder.config.pre_calibrated=1
odrv0.save_configuration()
```

2) 실전: 속도 제어

```
import odrive import
time

odrv0 = odrive.find_any()
odrv0.axis0.controller.config.control_mode=odrive.utils.ControlMode.VELOCITY_CONTROL
odrv0.axis0.controller.config.input_mode=odrive.utils.InputMode.VEL_RAMP
odrv0.axis0.controller.config.vel_ramp_rate=50
odrv0.axis0.requested_state=odrive.utils.AxisState.CLOSED_LOOP_CONTROL
odrv0.axis0.controller.input_vel=15
odrive.utils.dump_errors(odrv0) time.sleep(5)
odrv0.axis0.controller.input_vel=0
```

3) 실전: 위치 제어

```
import odrive

odrv0 = odrive.find_any()
odrv0.axis0.controller.config.control_mode=odrive.utils.ControlMode.POSITION_CONTROL
odrv0.axis0.controller.config.input_mode=odrive.utils.InputMode.POSITIVE
odrv0.axis0.requested_state=odrive.utils.AxisState.CLOSED_LOOP_CONTROL
odrv0.axis0.controller.input_pos=10
```

4) 실전: 데이터 수집

사용자는 연구 개발 및 통합 과정에서 모터 작동 데이터를 수집해야 할 경우가 많습니다. 예를 들어 전압 및 전류 변화, 위치 및 속도 변화 등을 수집하는 경우입니다. Python SDK는 강력한 데이터 수집 기능을 통합하고 있어 간단한 스크립트로 방대한 양의 작동 데이터를 수집할 수 있으며, 이를 통해 연구 개발 및 통합 작업을 더욱 간편하게 만들 수 있습니다.

아래 코드는 앞의 위치 제어 예제를 바탕으로 실시간 위치 및 전류 데이터 수집 기능을 추가하고, 데이터를 CSV 파일로 저장하는 예시입니다.

```
import odrive import
numpy as np

odrv0 = odrive.find_any() cap
=
odrive.utils.BulkCapture(lambda:[odrv0.axis0.motor.current_control.lq_measured,
odrv0.axis0.encoder.pos_estimate], data_rate=500, duration=2.5)
odrv0.axis0.controller.config.control_mode=odrive.utils.ControlMode.POSITION_CONTROL
odrv0.axis0.controller.config.input_mode = odrive.utils.InputMode.POS_FI LTER
odrv0.axis0.requested_state=odrive.utils.AxisState.CLOSED_LOOP_CONTROL
odrv0.axis0.controller.input_pos=10 np.savetxt("d:/test.csv",cap.data,delimiter=',')
```

여기서 BulkCapture를 수행하는 문장에서, data_rate는 샘플링 주파수를 나타내며 단위는 hz이고, duration은 샘플링 시간을 나타내며 단위는 초입니다. 여기서 lambda 표현식에는 데이터 분석을 용이하게 하기 위해 어떤 수학적 연산이라도 삽입할 수 있습니다.

4.6.3 Arduino SDK

사용자는 Arduino를 사용하여 CAN 버스를 통해 모터를 제어할 수 있으며, 저수준 프로토콜은 4.1절에 설명된 바와 같습니다.

호환 가능한 하드웨어/

라이브러리:

- ✓ CAN 인터페이스가 내장된 Arduino(예: Arduino UNO R4 Minima, Arduino UNO R4 WIFI 등)
- ✓ CAN 인터페이스가 내장된 Teensy 개발 보드는 호환되는 FlexCAN_T4 라이브러리(Teensy 4.0 및 Teensy 4.1)를 사용하여 액세스할 수 있습니다
- ✓ 기타 Arduino 호환 보드는 MCP2515 기반 CAN 확장 보드를 사용하여 접근할 수 있습니다. 다

음은 모터가 Arduino의 위치 제어 명령에 응답하도록 구성하는 방법을 보여주는 예시입니다:

➤ 모터 구성

3.1.3의 기본 구성 외에도 제어 대역폭을 20rad/s로 설정합니다(Arduino Uno의 전송 속도가 제한적이므로 제어 대역폭을 너무 높게 설정할 필요는 없습니다. 더 빠른 Arduino를 사용하는 경우 이 대역폭 값을 높일 수 있습니다):

➤ CAN 구성

다음 방법에 따라 CAN을 구성합니다:

➤ ODriveArduino 라이브러리 설치

다음 단계에 따라 ODriveArduino 라이브러리를 설치합니다(사용자가 이미 Arduino IDE를 설치했다고 가정함):

- 1) Arduino IDE를 엽니다

- 2) Sketch -> Include Library -> Manage Libraries
 - 3) "ODriveArduino"를 입력하여 검색
 - 4) 검색된 ODriveArduino 라이브러리를 클릭하여 설치하세요
- Arduino 소스 코드

```
#include <Arduino.h> #include
"ODriveCAN.h"

// 이 예제에 대한 설명은 다음에서 확인할 수 있습니다:
// https://docs.odriverobotics.com/v/latest/guides/arduino-can-guide.html

/* 예제 스케치 구성 ----- */

// CAN 버스 전송 속도. 버스에 연결된 모든 장치의 속도가 일치하도록 설정하십시오 #define
CAN_BAUDRATE 500000

// odrv0용 ODrive 노드 ID #define
ODRV0_NODE_ID 0

// 사용 중인 하드웨어에 해당하는 줄의 주석을 해제하십시오.
// 하드웨어 설정에 맞게 세부 사항을 조정하려면 "보드별 설정"도 참조하십시오.

// #define IS_TEENSY_BUILTIN // CAN 인터페이스가 내장된 Teensy 보드(예: Teensy 4.1). 사용할 인
터페이스를 선택하려면 아래를 참조하십시오.

// #define IS_ARDUINO_BUILTIN // CAN 인터페이스가 내장된 Arduino 보드(예: Arduino Uno R4
Minima)

// #define IS_MCP2515 // 외부 MCP2515 기반 확장 모듈이 장착된 모든 보드. 모듈 구성 방법은 아래를 참조
하십시오.

-----

/* 보드별 포함 파일 */

#if defined(IS_TEENSY_BUILTIN) + defined(IS_ARDUINO_BUILTIN) + defined(IS_MCP2515) != 1
#warning "이 파일의 맨 위에서 하드웨어 옵션을 정확히 하나만 선택하십시오."

#if CAN_HOWMANY > 0 || CANFD_HOWMANY > 0
#define IS_ARDUINO_BUILTIN
#warning "이것이 HardwareCAN을 사용하는 것으로 추정합
니다" #else

#error "하드웨어 버전을 추측할 수 없습니다"
```



```

#endif #endif

#ifdef IS_ARDUINO_BUILTIN
// https://github.com/arduino/ArduinoCore-API/blob/master/api/HardwareCAN.h 참조
// 및 https://github.com/arduino/ArduinoCore-
renesas/tree/main/libraries/Arduino_CAN

#include <Arduino_CAN.h> #include
<ODriveHardwareCAN.hpp> #endif //
IS_ARDUINO_BUILTIN

#ifdef IS_MCP2515
// https://github.com/sandeepmistry/arduino-CAN/ 참조 #include
"MCP2515.h"
#include "ODriveMCPCAN.hpp"
#endif // IS_MCP2515

#ifdef IS_TEENSY_BUILTIN
// https://github.com/tonton81/FlexCAN_T4 참조
// https://github.com/tonton81/FlexCAN_T4.git를 /src로 복사 #include
<FlexCAN_T4.h>
#include "ODriveFlexCAN.hpp"
struct ODriveStatus; // Teensy 컴파일 오류를 방지하기 위한 임시 조치 #endif //
IS_TEENSY_BUILTIN

/* 보드별 설정 */

/* Teensy */
-----

#ifdef IS_TEENSY_BUILTIN

FlexCAN_T4<CAN1, RX_SIZE_256, TX_SIZE_16> can_intf;

bool setupCan() {
    can_intf.begin();
    can_intf.setBaudRate(CAN_BAUDRATE);
    can_intf.setMaxMB(16);
    can_intf.enableFIFO();

```

```

    can_intf.enableFIFOInterrupt();
    can_intf.onReceive(onCanMessage); return
    true;
}

#endif // IS_TEENSY_BUILTIN

/* MCP2515 기반 확장 모듈 -*/ #ifdef

IS_MCP2515

MCP2515Class& can_intf = CAN;

// MCP2515에 사용되는 칩 선택 핀 #define
MCP2515_CS 10

// MCP2515에 사용되는 인터럽트 핀
// 참고: 모든 아두이노 핀이 인터럽트 지원 핀은 아니므로, 사용 중인 보드의 설명서를 확인하세요!
#define MCP2515_INT 2

// MCP2515 브레이크아웃 보드의 수정 발진기 주파수.
// 일반적인 값은 16 MHz, 12 MHz, 8 MHz입니다. #define
MCP2515_CLK_HZ 8000000

static inline void receiveCallback(int packet_size) { if
(packet_size > 8) {
    return; // 지원되지 않음
}
    CanMsg msg = {.id = (unsigned int)CAN.packetId(), .len = (uint8_t)packet_size};
    CAN.readBytes(msg.buffer, packet_size);
    onCanMessage(msg);
}

bool setupCan() {
    // CAN 버스 인터페이스 구성 및 초기화 CAN.setPins(MCP2515_CS, MCP2515_INT);
    CAN.setClockFrequency(MCP2515_CLK_HZ); if
    (!CAN.begin(CAN_BAUDRATE)) {
        return false;
    }
}

```

```

    CAN.onReceive(receiveCallback);
    return true;
}

#endif // IS_MCP2515

/* 내장 CAN이 있는 Arduino */ #ifdef

IS_ARDUINO_BUILTIN

HardwareCAN& can_intf = CAN;

bool setupCan() {
    return can_intf.begin((CanBitRate)CAN_BAUDRATE);
}

#endif

/* 예제 스케치 */
-----

// ODrive 객체 생성

ODriveCAN odrv0(wrap_can_intf(can_intf), ODRV0_NODE_ID); // 표준 CAN 메시지 ID

ODriveCAN* odrives[] = {&odrv0}; // 모든 ODriveCAN 인스턴스가 여기에 포함되었는지 확인

struct ODriveUserData { Heartbeat_msg_t
    last_heartbeat; bool received_heartbeat =
    false;
    Get_Encoder_Estimates_msg_t last_feedback; bool
    received_feedback = false;
};

// 각 ODrive에 대해 애플리케이션별 사용자 데이터를 저장합니다. ODriveUserData
odrv0_user_data;

// ODrive에서 하트비트 메시지가 도착할 때마다 호출됨 void
onHeartbeat(Heartbeat_msg_t& msg, void* user_data) {
    ODriveUserData* odrv_user_data = static_cast<ODriveUserData*>(user_data); odrv_user_data-
    >last_heartbeat = msg;
    odrv_user_data->received_heartbeat = true;
}

```

```
// ODrive에서 피드백 메시지가 도착할 때마다 호출됨
void onFeedback(Get_Encoder_Estimates_msg_t& msg, void* user_data) { ODriveUserData*
    odrv_user_data = static_cast<ODriveUserData*>(user_data); odrv_user_data->last_feedback =
    msg;
    odrv_user_data->received_feedback = true;
}

// CAN 버스에 도착하는 모든 메시지에 대해 호출됨 void
onCanMessage(const CanMsg& msg) {
    for (auto odrive: odrives) {
        onReceive(msg, *odrive);
    }
}

void setup() {
    Serial.begin(115200);

    // PC 측에서 시리얼 포트가 열릴 때까지 최대 3초 동안 대기합니다.
    // PC가 연결되지 않더라도 계속 진행합니다. for (int
    i = 0; i < 30 && !Serial; ++i) {
        delay(100);
    }
    delay(200);

    Serial.println("ODriveCAN 데모 시작");

    // 하트비트 및 인코더 피드백 메시지에 대한 콜백 등록 odrv0.onFeedback(onFeedback,
    &odrv0_user_data); odrv0.onStatus(onHeartbeat, &odrv0_user_data);

    // CAN 버스 인터페이스를 구성하고 초기화합니다. 이 함수는
    // 사용 중인 하드웨어 및 CAN 스택에 따라 다릅니다. if
    (!setupCan()) {
        Serial.println("CAN 초기화 실패: 리셋 필요"); while (true); // 무한
        루프
    }

    Serial.println("ODrive 대기 중..."); while
    (!odrv0_user_data.received_heartbeat) {
        pumpEvents(can_intf);
        delay(100);
    }
```

```

Serial.println("ODrive를 찾았습니다");

// 버스 전압 및 전류 요청 (1초 타임아웃) Serial.println("버스 전압 및 전류 읽기 시도
중"); Get_Bus_Voltage_Current_msg_t vbus;
if (!odrv0.request(vbus, 1)) { Serial.println("vbus
요청 실패!"); while (true); // 무한 루프
}

Serial.print("DC 전압 [V]: ");
Serial.println(vbus.Bus_Voltage);
Serial.print("DC 전류 [A]: ");
Serial.println(vbus.Bus_Current);

Serial.println("페루프 제어 활성화 중..."); while
(odrv0_user_data.last_heartbeat.Axis_State !=
ODriveAxisState::AXIS_STATE_CLOSED_LOOP_CONTROL) {
    odrv0.clearErrors();
    delay(1); odrv0.setState(ODriveAxisState::AXIS_STATE_CLOSED_LOOP_CONTROL);

    // 150ms 동안 이벤트를 펌핑합니다. 이 지연은 두 가지 이유로 필요합니다;
    // 1. DC 전원 공급 중단과 같은 오류 상황이 발생하면 ODrive는
    // 잠시 CLOSED_LOOP_CONTROL 상태로 진입하려고 시도할 수 있으므로, 이를 신뢰할 수 없습니다
    // 첫 번째 하트비트 응답을 신뢰할 수 없으므로, 최소 두 번의 응답을 수신해야 합니다
    // 하트비트(기본 간격 100ms)를 수신해야 합니다.
    // 2. 버스가 혼잡한 경우 setState 명령이 통과되지 않을 수 있습니다
    // 즉시 전달되지 않고 지연될 수 있습니다.
    for (int i = 0; i < 15; ++i) {
        delay(10);
        pumpEvents(can_intf);
    }
}

Serial.println("ODrive 실행 중!");
}

void loop() {
    pumpEvents(can_intf); // 일부 플랫폼에서는 수신되는 피드백 CAN 메시지를 처리하기 위해 이 코드가
    필요합니다

```

```

float SINE_PERIOD = 2.0f; // 위치 명령 사인파의 주기(초)

float t = 0.001 * millis();

float phase = t * (TWO_PI / SINE_PERIOD);

odrv0.setPosition(
    sin(phase), // 위치

    cos(phase) * (TWO_PI / SINE_PERIOD) // 속도 피드포워드 (선택 사항)
);

// Serial Plotter용 위치 및 속도 출력 if
(odrv0_user_data.received_feedback) {
    Get_Encoder_Estimates_msg_t feedback = odrv0_user_data.last_feedback;
    odrv0_user_data.received_feedback = false;
    Serial.print("odrv0-pos:");
    Serial.print(feedback.Pos_Estimate);
    Serial.print(","); Serial.print("odrv0-
    vel:"); Serial.println(feedback.Vel_Estimate);
}
}

```

4.6.4 ROS SDK

다음 단계는 Ubuntu 23.04 및 ROS2 Iron에서 테스트 및 검증되었으나, MAC 및 Windows 플랫폼은 지원하지 않으며, 다른 ROS2 버전에서는 검증되지 않았으므로 수정 후 사용해야 합니다.

4.6.4.1 odrive_can 패키지 설치

1. 새로운 ROS2 워크스페이스 생성 (<https://docs.ros.org/en/iron/index.html> 참조)
2. git clone https://github.com/odriverobotics/odrive_can 명령어를 사용하여 코드를 위 워크스페이스 디렉토리의 src 디렉토리에 다운로드합니다
3. 터미널에서 워크스페이스의 루트 디렉토리로 이동한 후 다음 명령을 실행합니다:

```
colcon build --packages-select odrive_can
```

4. 실행 전 환경 준비:

```
source ./install/setup.bash
```

5. 예제 노드 실행:

```
ros2 launch odrive_can example_launch.yaml
```

4.6.4.2 서비스 호출 및 메시지 확인

위 odrive_can_node 노드가 odrive_axis0 네임스페이스에서 실행된다고 가정합니다(./launch/example_launch.yaml에서 설정 가능). 4.4.4.1절의 노드가 실행되면, 다음과 같이 게시된 토픽 메시지를 확인할 수 있습니다:

```
ros2 topic echo /odrive_axis0/controller_status ros2 topic  
echo /odrive_axis0/odrive_status
```

그리고 공개된 서비스 인터페이스를 호출하면, 다음과 같이 호출하여 모터 보정을 시작할 수 있습니다:

```
ros2 service call /odrive_axis0/request_axis_state  
/odrive_can/srv/AxisState "{axis_requested_state: 4}"
```

5 자주 묻는 질문 및 오류 코드 (업데이트 예정)

5.6 자주 묻는 질문(FAQ)

5.7 오류 코드

오류 범주	오류 코드	odrivetool 표시	설명
시스템 오류	0x00000002	DC_BUS_UNDER_VOLTAGE	전원 전압이 너무 낮음
	0x00000004	DC_BUS_OVER_VOLTAGE	전원 전압 과다
	0x00000008	DC_BUS_OVER_REGEN_CURRENT	전원 역방향(충전) 전류 과다 높음
	0x00000010	DC_BUS_OVER_CURRENT	전원 정방향(방전) 전류 과다 과
드라이브 이상	0x00000001	INVALID_STATE	드라이버 상태 오류
	0x00000040	MOTOR_FAILED	모터 오류
	0x00000100	ENCODER_FAILED	인코더 오류
	0x00000200	CONTROLLER_FAILED	컨트롤러 오류
	0x00001000	MIN_ENDSTOP_PRESSED	하한 리미트 작동
	0x00002000	MAX_ENDSTOP_PRESSED	상한 리미트 트리거
	0x00004000	ESTOP_REQUESTED	비상 정지
	0x00020000	HOMING_WITHOUT_ENDSTOP	제로 위치 복귀, 그러나 리미트 스위치 없음
	0x00080000	알 수 없는 위치	위치 정보 없음
모터 이상	0x00000001	PHASE_RESISTANCE_OUT_OF_RANGE	상간 저항이 정상 범위를 벗어남
	0x00000002	PHASE_INDUCTANCE_OUT_OF_RANGE	상간 인덕턴스가 정상 범위를 벗어남
	0x00000010	CONTROL_DEADLINE_MISSED	FOC 주파수가 너무 높음
	0x00000080	MODULATION_MAGNITUDE	SVM 변조 이상

	0x00000400	전류 감지 포화	상 전류 포화
	0x00001000	CURRENT_LIMIT_VIOLATION	모터 전류 과다
	0x00020000	MOTOR_THERMISTOR_OVER_TEMP	모터 과열
	0x00040000	FET_서미스터_과열	드라이버 과열
	0x00080000	TIMER_UPDATE_MISSED	FOC 처리 지연
	0x00100000	전류 측정값 미사용	상 전류 샘플링 손실
	0x00200000	CONTROLLER_FAILED	제어 오류
	0x00400000	I_BUS_OUT_OF_RANGE	버스 전류 초과
	0x00800000	제동 저항기 해제	브레이크 저항 구동 이상
	0x01000000	SYSTEM_LEVEL	시스템 레벨 오류
	0x02000000	BAD_TIMING	상 전류 샘플링 지연
	0x04000000	UNKNOWN_PHASE_ESTIMATE	모터 위치 미확인
	0x08000000	UNKNOWN_PHASE_VEL	모터 속도 알 수 없음
	0x10000000	UNKNOWN_TORQUE	토크 알 수 없음
	0x20000000	UNKNOWN_CURRENT_COMMAND	토크 제어 알 수 없음
	0x40000000	UNKNOWN_CURRENT_MEASUREMENT	전류 샘플링 값 알 수 없음
	0x80000000	UNKNOWN_VBUS_VOLTAGE	전압 샘플링 값 알 수 없음
	0x100000000	UNKNOWN_VOLTAGE_COMMAND	전압 제어 알 수 없음
	0x200000000	UNKNOWN_GAINS	전류 루프 이득 알 수 없음
	0x400000000	CONTROLLER_INITIALIZING	컨트롤러 초기화 오류
	0x800000000	UNBALANCED_PHASES	3상 불균형
제어 오류	0x00000001	과속	과속
	0x00000002	INVALID_INPUT_MODE	제어 입력 모드 오류
	0x00000004	UNSTABLE_GAIN	위상 잠금 루프 이득 불안정
	0x00000020	INVALID_ESTIMATE	위치/속도 불안정
	0x00000080	SPINOUT_DETECTED	기계적 출력과 전기적 출력이 일치 하지 않음(인코더 보정 오류, 또는 자석 불안정)
인코 더 이 상	0x00000001	UNSTABLE_GAIN	인코더 대역폭 과대
	0x00000002	CPR_POLEPAIRE_MISMATCH	CPR 및 극쌍 수 불일치
	0x00000004	NO_RESPONSE	인코더 응답 없음
	0x00000400	SEC_ENC_COM_FAIL	두 번째 인코더 통신 오류